



آمار و احتمال مهندسی

Engineering Probability & Statistics

Lecture Notes ■ جزوه درسی

Complete Lecture Notes

Modules 1-16

جزوه کامل درس - مائولهای ۱ تا ۱۶

Instructor: Amin Azari

استاد: امین آذری github.com/AminAzari

نیمسال پاییز ۱۴۰۵-۱۴۰۶

Fall Semester 2026-2027

Contents

Module 1	Probability	11
1.1	Introduction: Why Probability in Engineering?	14
1.2	Random Experiments	15
1.3	Sample Spaces	16
1.4	Events	17
1.5	Probability Axioms	18
1.6	Probability Rules	19
1.7	Conditional Probability	20
1.8	Independence	22
1.9	Bayes' Theorem	24
1.10	Total Probability Theorem	26
1.11	MATLAB Examples	27
1.12	Summary: Key Distinctions	31
1.13	Practice Problems	32
1.14	Key Takeaways	35
Module 2	One Random Variable and Its Functions	36
2.1	Introduction: Why Random Variables?	39
2.2	Definition of a Random Variable	40
2.3	Discrete Random Variables	41
2.4	Continuous Random Variables	42
2.5	Probability Mass Function (PMF)	43

2.6	Probability Density Function (PDF)	44
2.7	Cumulative Distribution Function (CDF)	45
2.8	KEY DISTINCTION: Mass vs. Density vs. Cumulative	47
2.9	Transformations of Random Variables	49
2.10	MATLAB Examples	51
2.11	Practice Problems	55
2.12	Key Takeaways	58
Module 3	Important Probability Distributions	59
3.1	Introduction: Choosing the Right Distribution	62
3.2	Bernoulli Distribution	63
3.3	Binomial Distribution	65
3.4	Geometric Distribution	67
3.5	Negative Binomial Distribution	69
3.6	Poisson Distribution	71
3.7	Uniform Distribution (Continuous)	74
3.8	Exponential Distribution	76
3.9	Gaussian (Normal) Distribution	79
3.10	Gamma Distribution	83
3.11	Rayleigh Distribution	85
3.12	Chi-Square Distribution	87
3.13	Distribution Selection Guide	89
3.14	Summary Table	91
3.15	Practice Problems	92

Module 4	Expectation and Moments	93
4.1	Introduction: Why Summary Statistics?	96
4.2	Expected Value (Mean)	97
4.3	Expected Value of a Function: $E[g(X)]$	98
4.4	Variance	99
4.5	Standard Deviation	100
4.6	Properties of Expectation	101
4.7	Properties of Variance	102
4.8	Higher-Order Moments	103
4.9	Moment Generating Function (MGF)	104
4.10	Engineering Applications of Moments	106
4.11	MATLAB Examples	107
4.12	Practice Problems	110
4.13	Key Takeaways	112
Module 5	Important Random-Variable Functions	113
5.1	Introduction: Why Transform Random Variables?	116
5.2	Linear Transformations: $Y = aX + b$	117
5.3	General Monotonic Transformations	118
5.4	Nonlinear Transformations: Non-Monotonic Case	119
5.5	The Jacobian Method	120
5.6	Engineering Application: Rayleigh from Gaussian	121
5.7	MATLAB Examples	122
5.8	Practice Problems	125

Module 6 Two Random Variables	127
6.1 Introduction: Why Study Joint Distributions?	130
6.2 Joint PMF (Discrete Case)	131
6.3 Joint PDF (Continuous Case)	132
6.4 Joint CDF	133
6.5 Marginal Distributions	134
6.6 Conditional Distributions	135
6.7 Independence of Two Random Variables	136
6.8 Conditional Expectation	137
6.9 MATLAB Examples	138
6.10 Practice Problems	140
Module 7 Functions of Two Random Variables	141
7.1 Introduction	144
7.2 Sum of Two Random Variables: $Z = X + Y$	145
7.3 Difference: $Z = X - Y$	146
7.4 Product: $Z = XY$	147
7.5 Ratio: $Z = X/Y$	148
7.6 Maximum: $Z = \max(X, Y)$	149
7.7 Minimum: $Z = \min(X, Y)$	150
7.8 General Bivariate Transformation (Jacobian Method)	151
7.9 MATLAB Examples	152
7.10 Practice Problems	154

Module 8	Conditional Expectation and Variance	155
8.1	Introduction	158
8.2	Conditional Expectation $E[X Y=y]$	159
8.3	Conditional Variance $\text{Var}(X Y=y)$	160
8.4	Law of Total Expectation (Tower Property)	161
8.5	Law of Total Variance	162
8.6	MATLAB Examples	163
8.7	Practice Problems	165
Module 9	Covariance and Correlation	166
9.1	Introduction: Measuring Linear Relationships	169
9.2	Covariance	170
9.3	Correlation Coefficient	171
9.4	Cross-Correlation	172
9.5	Independence vs. Uncorrelatedness	173
9.6	Correlation Matrix	174
9.7	Variance of Linear Combinations	175
9.8	MATLAB Examples	176
9.9	Practice Problems	178
Module 10	Multiple Random Variables	179
10.1	Introduction: From 2 to N Variables	182
10.2	Joint Distributions for N Random Variables	183
10.3	Random Vectors	184

10.4	Mean Vector	185
10.5	Covariance Matrix	186
10.6	Multivariate Gaussian Distribution	187
10.7	Linear Transformations of Random Vectors	188
10.8	MATLAB Examples	189
10.9	Practice Problems	191
Module 11	Central Limit Theorem	192
11.1	Introduction: Why Is the Gaussian Everywhere?	195
11.2	Law of Large Numbers (LLN)	196
11.3	Central Limit Theorem (CLT)	197
11.4	Rate of Convergence	198
11.5	Visual Demonstration	199
11.6	Engineering Significance	200
11.7	MATLAB Monte Carlo Demonstrations	201
11.8	Approximations Using CLT	204
11.9	Practice Problems	205
Module 12	Introduction to Statistics	206
12.1	Introduction: From Probability to Statistics	209
12.2	Population vs. Sample	210
12.3	Parameter vs. Statistic	211
12.4	Random Sample	212
12.5	Sampling Distributions	213

12.6	Sample Mean $\bar{X}$	214
12.7	Sample Variance S^2	215
12.8	Sample Standard Deviation	216
12.9	Connection to CLT	217
12.10	MATLAB Examples	218
12.11	Practice Problems	220
Module 13	Parameter Estimation	221
13.1	Introduction	224
13.2	Estimator vs. Estimate	225
13.3	Properties of Estimators	226
13.4	Method of Moments (MoM)	227
13.5	Maximum Likelihood Estimation (MLE)	228
13.6	Engineering Examples	230
13.7	Comparing Estimators	231
13.8	MATLAB Examples	232
13.9	Practice Problems	234
Module 14	Confidence Intervals	235
14.1	Introduction: Why Point Estimates Aren't Enough	238
14.2	Correct Interpretation of Confidence Intervals	239
14.3	CI for Mean — Known Variance (Z-interval)	240
14.4	CI for Mean — Unknown Variance (t-interval)	241
14.5	CI for Variance	242

14.6	CI for Proportions	243
14.7	Factors Affecting CI Width	244
14.8	Sample Size Determination	245
14.9	MATLAB Demonstrations	246
14.10	Practice Problems	248
Module 15	Hypothesis Testing	249
15.1	Introduction: Making Decisions from Data	252
15.2	Framework	253
15.3	Types of Errors	254
15.4	Significance Level α	255
15.5	Power of a Test	256
15.6	The p-value	257
15.7	One-Sided vs. Two-Sided Tests	258
15.8	Tests for Means	259
15.9	Test for Variance (Chi-Square Test)	260
15.10	Step-by-Step Hypothesis Testing Procedure	261
15.11	Engineering Examples	262
15.12	MATLAB Examples	263
15.13	Practice Problems	265
Module 16	Engineering Statistical Applications	266
16.1	Communications	269
16.2	Signal Processing	271

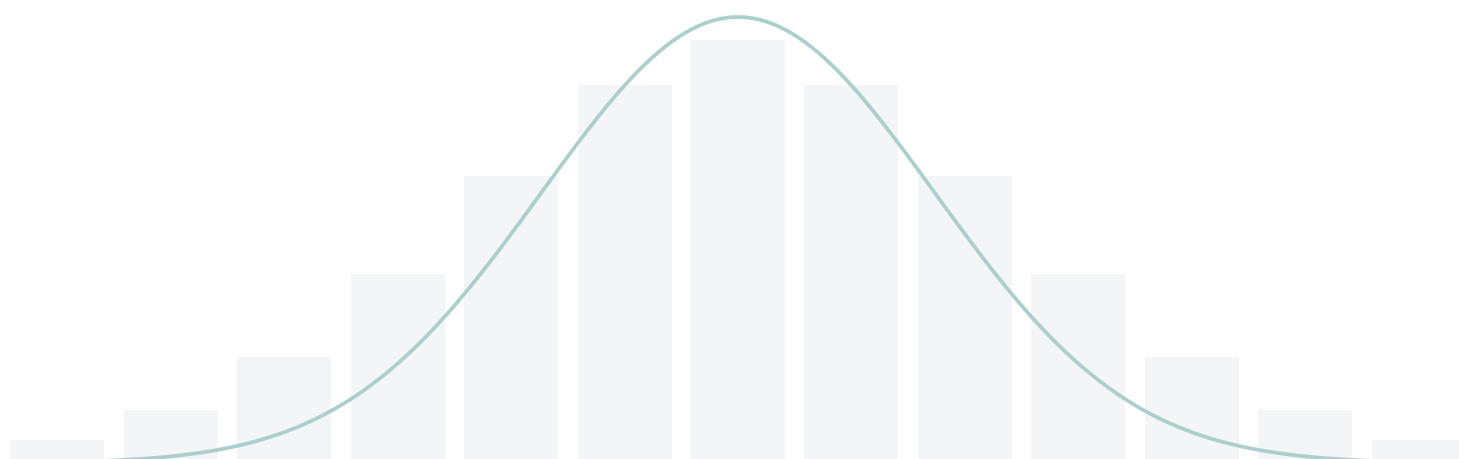
16.3	Electronics	273
16.4	Power Systems	274
16.5	Control Systems	275
16.6	Machine Learning / AI Connections	276
16.7	Comprehensive MATLAB Project: Communication System Analysis	277
16.8	Key Integration Points	279

Module 1

Probability

احتمال

1



In This Module

1.1	Introduction: Why Probability in Engineering?	14
1.2	Random Experiments	15
1.3	Sample Spaces	16
1.4	Events	17
1.5	Probability Axioms	18
1.6	Probability Rules	19
1.7	Conditional Probability	20
1.8	Independence	22
1.9	Bayes' Theorem	24
1.10	Total Probability Theorem	26
1.11	MATLAB Examples	27
1.12	Summary: Key Distinctions	31
1.13	Practice Problems	32
1.14	Key Takeaways	35

Learning Objectives

After completing this module, students will be able to:

1. Define random experiments and identify sample spaces for engineering scenarios
2. Apply set operations to define complex events
3. Use probability axioms to derive probability rules
4. Compute conditional probabilities
5. Determine statistical independence
6. Apply Bayes' theorem to engineering inference problems
7. Use the total probability theorem for multi-path analysis
8. Simulate random experiments in MATLAB

1.1 Introduction: Why Probability in Engineering?

■ The Engineering Reality

No engineering system operates in a perfectly deterministic world. Consider:

- ▶ A digital communication system transmits bits, but **noise** corrupts some bits randomly
- ▶ A manufacturing line produces resistors, but their values **vary** around the nominal
- ▶ A sensor measures temperature, but each reading has **random error**
- ▶ A wireless signal travels to a receiver, but **fading** causes random attenuation

Probability theory provides the mathematical framework to model, analyze, and design systems that operate under uncertainty.

■ Engineering Question

“If I transmit 1000 bits over a noisy channel, how many errors should I expect? Can I guarantee fewer than 5 errors?”

To answer this, we need:

1. A model of the random phenomenon (bit errors)
2. Mathematical tools to compute probabilities
3. Methods to make engineering decisions based on those probabilities

1.2 Random Experiments

Definition

A **random experiment** is a procedure that:

1. Can be repeated under identical conditions
2. Has a well-defined set of possible outcomes
3. Has an outcome that cannot be predicted with certainty before the experiment is performed

■ Engineering Examples

Experiment	Context	Uncertainty Source
Transmit one bit over a noisy channel	Communications	Thermal noise
Measure a resistor's value	Electronics	Manufacturing variation
Count packets arriving in 1 second	Networking	Traffic randomness
Measure received signal power	Wireless	Fading, interference
Test a component until failure	Reliability	Material defects, wear
Sample ADC output voltage	Signal processing	Quantization + noise

■ Key Distinction

Deterministic	Random
Output of a logic gate with known inputs	Output of a logic gate with noisy inputs
Resistance calculated from color code	Actual measured resistance
Free-space path loss at known distance	Actual received power with fading

1.3 Sample Spaces

Definition

The **sample space** S (or Ω) is the set of all possible outcomes of a random experiment.

■ Types of Sample Spaces

Discrete (Countable):

- ▶ Transmit one bit: $S = \{0, 1\}$ (received correctly or in error)
- ▶ Number of errors in a packet of n bits: $S = \{0, 1, 2, \dots, n\}$
- ▶ Number of arrivals at a server: $S = \{0, 1, 2, 3, \dots\}$

Continuous (Uncountable):

- ▶ Measured voltage: $S = \mathbb{R}$ (or a practical range like $[0, 5]$ V)
- ▶ Time until component failure: $S = [0, \infty)$
- ▶ Phase of received signal: $S = [0, 2\pi)$

Engineering Example: Communication System

A transmitter sends a bit (0 or 1) over a noisy channel. The receiver makes a decision.

Sample Space for (transmitted, received) pairs:
 $S = \{(0,0), (0,1), (1,0), (1,1)\}$

Where:

- $(0,0)$ = transmitted 0, received 0 (correct)
- $(0,1)$ = transmitted 0, received 1 (error)
- $(1,0)$ = transmitted 1, received 0 (error)
- $(1,1)$ = transmitted 1, received 1 (correct)

1.4 Events

Definition

An **event** is a subset of the sample space. An event **occurs** if the outcome of the experiment belongs to that subset.

Types of Events

- ▶ **Simple event:** Contains exactly one outcome. $E = \{(0,1)\}$
- ▶ **Compound event:** Contains multiple outcomes. $E = \{(0,1), (1,0)\}$ (any error)
- ▶ **Certain event:** The entire sample space S (always occurs)
- ▶ **Impossible event:** The empty set $\emptyset$ (never occurs)

Set Operations on Events

Operation	Notation	Meaning	Engineering Interpretation
Union	$A \cup B$	A or B (or both) occurs	Either failure mode A or B triggers alarm
Intersection	$A \cap B$	Both A and B occur	Both sensors detect the signal
Complement	A^c or $\bar{A}$	A does not occur	Component does NOT fail
Difference	$A - B = A \cap B^c$	A occurs but not B	Sensor A triggers but sensor B doesn't
Mutually exclusive	$A \cap B = \emptyset$	A and B cannot both occur	Component can't be both working and failed

Engineering Example: Manufacturing Defects

A circuit board inspection checks for three types of defects:

- ▶ A = solder bridge defect
- ▶ B = missing component
- ▶ C = wrong component value

Events:

- ▶ $A \cup B \cup C$ = “board has at least one defect”
- ▶ $A \cap B$ = “board has both a solder bridge AND a missing component”
- ▶ $(A \cup B \cup C)^c$ = “board is defect-free”

1.5 Probability Axioms

■ Kolmogorov's Axioms

For a sample space S with events defined on it, a probability function P satisfies:

Axiom 1 (Non-negativity):

$$P(A) \geq 0 \quad \text{for any event } A$$

Axiom 2 (Normalization):

$$P(S) = 1$$

Axiom 3 (Countable Additivity):

For mutually exclusive events $A_1, A_2, A_3, \dots$:

$$P(A_1 \cup A_2 \cup A_3 \cup \dots) = P(A_1) + P(A_2) + P(A_3) + \dots$$

■ Properties Derived from Axioms

Property 1: Complement Rule

$$P(A^c) = 1 - P(A)$$

Proof: $S = A \cup A^c$ and $A \cap A^c = \emptyset$, so $P(S) = P(A) + P(A^c) = 1$.

Engineering meaning: $P(\text{component works}) = 1 - P(\text{component fails})$

Property 2: Impossible Event

$$P(\emptyset) = 0$$

Property 3: Monotonicity

If $A \subseteq B$, then $P(A) \leq P(B)$

Property 4: Bound

$$0 \leq P(A) \leq 1$$

Property 5: Inclusion-Exclusion (Two Events)

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

Engineering meaning: When computing the probability of “failure A OR failure B,” we must subtract the overlap where both fail simultaneously.

1.6 Probability Rules

■ Addition Rule

For any two events:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

For mutually exclusive events ($A \cap B = \emptyset$):

$$P(A \cup B) = P(A) + P(B)$$

■ Complement Rule

$$P(A) = 1 - P(A^c)$$

This is often the easiest way to compute probabilities:

$$P(\text{at least one error in 100 bits}) = 1 - P(\text{zero errors in 100 bits})$$

■ Inclusion-Exclusion (Three Events)

$$P(A \cup B \cup C) = P(A) + P(B) + P(C) - P(A \cap B) - P(A \cap C) - P(B \cap C) + P(A \cap B \cap C)$$

Engineering Example: Network Redundancy

A data center has three independent network links. Each link fails with probability 0.05.

- ▶ A = link 1 fails, B = link 2 fails, C = link 3 fails
- ▶ $P(\text{at least one fails}) = P(A \cup B \cup C)$
- ▶ Using inclusion-exclusion:

$$P(A \cup B \cup C) = 3(0.05) - 3(0.05)^2 + (0.05)^3 = 0.15 - 0.0075 + 0.000125 = 0.142625$$

$$\text{Alternatively: } P(\text{at least one fails}) = 1 - P(\text{none fail}) = 1 - (0.95)^3 = 0.142625 \checkmark$$

1.7 Conditional Probability

Definition

The **conditional probability** of event A given that event B has occurred:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0$$

■ Interpretation

Conditional probability **updates** our knowledge. Once we know B occurred, the effective sample space shrinks from S to B, and we ask: “Within B, how likely is A?”

■ Key Distinction

Concept	Notation	Meaning
Probability of A	$P(A)$	Likelihood of A with no additional information
Conditional probability	$P(A B)$	Likelihood of A given that B has occurred
Joint probability	$P(A \cap B)$	Likelihood that both A and B occur

These are related: $P(A \cap B) = P(A|B) \cdot P(B) = P(B|A) \cdot P(A)$

Engineering Example: Detection Theory

A radar system detects aircraft:

- ▶ H_1 = aircraft is present (target)
- ▶ H_0 = aircraft is absent (no target)
- ▶ D = detector says “target present”

Important conditional probabilities:

- ▶ $P(D | H_1)$ = **probability of detection** (sensitivity)
- ▶ $P(D | H_0)$ = **probability of false alarm**
- ▶ $P(D^c | H_1)$ = **probability of miss** = $1 - P(D | H_1)$

Suppose: $P(H_1) = 0.01$, $P(D|H_1) = 0.95$, $P(D|H_0) = 0.02$

What is $P(\text{target actually present} | \text{detector says present})$?

This requires **Bayes’ theorem** (Section 1.9).

Engineering Example: Communication Channel

A binary symmetric channel has crossover probability $p = 0.01$:

- ▶ $P(\text{receive } 1 \mid \text{sent } 0) = p = 0.01$
- ▶ $P(\text{receive } 0 \mid \text{sent } 1) = p = 0.01$
- ▶ $P(\text{receive } 0 \mid \text{sent } 0) = 1 - p = 0.99$
- ▶ $P(\text{receive } 1 \mid \text{sent } 1) = 1 - p = 0.99$

If bits are equally likely to be 0 or 1:

- ▶ $P(\text{sent } 0 \cap \text{receive } 1) = P(\text{receive } 1 \mid \text{sent } 0) \cdot P(\text{sent } 0) = 0.01 \times 0.5 = 0.005$
- ▶ $P(\text{error}) = P(\text{sent } 0 \cap \text{receive } 1) + P(\text{sent } 1 \cap \text{receive } 0) = 0.005 + 0.005 = 0.01$

■ Properties of Conditional Probability

Conditional probability is itself a valid probability measure:

1. $P(A|B) \geq 0$
2. $P(S|B) = 1$
3. $P(A_1 \cup A_2|B) = P(A_1|B) + P(A_2|B)$ if $A_1 \cap A_2 = \emptyset$

■ Multiplication Rule (Chain Rule)

$$P(A \cap B) = P(A|B) \cdot P(B) = P(B|A) \cdot P(A)$$

For three events:

$$P(A \cap B \cap C) = P(A) \cdot P(B|A) \cdot P(C|A \cap B)$$

1.8 Independence

Definition

Events A and B are **statistically independent** if:

$$P(A \cap B) = P(A) \cdot P(B)$$

Equivalently: $P(A|B) = P(A)$ — knowing B occurred gives no information about A.

■ Engineering Significance

Independence means **one event provides no information about the other**. This is a modeling assumption we often make:

- ▶ Noise samples at different times are independent
- ▶ Bit errors in different time slots are independent (memoryless channel)
- ▶ Component failures in different subsystems are independent

■ Pairwise vs. Mutual Independence

For three events A, B, C:

Pairwise independence (all three pairs):

- ▶ $P(A \cap B) = P(A)P(B)$
- ▶ $P(A \cap C) = P(A)P(C)$
- ▶ $P(B \cap C) = P(B)P(C)$

Mutual independence (all combinations):

- ▶ All pairwise conditions above, AND
- ▶ $P(A \cap B \cap C) = P(A)P(B)P(C)$

▲ Pairwise independence does NOT imply mutual independence!

Engineering Example: Independent Component Failures

A system has three independent modules, each with failure probability 0.1.

$$P(\text{all three fail}) = P(A)P(B)P(C) = (0.1)^3 = 0.001$$

$$P(\text{system works}) = P(\text{at least one works}) = 1 - P(\text{all fail}) = 1 - 0.001 = 0.999$$

This calculation is ONLY valid because failures are independent.

■ Testing Independence

To verify independence from data:

1. Compute $P(A)$, $P(B)$, and $P(A \cap B)$ empirically
2. Check if $P(A \cap B) \approx P(A) \cdot P(B)$

If not approximately equal, the events are **dependent**.

■ Common Misconception

✗ “Mutually exclusive events are independent”

This is **FALSE**. If $A \cap B = \emptyset$ and $P(A) > 0$, $P(B) > 0$, then:

- ▶ $P(A \cap B) = 0$
- ▶ $P(A)P(B) > 0$
- ▶ Therefore $P(A \cap B) \neq P(A)P(B)$

Mutually exclusive events are actually **maximally dependent** — if one occurs, the other cannot.

1.9 Bayes' Theorem

■ Derivation

From the definition of conditional probability:

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{P(B|A) \cdot P(A)}{P(B)}$$

■ Full Statement

If $B_1, B_2, \dots, B_n$ partition the sample space (mutually exclusive and exhaustive):

$$P(B_i|A) = \frac{P(A|B_i) \cdot P(B_i)}{\sum_{j=1}^n P(A|B_j) \cdot P(B_j)}$$

■ Terminology

Term	Name	Engineering Meaning
$P(B_i)$	Prior probability	What we believed before observing data
$P(A B_i)$	Likelihood	How likely is the observation given each hypothesis
$P(B_i A)$	Posterior probability	Updated belief after observing data
$P(A)$	Evidence	Total probability of the observation

Engineering Example: Fault Diagnosis

A circuit board can have a fault in one of three subsystems:

- ▶ B_1 = power supply fault, $P(B_1) = 0.3$
- ▶ B_2 = processor fault, $P(B_2) = 0.5$
- ▶ B_3 = memory fault, $P(B_3) = 0.2$

A diagnostic test produces symptom A (overheating):

- ▶ $P(A|B_1) = 0.8$ (power supply faults usually cause overheating)
- ▶ $P(A|B_2) = 0.3$ (processor faults sometimes cause overheating)
- ▶ $P(A|B_3) = 0.1$ (memory faults rarely cause overheating)

Question: Given that overheating is observed, what is the most likely fault?

Solution

$$\begin{aligned} P(A) &= P(A|B_1)P(B_1) + P(A|B_2)P(B_2) + P(A|B_3)P(B_3) \\ &= (0.8)(0.3) + (0.3)(0.5) + (0.1)(0.2) \\ &= 0.24 + 0.15 + 0.02 = 0.41 \end{aligned}$$

$$P(B_1|A) = (0.8)(0.3) / 0.41 = 0.585 \text{ (most likely!)}$$

$$P(B_2|A) = (0.3)(0.5) / 0.41 = 0.366$$

$$P(B_3|A) = (0.1)(0.2) / 0.41 = 0.049$$

Engineering Decision: Investigate the power supply first.

Engineering Example: Detection with Bayes

Returning to the radar example from Section 1.7:

► $P(H_1) = 0.01$, $P(D|H_1) = 0.95$, $P(D|H_0) = 0.02$

$$P(D) = P(D|H_1)P(H_1) + P(D|H_0)P(H_0) = (0.95)(0.01) + (0.02)(0.99) = 0.0095 + 0.0198 = 0.0293$$

$$P(H_1|D) = (0.95)(0.01) / 0.0293 = 0.324$$

Interpretation: Even with a good detector (95% detection, 2% false alarm), when targets are rare (1%), most detections are false alarms! Only 32.4% of alarms correspond to real targets.

Engineering Lesson: System design must account for **prior probabilities**, not just detector performance.

1.10 Total Probability Theorem

■ Statement

If $B_1, B_2, \dots, B_n$ form a **partition** of S (mutually exclusive, exhaustive):

$$P(A) = \sum_{i=1}^n P(A|B_i) \cdot P(B_i)$$

■ Interpretation

To find the total probability of event A , decompose into all possible “paths” to A through the partition.

Engineering Example: Multi-Path Communication

A packet can be routed through three different paths:

- ▶ Path 1: chosen 50% of the time, error probability 0.01
- ▶ Path 2: chosen 30% of the time, error probability 0.05
- ▶ Path 3: chosen 20% of the time, error probability 0.10

$$\begin{aligned} P(\text{packet error}) &= P(\text{error}|\text{path 1})P(\text{path 1}) + P(\text{error}|\text{path 2})P(\text{path 2}) + P(\text{error}|\text{path 3})P(\text{path 3}) \\ &= (0.01)(0.50) + (0.05)(0.30) + (0.10)(0.20) \\ &= 0.005 + 0.015 + 0.020 = 0.040 \end{aligned}$$

Overall error rate: 4%

Engineering Example: Manufacturing with Multiple Suppliers

A company sources capacitors from two suppliers:

- ▶ Supplier A: provides 60% of capacitors, defect rate 2%
- ▶ Supplier B: provides 40% of capacitors, defect rate 5%

$$\begin{aligned} P(\text{defective}) &= P(\text{defective}|A)P(A) + P(\text{defective}|B)P(B) \\ &= (0.02)(0.60) + (0.05)(0.40) \\ &= 0.012 + 0.020 = 0.032 \end{aligned}$$

If a defective capacitor is found, which supplier likely produced it?

$$P(A|\text{defective}) = (0.02)(0.60) / 0.032 = 0.375$$

$$P(B|\text{defective}) = (0.05)(0.40) / 0.032 = 0.625$$

Despite providing fewer units, Supplier B is more likely responsible for a given defect.

1.11 MATLAB Examples

Example 1: Simulating Coin Flips (Random Experiment)

MATLAB

```

%% Simulating a Random Experiment: Biased Coin
% A communication channel has bit error probability  $p = 0.1$ 
% Simulate transmitting  $N$  bits and count errors

p = 0.1;           % Bit error probability
N = 10000;         % Number of bits transmitted

% Generate random outcomes: 1 = error, 0 = correct
errors = rand(1, N) < p;

% Empirical probability of error
p_empirical = sum(errors) / N;

fprintf('Theoretical P(error) = %.4f\n', p);
fprintf('Empirical P(error)   = %.4f (from %d trials)\n', p_empirical, N);
fprintf('Difference           = %.4f\n', abs(p - p_empirical));

```

Example 2: Verifying the Addition Rule

MATLAB

```

%% Verifying Inclusion-Exclusion with Simulation
% Two independent failure modes:
% A = overheating ( $P = 0.05$ )
% B = power surge ( $P = 0.03$ )

N = 100000;        % Number of trials
pA = 0.05;
pB = 0.03;

% Generate independent failures
A = rand(1, N) < pA;
B = rand(1, N) < pB;

% Empirical probabilities
pA_emp = mean(A);
pB_emp = mean(B);
pAB_emp = mean(A & B);           % Both occur
pAuB_emp = mean(A | B);         % At least one occurs

% Theoretical (independent)
pAB_theory = pA * pB;
pAuB_theory = pA + pB - pA*pB;

fprintf('P(A ∩ B): Theory = %.5f, Empirical = %.5f\n', pAB_theory, pAB_emp);
fprintf('P(A ∪ B): Theory = %.5f, Empirical = %.5f\n', pAuB_theory, pAuB_emp);

```

Example 3: Conditional Probability Simulation

MATLAB

```
%% Conditional Probability: Binary Symmetric Channel
% P(error | sent 0) = P(error | sent 1) = p = 0.1

p_crossover = 0.1;
N = 100000;

% Randomly generate transmitted bits (equally likely 0 or 1)
transmitted = randi([0, 1], 1, N);

% Channel: flip each bit with probability p_crossover
noise = rand(1, N) < p_crossover;
received = xor(transmitted, noise);

% Compute conditional probabilities
sent_0_idx = (transmitted == 0);
sent_1_idx = (transmitted == 1);

% P(received = 1 | sent = 0)
p_1_given_sent0 = mean(received(sent_0_idx));
% P(received = 0 | sent = 1)
p_0_given_sent1 = mean(received(sent_1_idx));

fprintf('P(receive 1 | sent 0) = %.4f (theory: %.4f)\n', p_1_given_sent0,
    ↪ p_crossover);
fprintf('P(receive 0 | sent 1) = %.4f (theory: %.4f)\n', p_0_given_sent1,
    ↪ p_crossover);
```

■ Example 4: Bayes' Theorem Simulation

MATLAB

```

%% Bayes' Theorem: Fault Diagnosis Simulation
N = 100000;

% Prior probabilities of fault types
p_power = 0.3;    p_proc = 0.5;    p_mem = 0.2;

% Likelihoods of symptom (overheating) given fault type
p_sym_power = 0.8; p_sym_proc = 0.3; p_sym_mem = 0.1;

% Simulate: generate fault type for each trial
fault_type = zeros(1, N);
r = rand(1, N);
fault_type(r < p_power) = 1;           % Power supply
fault_type(r >= p_power & r < p_power + p_proc) = 2; % Processor
fault_type(r >= p_power + p_proc) = 3; % Memory

% Generate symptom based on fault type
symptom = zeros(1, N);
symptom(fault_type == 1) = rand(1, sum(fault_type==1)) < p_sym_power;
symptom(fault_type == 2) = rand(1, sum(fault_type==2)) < p_sym_proc;
symptom(fault_type == 3) = rand(1, sum(fault_type==3)) < p_sym_mem;

% Compute posterior: P(fault_type | symptom = 1)
sym_idx = logical(symptom);
p_power_given_sym = mean(fault_type(sym_idx) == 1);
p_proc_given_sym = mean(fault_type(sym_idx) == 2);
p_mem_given_sym = mean(fault_type(sym_idx) == 3);

% Theoretical values
p_sym = p_sym_power*p_power + p_sym_proc*p_proc + p_sym_mem*p_mem;
theory_power = p_sym_power*p_power / p_sym;
theory_proc = p_sym_proc*p_proc / p_sym;
theory_mem = p_sym_mem*p_mem / p_sym;

fprintf('P(power fault | symptom): Empirical = %.3f, Theory = %.3f\n', p_power_given_sym,
    ⇨ theory_power);
fprintf('P(proc fault | symptom): Empirical = %.3f, Theory = %.3f\n', p_proc_given_sym,
    ⇨ theory_proc);
fprintf('P(mem fault | symptom): Empirical = %.3f, Theory = %.3f\n', p_mem_given_sym,
    ⇨ theory_mem);

```

Example 5: Independence Test

MATLAB

```

%% Testing Independence vs. Dependence
N = 100000;

% Case 1: Independent events
A_ind = rand(1, N) < 0.3;
B_ind = rand(1, N) < 0.4; % Generated independently

pA = mean(A_ind);
pB = mean(B_ind);
pAB = mean(A_ind & B_ind);
fprintf('Independent case:\n');
fprintf(' P(A)*P(B) = %.4f, P(AnB) = %.4f\n', pA*pB, pAB);

% Case 2: Dependent events (B depends on A)
A_dep = rand(1, N) < 0.3;
B_dep = zeros(1, N);
B_dep(A_dep == 1) = rand(1, sum(A_dep)) < 0.7; % P(B|A) = 0.7
B_dep(A_dep == 0) = rand(1, sum(~A_dep)) < 0.2; % P(B|not A) = 0.2

pA = mean(A_dep);
pB = mean(B_dep);
pAB = mean(A_dep & B_dep);
fprintf('Dependent case:\n');
fprintf(' P(A)*P(B) = %.4f, P(AnB) = %.4f\n', pA*pB, pAB);
fprintf(' These differ => events are dependent\n');

```

1.12 Summary: Key Distinctions

Concept	Definition	Engineering Meaning
Event	A subset of the sample space	A specific outcome or set of outcomes we care about
Probability	$P(A)$ = measure of likelihood	Long-run relative frequency of occurrence
Conditional Probability	$P(A B)$ = updated probability given B	How knowledge of one event changes another's likelihood
Independence	$P(A \cap B) = P(A)P(B)$	Knowing one event tells nothing about the other

1.13 Practice Problems

Problem 1: Component Reliability

A system has two independent processors. Processor 1 fails with probability 0.02, Processor 2 fails with probability 0.03. The system fails only if BOTH processors fail.

- What is the probability that the system fails?
- What is the probability that at least one processor fails?
- Given that the system is still working, what is the probability that processor 1 has failed?

Solution

- $P(\text{system fails}) = P(\text{both fail}) = (0.02)(0.03) = 0.0006$
- $P(\text{at least one fails}) = 1 - P(\text{none fail}) = 1 - (0.98)(0.97) = 1 - 0.9506 = 0.0494$
- System working = NOT(both fail) = at least one works.
 $P(P1 \text{ failed} \mid \text{system works}) = P(P1 \text{ failed} \cap P2 \text{ works}) / P(\text{system works})$
 $= (0.02)(0.97) / (1 - 0.0006) = 0.0194 / 0.9994 \approx 0.0194$

Problem 2: Sensor Detection

A fire detection system has: $P(\text{fire}) = 0.001$, $P(\text{alarm} \mid \text{fire}) = 0.98$, $P(\text{alarm} \mid \text{no fire}) = 0.01$.

- What is $P(\text{alarm})$?
- If the alarm sounds, what is $P(\text{fire} \mid \text{alarm})$?
- Comment on the result.

Solution

- $P(\text{alarm}) = P(\text{alarm} \mid \text{fire})P(\text{fire}) + P(\text{alarm} \mid \text{no fire})P(\text{no fire})$
 $= (0.98)(0.001) + (0.01)(0.999) = 0.000980 + 0.009990 = 0.010970$
- $P(\text{fire} \mid \text{alarm}) = P(\text{alarm} \mid \text{fire})P(\text{fire}) / P(\text{alarm})$
 $= (0.98)(0.001) / 0.01097 = 0.0894$
- Only ~9% of alarms correspond to real fires! This is the base rate fallacy. Even with an excellent detector, rare events produce many false alarms.

Problem 3: Communication Error

A packet of 8 bits is transmitted. Each bit has independent error probability 0.05.

- What is $P(\text{no errors in the packet})$?
- What is $P(\text{exactly one error})$?
- What is $P(\text{at least one error})$?
- Write MATLAB code to verify by simulation.

Solution

- $P(\text{no errors}) = (0.95)^8 = 0.6634$
- $P(\text{exactly one error}) = C(8,1)(0.05)^1(0.95)^7 = 8 \times 0.05 \times 0.6983 = 0.2793$
- $P(\text{at least one error}) = 1 - P(\text{no errors}) = 1 - 0.6634 = 0.3366$
- MATLAB verification:

```

p = 0.05; n_bits = 8; N_packets = 100000;
errors_per_packet = sum(rand(n_bits, N_packets) < p, 1);
fprintf('P(0 errors) = %.4f (theory: %.4f)\n', mean(errors_per_packet==0), 0.95^8);
fprintf('P(1 error) = %.4f (theory: %.4f)\n', mean(errors_per_packet==1),
    ↪ 8*0.05*0.95^7);
fprintf('P(>=1 error) = %.4f (theory: %.4f)\n', mean(errors_per_packet>=1), 1-0.95^8);
  
```

MATLAB

Problem 4: Total Probability Application

A wireless system operates in three modes:

- Mode 1 (good channel): 60% of time, $P(\text{outage}) = 0.01$
- Mode 2 (moderate channel): 30% of time, $P(\text{outage}) = 0.10$
- Mode 3 (poor channel): 10% of time, $P(\text{outage}) = 0.50$

- What is the overall outage probability?
- Given an outage occurred, what is the probability the system was in Mode 3?

Solution

- $P(\text{outage}) = (0.01)(0.60) + (0.10)(0.30) + (0.50)(0.10) = 0.006 + 0.030 + 0.050 = 0.086$
- $P(\text{Mode 3} \mid \text{outage}) = (0.50)(0.10) / 0.086 = 0.581$

Problem 5: Independence Analysis

Two sensors monitor a process. Sensor A has false alarm probability 0.05. Sensor B has false alarm probability 0.03.

- (a) If sensors are independent, what is $P(\text{both give false alarm})$?
- (b) If sensors share a noise source and $P(\text{both false alarm}) = 0.01$, are they independent?
- (c) What is $P(\text{A false alarm} \mid \text{B false alarm})$ in case (b)?

Solution

- (a) If independent: $P(\text{both}) = (0.05)(0.03) = 0.0015$
- (b) If $P(\text{both}) = 0.01 \neq 0.0015$, they are NOT independent. The shared noise source creates dependence.
- (c) $P(A|B) = P(A \cap B) / P(B) = 0.01 / 0.03 = 0.333$

This is much higher than $P(A) = 0.05$, confirming strong dependence.

1.14 Key Takeaways

1. **Probability quantifies uncertainty** — essential for engineering design under uncertainty
2. **Conditional probability updates beliefs** — the foundation of detection, estimation, and diagnosis
3. **Independence simplifies analysis** — but must be justified, not assumed
4. **Bayes' theorem inverts conditional probabilities** — from “likelihood of evidence given cause” to “likelihood of cause given evidence”
5. **Base rates matter** — even good detectors produce false alarms when events are rare
6. **Simulation validates analysis** — MATLAB Monte Carlo confirms theoretical calculations

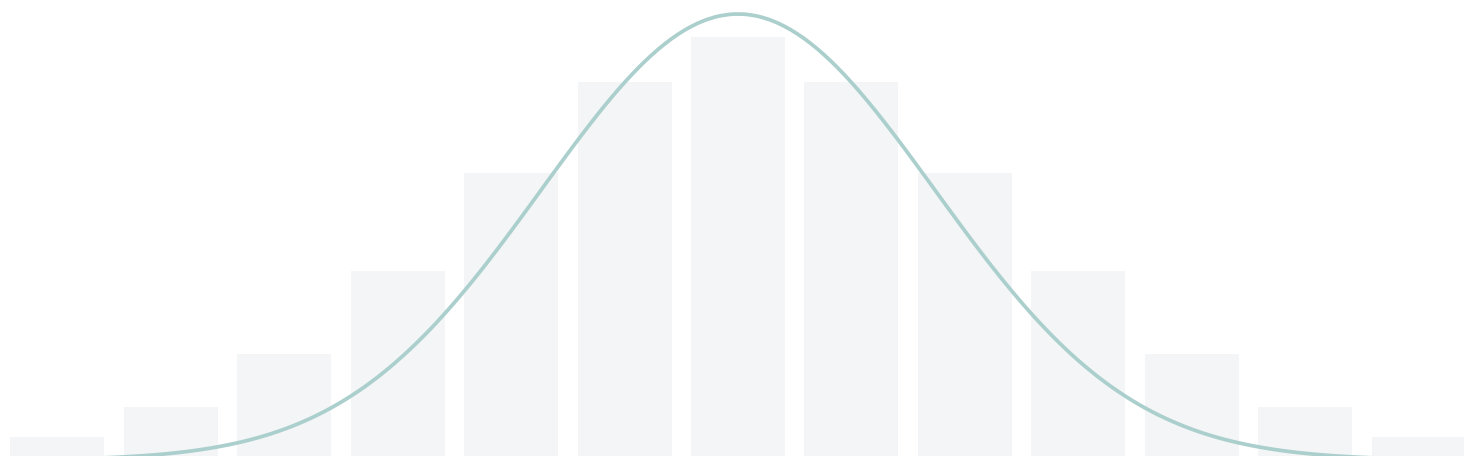
Next Module: Module 2 — One Random Variable and Its Functions

Module 2

One Random Variable and Its Functions

یک متغیر تصادفی و توابع آن

2



In This Module

2.1	Introduction: Why Random Variables?	39
2.2	Definition of a Random Variable	40
2.3	Discrete Random Variables	41
2.4	Continuous Random Variables	42
2.5	Probability Mass Function (PMF)	43
2.6	Probability Density Function (PDF)	44
2.7	Cumulative Distribution Function (CDF)	45
2.8	KEY DISTINCTION: Mass vs. Density vs. Cumulative	47
2.9	Transformations of Random Variables	49
2.10	MATLAB Examples	51
2.11	Practice Problems	55
2.12	Key Takeaways	58

Learning Objectives

After completing this module, students will be able to:

1. Define random variables and explain their role in engineering analysis
2. Distinguish between discrete and continuous random variables
3. Compute and interpret PMF, PDF, and CDF
4. Clearly explain the difference between probability mass, density, and cumulative probability
5. Apply transformations to random variables
6. Use MATLAB to generate, visualize, and analyze random variables

2.1 Introduction: Why Random Variables?

■ The Problem

In Module 1, we worked with events — subsets of a sample space. But engineering analysis requires **numbers**:

- ▶ How much noise voltage is present?
- ▶ How many packets arrived?
- ▶ What is the received signal power?

A **random variable** maps outcomes of a random experiment to real numbers, enabling us to use all the tools of calculus and algebra.

■ Engineering Motivation

We want to analyze...	We define a random variable...
Noise in a circuit	X = noise voltage (volts)
Bit errors in a frame	N = number of bit errors
Component lifetime	T = time until failure (hours)
Signal amplitude	A = received amplitude (V)
Packet delay	D = end-to-end delay (ms)

2.2 Definition of a Random Variable

■ Formal Definition

A **random variable** X is a function that maps each outcome ω in the sample space S to a real number:

$$X : S \rightarrow \mathbb{R}$$

$$\omega \mapsto X(\omega)$$

■ Interpretation

The random variable assigns a numerical value to each possible outcome of the experiment. The randomness comes from which outcome ω occurs — once ω is determined, $X(\omega)$ is a definite number.

■ Engineering Example

Experiment: Transmit 5 bits over a noisy channel.

Sample space: S = all possible sequences of correct/error for 5 bits ($2^5 = 32$ outcomes)

Random variable: X = number of bit errors

Outcome ω	$X(\omega)$
(C,C,C,C,C)	0
(E,C,C,C,C)	1
(E,E,C,C,C)	2
...	...
(E,E,E,E,E)	5

Now instead of working with 32 outcomes, we work with a single number $X \in \{0,1,2,3,4,5\}$.

2.3 Discrete Random Variables

Definition

A random variable X is **discrete** if it takes values from a **countable** set (finite or countably infinite).

■ Characteristics

- ▶ Values can be listed: $\{x_1, x_2, x_3, \dots\}$
- ▶ Each value has a specific probability (a “mass” of probability)
- ▶ Sum of all probabilities = 1

■ Engineering Examples

Random Variable	Possible Values	Context
Number of bit errors	$\{0, 1, 2, \dots, n\}$	Packet transmission
Number of retransmissions	$\{0, 1, 2, \dots\}$	ARQ protocol
Quantized voltage level	$\{0, 1, \dots, 2^N-1\}$	N-bit ADC output
Number of arrivals per second	$\{0, 1, 2, \dots\}$	Network traffic
Number of defective chips	$\{0, 1, \dots, n\}$	Wafer testing

2.4 Continuous Random Variables

Definition

A random variable X is **continuous** if it takes values from an **uncountable** set (interval of real numbers).

■ Characteristics

- ▶ X can take any value in a range (e.g., $[0, \infty)$ or $(-\infty, \infty)$)
- ▶ Probability of any single exact value is ZERO: $P(X = x) = 0$
- ▶ Probabilities are defined for intervals: $P(a \leq X \leq b)$

■ Engineering Examples

Random Variable	Range	Context
Thermal noise voltage	$(-\infty, \infty)$	Circuit noise
Time until failure	$[0, \infty)$	Reliability
Phase of received signal	$[0, 2\pi)$	Communications
Resistor value	$(R_0 - \delta, R_0 + \delta)$	Manufacturing
Channel gain	$[0, \infty)$	Wireless fading

■ Why $P(X = x) = 0$ for Continuous RVs

Imagine measuring a voltage. The probability that $V =$ exactly 3.14159265... volts (infinite precision) is zero. We can only ask about intervals: $P(3.14 < V < 3.15)$.

2.5 Probability Mass Function (PMF)

Definition

For a discrete random variable X , the **probability mass function** is:

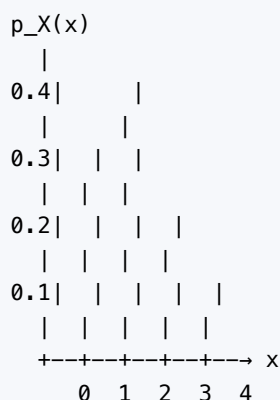
$$p_X(x) = P(X = x)$$

■ Properties

1. **Non-negativity:** $p_X(x) \geq 0$ for all x
2. **Normalization:** $\sum p_X(x_i) = 1$ (sum over all possible values)
3. **Probability computation:** $P(X \in A) = \sum_{x \in A} p_X(x)$

■ Visualization

The PMF is represented as **vertical lines** (impulses) at each possible value, with height equal to the probability.



■ Engineering Example: Bit Errors in a Byte

Transmit 8 bits, each with independent error probability $p = 0.1$.

X = number of bit errors.

$$p_X(k) = C(8,k) \cdot (0.1)^k \cdot (0.9)^{8-k}, \text{ for } k = 0, 1, \dots, 8$$

k	$p_X(k)$
0	0.4305
1	0.3826
2	0.1488
3	0.0331
4	0.0046
5+	< 0.001

2.6 Probability Density Function (PDF)

Definition

For a continuous random variable X , the **probability density function** $f_X(x)$ is defined such that:

$$P(a \leq X \leq b) = \int_a^b f_X(x) dx$$

Properties

1. **Non-negativity:** $f_X(x) \geq 0$ for all x
2. **Normalization:** $\int_{-\infty}^{\infty} f_X(x) dx = 1$
3. **Probability = Area under curve** between two points

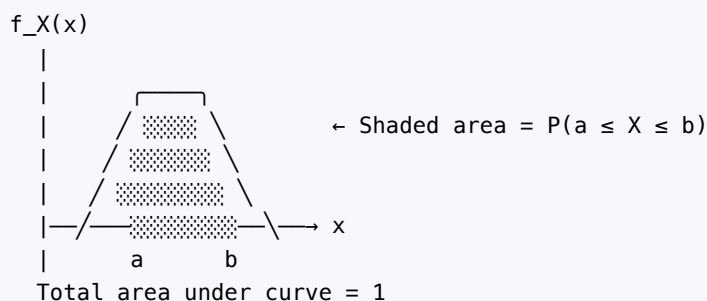
Critical Point: $f_X(x)$ is NOT a Probability!

▲ $f_X(x)$ can be GREATER than 1!

The PDF gives probability **density** — probability per unit length. Only the **integral** (area) gives probability.

Analogy: Mass density (kg/m^3) is not mass. You need to integrate density over volume to get mass. Similarly, probability density is not probability — you integrate over an interval to get probability.

Visualization



Engineering Example: Thermal Noise Voltage

Thermal noise voltage in a circuit often follows a Gaussian distribution:

$$f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-x^2/(2\sigma^2)}$$

where σ is the RMS noise voltage.

$$P(\text{noise exceeds } 2\sigma) = P(|X| > 2\sigma) = 1 - P(-2\sigma \leq X \leq 2\sigma) \approx 1 - 0.9545 = 0.0455$$

2.7 Cumulative Distribution Function (CDF)

Definition

For any random variable X (discrete or continuous), the **cumulative distribution function** is:

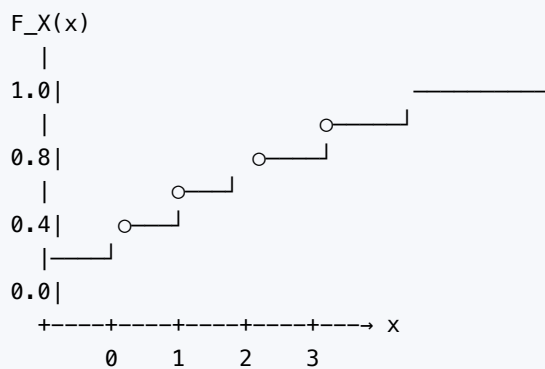
$$F_X(x) = P(X \leq x)$$

■ Properties

1. **Non-decreasing:** if $a < b$, then $F_X(a) \leq F_X(b)$
2. **Limits:** $F_X(-\infty) = 0$, $F_X(+\infty) = 1$
3. **Right-continuous:** $F_X(x)$ is continuous from the right
4. **Probability of interval:** $P(a < X \leq b) = F_X(b) - F_X(a)$

■ CDF for Discrete RVs

The CDF is a **staircase function** with jumps at each possible value:



Jump at $x = k$ has height $p_X(k)$.

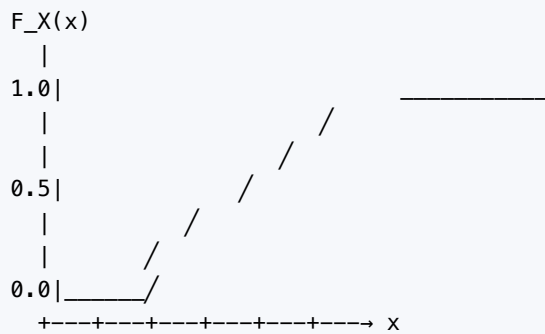
■ CDF for Continuous RVs

The CDF is a **smooth, continuous** curve:

$$F_X(x) = \int_{-\infty}^x f_X(t) dt$$

And conversely:

$$f_X(x) = \frac{dF_X(x)}{dx}$$



■ Relationship: PDF $\leftrightarrow$ CDF

Direction	Formula	Meaning
PDF $\rightarrow$ CDF	$F_X(x) = \int_{-\infty}^x f_X(t) dt$	Integrate density to get cumulative probability
CDF $\rightarrow$ PDF	$f_X(x) = dF_X/dx$	Differentiate CDF to get density

2.8 KEY DISTINCTION: Mass vs. Density vs. Cumulative

This is one of the most important conceptual distinctions in probability theory.

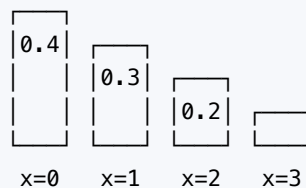
■ Comparison Table

Property	PMF $p_X(x)$	PDF $f_X(x)$	CDF $F_X(x)$
Applies to	Discrete RVs only	Continuous RVs only	Both
Gives directly	$P(X = x)$	Probability density at x	$P(X \leq x)$
Range of values	$[0, 1]$	$[0, \infty)$ — can exceed 1!	$[0, 1]$
Sum/Integral	$\sum p_X(x) = 1$	$\int f_X(x)dx = 1$	Starts at 0, ends at 1
Get probability	$P(X=x) = p_X(x)$	$P(a \leq X \leq b) = \int f_X(x)dx$	$P(a < X \leq b) = F(b) - F(a)$

■ Visual Analogy: The Warehouse

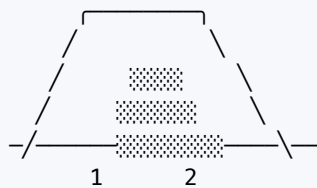
Imagine probability as “material” distributed across possible values:

Discrete (PMF): Like storing material in numbered boxes. Each box has a specific mass. The PMF tells you the mass in each box.



Total mass in all boxes = 1

Continuous (PDF): Like spreading material along a continuous shelf. The density tells you how thick the material is at each point. You find mass by computing area (thickness $\times$ width).



← Area of shaded region = $P(1 \leq X \leq 2)$

Total area under curve = 1

Cumulative (CDF): Like a running total — how much material have you accumulated up to this point?

■ Why This Matters in Engineering

Example: A noise voltage X has PDF $f_X(x) = 2$ for $x \in [0, 0.5]$.

- ▶ $f_X(0.3) = 2$ — this is NOT a probability! It means the density is 2 V^{-1} .
- ▶ $P(0.2 \leq X \leq 0.4) = \int_{0.2}^{0.4} 2 \, dx = 2 \times 0.2 = 0.4$ — THIS is a probability.
- ▶ $P(X = 0.3) = 0$ — a single point has zero probability for continuous RVs.

■ Common Errors to Avoid

✗ Wrong Statement	✓ Correct Statement
“The probability at $x=2$ is $f(2)$ ”	“The probability density at $x=2$ is $f(2)$ ”
“ $f(x)$ must be ≤ 1 ”	“ $f(x)$ can be any non-negative value”
“ $P(X=3) = f(3)$ for continuous X ”	“ $P(X=3) = 0$ for continuous X ”
“PMF and PDF are the same thing”	“PMF gives mass; PDF gives density”

2.9 Transformations of Random Variables

■ Motivation

In engineering, we often know the distribution of one quantity but need the distribution of a **function** of that quantity:

- ▶ Know voltage $V \rightarrow$ need power $P = V^2/R$
- ▶ Know linear signal $X \rightarrow$ need dB value $Y = 10 \cdot \log_{10}(X)$
- ▶ Know noise $N \rightarrow$ need $|N|$ (rectified noise)

Problem Statement

Given: X with known PDF $f_X(x)$ or PMF $p_X(x)$

Find: The distribution of $Y = g(X)$

■ Case 1: Discrete RV Transformation

If X is discrete with PMF $p_X(x)$, and $Y = g(X)$:

$$p_Y(y) = \sum_{x:g(x)=y} p_X(x)$$

Sum the probabilities of all x values that map to the same y .

Example: $X \in \{-2, -1, 0, 1, 2\}$ with equal probability 0.2 each.
 $Y = X^2$

$Y = X^2$	X values mapping to Y	$p_Y(y)$
0	$\{0\}$	0.2
1	$\{-1, 1\}$	0.4
4	$\{-2, 2\}$	0.4

■ Case 2: Monotonic Transformation (Continuous)

If $Y = g(X)$ where g is strictly monotonic and differentiable:

$$f_Y(y) = f_X(g^{-1}(y)) \cdot \left| \frac{dg^{-1}(y)}{dy} \right|$$

Or equivalently:

$$f_Y(y) = \left. \frac{f_X(x)}{|g'(x)|} \right|_{x=g^{-1}(y)}$$

Derivation via CDF method:

1. Start with CDF: $F_Y(y) = P(Y \leq y) = P(g(X) \leq y)$
2. Invert: $= P(X \leq g^{-1}(y))$ if g is increasing
3. Differentiate: $f_Y(y) = f_X(g^{-1}(y)) \cdot d/dy[g^{-1}(y)]$

■ Case 3: Non-Monotonic Transformation

If g is not monotonic, split the domain into intervals where g IS monotonic:

$$f_Y(y) = \sum_i \frac{f_X(x_i)}{|g'(x_i)|}$$

where $x_1, x_2, \dots$ are all solutions of $g(x) = y$.

Engineering Example: Power from Voltage

A noise voltage X has PDF $f_X(x)$ (e.g., Gaussian with zero mean).

Power dissipated in a 1Ω resistor: $P = X^2$

Since $Y = X^2$ is NOT monotonic (both $+x$ and $-x$ give the same y):

For $y > 0$: $x = +\sqrt{y}$ and $x = -\sqrt{y}$ are both solutions.

$g(x) = x^2$, so $g'(x) = 2x$

$$f_Y(y) = \frac{f_X(\sqrt{y})}{2\sqrt{y}} + \frac{f_X(-\sqrt{y})}{2\sqrt{y}}, \quad y > 0$$

If X is zero-mean Gaussian: $f_X(x) = f_X(-x)$, so:

$$f_Y(y) = \frac{f_X(\sqrt{y})}{\sqrt{y}}, \quad y > 0$$

Engineering Example: Linear Amplification

Signal $Y = aX + b$ (amplifier with gain a and offset b).

X has PDF $f_X(x)$.

$g^{-1}(y) = (y-b)/a$, and $|d/dy[(y-b)/a]| = 1/|a|$

$$f_Y(y) = \frac{1}{|a|} f_X\left(\frac{y-b}{a}\right)$$

2.10 MATLAB Examples

Example 1: Plotting a PMF

```

%% PMF of Number of Bit Errors in a Byte
n = 8;           % bits per byte
p = 0.1;         % bit error probability
k = 0:n;         % possible values

% Compute PMF (binomial distribution)
pmf = binopdf(k, n, p);

% Plot
figure;
stem(k, pmf, 'filled', 'LineWidth', 2);
xlabel('Number of Errors (k)');
ylabel('P(X = k)');
title('PMF: Bit Errors in 8-bit Byte (p = 0.1)');
grid on;

% Verify normalization
fprintf('Sum of PMF = %.6f (should be 1)\n', sum(pmf));

```

MATLAB

Example 2: Plotting a PDF and Verifying Properties

```

%% PDF of Gaussian Noise Voltage
sigma = 0.5;     % RMS noise voltage
x = linspace(-3*sigma, 3*sigma, 1000);

% Compute PDF
pdf_x = normpdf(x, 0, sigma);

% Plot
figure;
plot(x, pdf_x, 'b-', 'LineWidth', 2);
xlabel('Voltage (V)');
ylabel('f_X(x) [V^{-1}]');
title('PDF: Gaussian Noise (σ = 0.5 V)');
grid on;

% Verify normalization (numerical integration)
dx = x(2) - x(1);
total_area = trapz(x, pdf_x);
fprintf('Total area under PDF = %.6f (should be 1)\n', total_area);

% Note: PDF values can exceed 1!
fprintf('Maximum PDF value = %.4f (can be > 1)\n', max(pdf_x));

% Compute P(|X| > 1V)
prob_exceed = 1 - normcdf(1, 0, sigma) + normcdf(-1, 0, sigma);
fprintf('P(|X| > 1V) = %.4f\n', prob_exceed);

```

MATLAB

Example 3: CDF Visualization

MATLAB

```
%% Comparing CDF for Discrete vs Continuous

figure;

% Subplot 1: Discrete CDF (Poisson, packets per second)
subplot(1,2,1);
lambda = 3;      % average arrivals per second
k = 0:10;
cdf_discrete = poisscdf(k, lambda);
stairs(k, cdf_discrete, 'r-', 'LineWidth', 2);
xlabel('Number of Packets');
ylabel('F_X(x) = P(X ≤ x)');
title('CDF: Discrete (Poisson, λ=3)');
grid on; ylim([0 1.1]);

% Subplot 2: Continuous CDF (Exponential, time to failure)
subplot(1,2,2);
mu = 5;          % mean time to failure (hours)
t = linspace(0, 20, 500);
cdf_continuous = expcdf(t, mu);
plot(t, cdf_continuous, 'b-', 'LineWidth', 2);
xlabel('Time (hours)');
ylabel('F_X(x) = P(X ≤ x)');
title('CDF: Continuous (Exponential, μ=5)');
grid on; ylim([0 1.1]);
```

■ Example 4: Transformation of a Random Variable

MATLAB

```

%% Transformation: Power from Noise Voltage
% X = noise voltage, Gaussian(0, sigma)
% Y = X^2 = instantaneous power (in 1-ohm load)

sigma = 1;
N = 100000;

% Generate Gaussian noise samples
X = sigma * randn(1, N);

% Transform: power = voltage^2
Y = X.^2;

% Plot empirical PDF of Y using histogram
figure;
subplot(2,1,1);
histogram(X, 100, 'Normalization', 'pdf');
hold on;
x_theory = linspace(-4, 4, 500);
plot(x_theory, normpdf(x_theory, 0, sigma), 'r-', 'LineWidth', 2);
xlabel('Voltage X'); ylabel('f_X(x)');
title('Input: Gaussian Noise Voltage');
legend('Empirical', 'Theoretical');

subplot(2,1,2);
histogram(Y, 100, 'Normalization', 'pdf');
hold on;
% Theoretical: Y ~ chi-squared with 1 degree of freedom
y_theory = linspace(0.01, 10, 500);
pdf_y_theory = chi2pdf(y_theory, 1);
plot(y_theory, pdf_y_theory, 'r-', 'LineWidth', 2);
xlabel('Power Y = X^2'); ylabel('f_Y(y)');
title('Output: Instantaneous Power');
legend('Empirical', 'Theoretical ( $\chi^2_1$ )');

```

■ Example 5: PMF vs PDF vs CDF Comparison

MATLAB

```

%% Complete Visualization: PMF/PDF and CDF side by side
figure;

% --- Discrete Case: Number of retransmissions ---
% Geometric distribution: number of trials until first success
p_success = 0.3; % success probability per trial
k = 1:15;

% PMF
subplot(2,2,1);
pmf_geom = geopdf(k-1, p_success); % MATLAB's geopdf counts failures
stem(k, pmf_geom, 'filled', 'LineWidth', 1.5);
xlabel('k (transmissions)'); ylabel('P(X = k)');
title('Discrete PMF: Retransmissions');
grid on;

```

```
% CDF
subplot(2,2,2);
cdf_geom = geocdf(k-1, p_success);
stairs(k, cdf_geom, 'LineWidth', 2);
xlabel('k'); ylabel('P(X ≤ k)');
title('Discrete CDF');
grid on; ylim([0 1.1]);

% --- Continuous Case: Delay time ---
% Exponential distribution
mu = 2; % mean delay in ms
t = linspace(0, 10, 500);

% PDF
subplot(2,2,3);
pdf_exp = exppdf(t, mu);
plot(t, pdf_exp, 'LineWidth', 2);
xlabel('Delay (ms)'); ylabel('f_X(x) [ms^{-1}]');
title('Continuous PDF: Delay');
grid on;

% CDF
subplot(2,2,4);
cdf_exp = expcdf(t, mu);
plot(t, cdf_exp, 'LineWidth', 2);
xlabel('Delay (ms)'); ylabel('P(X ≤ x)');
title('Continuous CDF');
grid on; ylim([0 1.1]);

sgtitle('PMF/PDF vs CDF: Discrete and Continuous', 'FontSize', 14);
```

2.11 Practice Problems

Problem 1: Identifying RV Type

Classify each as discrete or continuous and state the range:

- (a) Number of dropped calls in one hour
- (b) Received signal strength (dBm)
- (c) Number of bits in error in a 1500-byte Ethernet frame
- (d) Time between packet arrivals
- (e) Quantizer output level (3-bit quantizer)

Solution

- (a) Discrete, $X \in \{0, 1, 2, \dots\}$
- (b) Continuous, $X \in (-\infty, \infty)$ practically $\sim (-120, 0)$ dBm
- (c) Discrete, $X \in \{0, 1, 2, \dots, 12000\}$
- (d) Continuous, $T \in [0, \infty)$
- (e) Discrete, $X \in \{0, 1, 2, 3, 4, 5, 6, 7\}$

Problem 2: PMF Properties

A random variable X has PMF: $p_X(0) = 0.1$, $p_X(1) = 0.3$, $p_X(2) = c$, $p_X(3) = 0.2$.

- (a) Find c .
- (b) Find $P(X \geq 2)$.
- (c) Find $F_X(1.5)$.
- (d) Find $P(0.5 < X \leq 2.5)$.

Solution

- (a) Sum = 1: $0.1 + 0.3 + c + 0.2 = 1 \rightarrow c = 0.4$
- (b) $P(X \geq 2) = p_X(2) + p_X(3) = 0.4 + 0.2 = 0.6$
- (c) $F_X(1.5) = P(X \leq 1.5) = p_X(0) + p_X(1) = 0.1 + 0.3 = 0.4$
- (d) $P(0.5 < X \leq 2.5) = p_X(1) + p_X(2) = 0.3 + 0.4 = 0.7$

Problem 3: PDF and CDF

A random variable has PDF:

$f_X(x) = cx$ for $0 \leq x \leq 4$, and 0 otherwise.

- (a) Find c .
- (b) Find the CDF $F_X(x)$.
- (c) Find $P(1 \leq X \leq 3)$.
- (d) Find the value x_0 such that $P(X \leq x_0) = 0.5$.

Solution

- (a) $\int_0^4 cx \, dx = c \cdot [x^2/2]_0^4 = c \cdot 8 = 1 \rightarrow c = 1/8$
- (b) $F_X(x) = \int_0^x (t/8) dt = x^2/16$, for $0 \leq x \leq 4$
 $F_X(x) = 0$ for $x < 0$; $F_X(x) = 1$ for $x > 4$
- (c) $P(1 \leq X \leq 3) = F_X(3) - F_X(1) = 9/16 - 1/16 = 8/16 = 0.5$
- (d) $x_0^2/16 = 0.5 \rightarrow x_0^2 = 8 \rightarrow x_0 = 2\sqrt{2} \approx 2.83$

Problem 4: Transformation

A sensor measures temperature X uniformly distributed on $[20, 30]$ °C.

The output voltage is $Y = 0.1X - 2$ (linear sensor).

- (a) Find the PDF of Y .
- (b) Find the range of Y .
- (c) Find $P(Y > 0.5)$.

Solution

- (a) $X \sim \text{Uniform}[20, 30]$, so $f_X(x) = 1/10$ for $20 \leq x \leq 30$.
 $Y = 0.1X - 2$, so $a = 0.1$, $b = -2$.
 $f_Y(y) = (1/|a|) \cdot f_X((y-b)/a) = (1/0.1) \cdot (1/10) = 1$ for the valid range.
- (b) When $X = 20$: $Y = 0.1(20) - 2 = 0$; When $X = 30$: $Y = 0.1(30) - 2 = 1$
Range of Y : $[0, 1]$. So $Y \sim \text{Uniform}[0, 1]$.
- (c) $P(Y > 0.5) = 1 - F_Y(0.5) = 1 - 0.5 = 0.5$

Problem 5: Non-Monotonic Transformation

A voltage $X \sim \text{Uniform}[-1, 1]$ V. The instantaneous power is $Y = X^2$.

- (a) Find the CDF of Y .
- (b) Find the PDF of Y .
- (c) Verify using MATLAB simulation.

Solution

- (a) For $0 \leq y \leq 1$:

$$F_Y(y) = P(X^2 \leq y) = P(-\sqrt{y} \leq X \leq \sqrt{y}) = F_X(\sqrt{y}) - F_X(-\sqrt{y})$$

Since $X \sim \text{Uniform}[-1, 1]$: $F_X(x) = (x+1)/2$

$$F_Y(y) = (\sqrt{y} + 1)/2 - (-\sqrt{y} + 1)/2 = \sqrt{y}$$

- (b) $f_Y(y) = dF_Y/dy = 1/(2\sqrt{y})$ for $0 < y \leq 1$

- (c) MATLAB:

```
N = 100000;
X = 2*rand(1,N) - 1; % Uniform[-1,1]
Y = X.^2;
histogram(Y, 100, 'Normalization', 'pdf');
hold on;
y = linspace(0.01, 1, 200);
plot(y, 1./(2*sqrt(y)), 'r-', 'LineWidth', 2);
legend('Empirical', 'Theory: 1/(2√y)');
xlabel('Y = X^2'); ylabel('f_Y(y)');
```

MATLAB

2.12 Key Takeaways

1. **Random variables** convert experimental outcomes into numbers for mathematical analysis
2. **PMF** gives the actual probability for discrete RVs (bar heights sum to 1)
3. **PDF** gives probability density for continuous RVs (area gives probability, not height)
4. **CDF** works universally — gives cumulative probability $P(X \leq x)$
5. **Transformations** let us derive distributions of functions of random variables
6. The **CDF method** (find CDF first, then differentiate) is often the safest approach for transformations

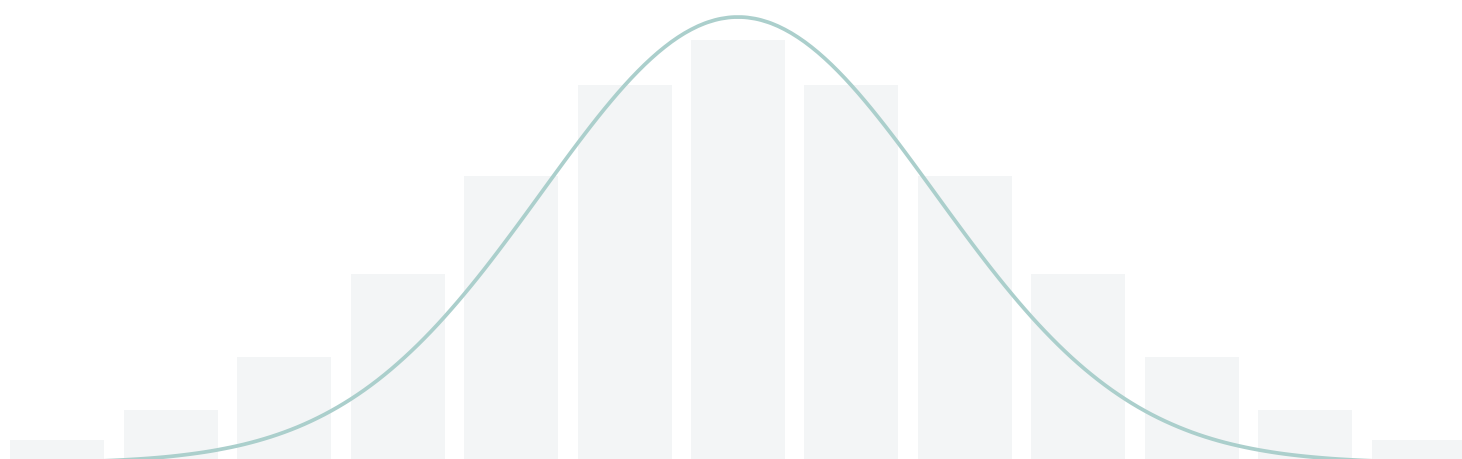
Next Module: Module 3 — Important Probability Distributions

Module 3

Important Probability Distributions

توزیع‌های مهم احتمال

3



In This Module

3.1	Introduction: Choosing the Right Distribution	62
3.2	Bernoulli Distribution	63
3.3	Binomial Distribution	65
3.4	Geometric Distribution	67
3.5	Negative Binomial Distribution	69
3.6	Poisson Distribution	71
3.7	Uniform Distribution (Continuous)	74
3.8	Exponential Distribution	76
3.9	Gaussian (Normal) Distribution	79
3.10	Gamma Distribution	83
3.11	Rayleigh Distribution	85
3.12	Chi-Square Distribution	87
3.13	Distribution Selection Guide	89
3.14	Summary Table	91
3.15	Practice Problems	92

Learning Objectives

After completing this module, students will be able to:

1. Identify which probability distribution models a given engineering scenario
2. State the PMF/PDF, CDF, mean, and variance for all major distributions
3. Explain the physical interpretation and engineering context for each distribution
4. Determine when to use (and when NOT to use) each distribution
5. Implement distributions in MATLAB for simulation
6. Recognize distribution patterns from data characteristics

3.1 Introduction: Choosing the Right Distribution

■ The Engineering Challenge

When modeling a random phenomenon, the critical question is:

“Which probability distribution best describes this uncertain quantity?”

Choosing the wrong distribution leads to incorrect predictions, improper designs, and system failures.

■ How to Choose

The distribution should match the **physical mechanism** generating randomness:

- ▶ Counting successes → Binomial
- ▶ Waiting for events → Exponential/Geometric
- ▶ Sum of many small effects → Gaussian
- ▶ Signal envelope in fading → Rayleigh

This module provides the tools to make this choice correctly.

3.2 Bernoulli Distribution

■ Physical Interpretation

Models a **single trial** with two possible outcomes: “success” (1) or “failure” (0).

■ Engineering Context

Any binary random outcome:

- ▶ One bit transmitted: error or correct
- ▶ One component tested: pass or fail
- ▶ One packet sent: delivered or lost
- ▶ One detection attempt: target present or absent

Mathematical Definition

$X \sim \text{Bernoulli}(p)$, where p = probability of success.

■ PMF

$$p_X(x) = \begin{cases} p & x = 1 \\ 1 - p = q & x = 0 \end{cases}$$

Compact form: $p_X(x) = p^x(1-p)^{1-x}$ for $x \in \{0, 1\}$

■ CDF

$$F_X(x) = \begin{cases} 0 & x < 0 \\ 1 - p & 0 \leq x < 1 \\ 1 & x \geq 1 \end{cases}$$

■ Mean

$$E[X] = p$$

■ Variance

$$\text{Var}(X) = p(1 - p)$$

Maximum variance at $p = 0.5$ (maximum uncertainty).

■ When to Use

- ✓ Single binary trial with fixed probability
- ✓ Building block for more complex distributions
- ✓ Modeling ON/OFF states, pass/fail outcomes

■ When NOT to Use

- ✗ Multiple trials (use Binomial)
- ✗ Outcome is not binary
- ✗ Probability changes between trials

■ MATLAB Implementation

```
% Generate Bernoulli random variables
p = 0.1;           % Bit error probability
N = 10000;
X = binornd(1, p, 1, N); % or: X = rand(1,N) < p;

% Verify
fprintf('Mean: %.4f (theory: %.4f)\n', mean(X), p);
fprintf('Var: %.4f (theory: %.4f)\n', var(X), p*(1-p));
```

MATLAB

3.3 Binomial Distribution

■ Physical Interpretation

Models the **number of successes** in n **independent** trials, each with the same success probability p .

■ Engineering Context

- ▶ Number of bit errors in n transmitted bits
- ▶ Number of defective components in a batch of n
- ▶ Number of successful packet transmissions out of n attempts
- ▶ Number of sensors that detect a signal (out of n sensors)

Mathematical Definition

$$X \sim \text{Binomial}(n, p)$$

■ PMF

$$p_X(k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad k = 0, 1, 2, \dots, n$$

where $C(n,k) = n! / (k!(n-k)!)$

■ CDF

$$F_X(k) = \sum_{i=0}^{\lfloor k \rfloor} \binom{n}{i} p^i (1-p)^{n-i}$$

No closed-form expression — use tables or MATLAB.

■ Mean

$$E[X] = np$$

■ Variance

$$\text{Var}(X) = np(1-p)$$

■ When to Use

- ✓ Fixed number of trials n
- ✓ Each trial is independent
- ✓ Each trial has the same probability p
- ✓ Counting the number of “successes”

■ When NOT to Use

- ✗ Trials are not independent (correlated failures)
- ✗ Probability changes between trials

- ✗ n is not fixed (use Negative Binomial or Poisson)
- ✗ Very large n with small p (use Poisson approximation)

■ MATLAB Implementation

MATLAB

```
% Binomial Distribution: Bit Errors in a Frame
n = 100;           % bits per frame
p = 0.05;          % bit error probability
k = 0:20;          % range of interest

% PMF and CDF
pmf = binopdf(k, n, p);
cdf_vals = binocdf(k, n, p);

% Plot PMF
figure;
stem(k, pmf, 'filled', 'LineWidth', 1.5);
xlabel('Number of Errors (k)');
ylabel('P(X = k)');
title(sprintf('Binomial PMF: n=%d, p=%.2f', n, p));
grid on;

% Simulation verification
N_sim = 100000;
X_sim = binornd(n, p, 1, N_sim);
fprintf('E[X]: Theory = %.2f, Simulation = %.2f\n', n*p, mean(X_sim));
fprintf('Var(X): Theory = %.2f, Simulation = %.2f\n', n*p*(1-p), var(X_sim));
```

■ Simulation Example: Packet Error Rate

MATLAB

```
% Engineering Scenario: How many packets out of 50 have errors?
% Each packet has 1000 bits, BER = 1e-4
% A packet has an error if ANY bit is in error.

BER = 1e-4;
bits_per_packet = 1000;
n_packets = 50;

% P(packet error) = 1 - P(all bits correct)
p_packet_error = 1 - (1 - BER)^bits_per_packet;
fprintf('P(packet error) = %.4f\n', p_packet_error);

% Number of errored packets ~ Binomial(50, p_packet_error)
k = 0:10;
pmf_packets = binopdf(k, n_packets, p_packet_error);
fprintf('P(0 packet errors) = %.4f\n', pmf_packets(1));
fprintf('P(≤ 2 packet errors) = %.4f\n', sum(pmf_packets(1:3)));
```

3.4 Geometric Distribution

■ Physical Interpretation

Models the **number of trials until the first success** (or equivalently, the number of failures before the first success).

■ Engineering Context

- ▶ Number of transmissions until a packet is successfully delivered
- ▶ Number of components tested until finding a defective one
- ▶ Number of login attempts until successful authentication
- ▶ Number of time slots until channel becomes available

Mathematical Definition

$X \sim \text{Geometric}(p)$

Convention: X = number of the trial on which first success occurs ($X \in \{1, 2, 3, \dots\}$).

■ PMF

$$p_X(k) = (1 - p)^{k-1}p, \quad k = 1, 2, 3, \dots$$

($k-1$ failures, then one success)

■ CDF

$$F_X(k) = 1 - (1 - p)^k, \quad k = 1, 2, 3, \dots$$

■ Mean

$$E[X] = \frac{1}{p}$$

■ Variance

$$\text{Var}(X) = \frac{1 - p}{p^2}$$

■ Memoryless Property

The geometric distribution is the **ONLY** discrete distribution with the memoryless property:

$$P(X > m + n \mid X > m) = P(X > n)$$

Engineering meaning: If a system hasn't succeeded in m attempts, the probability of needing n more attempts is the same as starting fresh. "Past failures don't affect future probability."

■ When to Use

- ✓ Repeated independent trials until first success
- ✓ Each trial has the same probability
- ✓ Memoryless assumption is appropriate (e.g., independent retransmissions)

■ When NOT to Use

- ✗ Success probability changes over time (aging, learning)
- ✗ Trials are not independent
- ✗ Looking for the r -th success (use Negative Binomial)

■ MATLAB Implementation

```

%% Geometric Distribution: Retransmissions Until Success
p_success = 0.7; % P(successful transmission)
k = 1:15;

% PMF
pmf_geom = geopdf(k-1, p_success); % MATLAB counts failures (k-1)

% Simulation
N = 100000;
X_sim = geornd(p_success, 1, N) + 1; % +1 to count trial number

figure;
subplot(1,2,1);
stem(k, pmf_geom, 'filled');
xlabel('Trial Number k'); ylabel('P(X = k)');
title(sprintf('Geometric PMF (p = %.1f)', p_success));
grid on;

subplot(1,2,2);
histogram(X_sim, 'Normalization', 'probability', 'BinMethod', 'integers');
xlabel('Trial Number k'); ylabel('Relative Frequency');
title('Simulation Histogram');
grid on;

fprintf('E[X]: Theory = %.2f, Simulation = %.2f\n', 1/p_success, mean(X_sim));

```

MATLAB

3.5 Negative Binomial Distribution

■ Physical Interpretation

Models the **number of trials until the r-th success** (generalization of Geometric).

■ Engineering Context

- ▶ Number of transmissions until r packets are successfully delivered
- ▶ Number of components inspected until finding r defectives
- ▶ Number of time slots until r channels become available

Mathematical Definition

$X \sim \text{NegBin}(r, p)$ = number of trials until r-th success.

■ PMF

$$p_X(k) = \binom{k-1}{r-1} p^r (1-p)^{k-r}, \quad k = r, r+1, r+2, \dots$$

(Choose which r-1 of the first k-1 trials were successes, the k-th is the r-th success.)

■ CDF

No simple closed form. Computed by summation or MATLAB.

■ Mean

$$E[X] = \frac{r}{p}$$

■ Variance

$$\text{Var}(X) = \frac{r(1-p)}{p^2}$$

■ Relationship to Geometric

When $r = 1$, the Negative Binomial reduces to the Geometric distribution.

■ When to Use

- ✓ Repeated independent trials until r-th success
- ✓ Constant probability per trial
- ✓ Generalizing geometric to multiple required successes

■ When NOT to Use

- ✗ Only need first success (use Geometric — simpler)
- ✗ Trials not independent
- ✗ Fixed number of trials (use Binomial)

■ MATLAB Implementation

MATLAB

```
%% Negative Binomial: Transmissions Until r Successes
r = 5;           % Need 5 successful deliveries
p_success = 0.8; % P(success per trial)

% MATLAB's nbinpdf counts FAILURES before r-th success
% X_matlab = number of failures, so total trials = X_matlab + r
k_failures = 0:20;

pmf_nb = nbinpdf(k_failures, r, p_success);

figure;
stem(k_failures + r, pmf_nb, 'filled');
xlabel('Total Trials Until 5th Success');
ylabel('Probability');
title(sprintf('Negative Binomial: r=%d, p=%.1f', r, p_success));
grid on;

% Simulation
N = 100000;
X_sim = nbinrnd(r, p_success, 1, N) + r; % total trials
fprintf('E[trials]: Theory = %.2f, Simulation = %.2f\n', r/p_success, mean(X_sim));
```

3.6 Poisson Distribution

■ Physical Interpretation

Models the **number of events occurring in a fixed interval** (time, space, area) when events occur independently at a constant average rate.

■ Engineering Context

- ▶ Number of photons hitting a detector in $1 \mu\text{s}$
- ▶ Number of packets arriving at a router in 1 second
- ▶ Number of defects per cm^2 on a silicon wafer
- ▶ Number of cosmic ray events in a sensor per hour
- ▶ Number of interference spikes in a measurement window

Mathematical Definition

$X \sim \text{Poisson}(\lambda)$, where λ = average number of events per interval.

■ PMF

$$p_X(k) = \frac{\lambda^k e^{-\lambda}}{k!}, \quad k = 0, 1, 2, \dots$$

■ CDF

$$F_X(k) = e^{-\lambda} \sum_{i=0}^{\lfloor k \rfloor} \frac{\lambda^i}{i!}$$

■ Mean

$$E[X] = \lambda$$

■ Variance

$$\text{Var}(X) = \lambda$$

Unique property: Mean equals variance! This is a quick check — if sample mean $\approx$ sample variance, data may be Poisson.

■ Poisson as Binomial Approximation

When n is large and p is small, with $\lambda = np$:

$$\text{Binomial}(n, p) \approx \text{Poisson}(\lambda = np)$$

Rule of thumb: $n \geq 20$ and $p \leq 0.05$.

■ When to Use

- ✓ Counting events in a fixed interval
- ✓ Events occur independently
- ✓ Events occur at a constant average rate
- ✓ Rare events with many opportunities (large n , small p)
- ✓ Mean $\approx$ Variance in data

■ When NOT to Use

- ✗ Events are not independent (clustered arrivals)
- ✗ Rate varies with time (non-homogeneous process)
- ✗ Events are not rare (p not small) — use Binomial
- ✗ Mean $\neq$ Variance (look at overdispersion/underdispersion)

■ MATLAB Implementation

```
% Poisson Distribution: Network Packet Arrivals
lambda = 4.5;      % Average packets per millisecond
k = 0:15;

pmf_pois = poisspdf(k, lambda);
cdf_pois = poisscdf(k, lambda);

figure;
subplot(1,2,1);
stem(k, pmf_pois, 'filled', 'LineWidth', 1.5);
xlabel('Number of Arrivals (k)');
ylabel('P(X = k)');
title(sprintf('Poisson PMF ( $\lambda = %.1f$ )', lambda));
grid on;

subplot(1,2,2);
stairs(k, cdf_pois, 'LineWidth', 2);
xlabel('k'); ylabel('P(X ≤ k)');
title('Poisson CDF');
grid on;
```

MATLAB

■ Simulation Example: Photon Counting

MATLAB

```
%% Engineering Scenario: Photon Detection
% An optical receiver detects photons. Average rate: 10 photons/μs.
% What is P(detecting fewer than 5 photons in one μs)?

lambda = 10;    % average photons per μs
N = 100000;

% Analytical
p_fewer_5 = poisscdf(4, lambda);
fprintf('P(X < 5): Analytical = %.6f\n', p_fewer_5);

% Simulation
X_sim = poissrnd(lambda, 1, N);
p_fewer_5_sim = mean(X_sim < 5);
fprintf('P(X < 5): Simulation = %.6f\n', p_fewer_5_sim);

% Verify mean = variance (Poisson signature)
fprintf('Sample Mean = %.3f, Sample Variance = %.3f\n', mean(X_sim), var(X_sim));
```

3.7 Uniform Distribution (Continuous)

■ Physical Interpretation

Models a quantity that is **equally likely** to take any value in an interval $[a, b]$. Represents **maximum uncertainty** within a known range.

■ Engineering Context

- ▶ Phase of a received signal (unknown, equally likely in $[0, 2\pi)$)
- ▶ Round-off error in an ADC (uniform in $[-\Delta/2, \Delta/2]$ where Δ = quantization step)
- ▶ Arrival time within a time slot (unknown position)
- ▶ Random access protocol: choose a random backoff time

Mathematical Definition

$$X \sim \text{Uniform}(a, b) \text{ or } X \sim U(a, b)$$

■ PDF

$$f_X(x) = \begin{cases} \frac{1}{b-a} & a \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$$

■ CDF

$$F_X(x) = \begin{cases} 0 & x < a \\ \frac{x-a}{b-a} & a \leq x \leq b \\ 1 & x > b \end{cases}$$

■ Mean

$$E[X] = \frac{a+b}{2}$$

■ Variance

$$\text{Var}(X) = \frac{(b-a)^2}{12}$$

■ When to Use

- ✓ All values in a range are equally likely
- ✓ No information favors any particular value
- ✓ Quantization error modeling
- ✓ Phase of unmodulated carrier

■ When NOT to Use

- ✗ Values near the center are more likely (use Gaussian)
- ✗ Values are non-negative with exponential decay (use Exponential)
- ✗ Range is unbounded

■ MATLAB Implementation

```
%% Uniform Distribution: Quantization Error
a = -0.5; b = 0.5; % Quantization step = 1 LSB
N = 100000;

X = a + (b-a)*rand(1, N); % or: unifrnd(a, b, 1, N)

figure;
histogram(X, 50, 'Normalization', 'pdf');
hold on;
plot([a a b b], [0 1/(b-a) 1/(b-a) 0], 'r-', 'LineWidth', 2);
xlabel('Quantization Error'); ylabel('f_X(x)');
title('Uniform PDF: Quantization Error');
legend('Simulated', 'Theoretical');

fprintf('Mean: Theory=%.3f, Sim=%.3f\n', (a+b)/2, mean(X));
fprintf('Var: Theory=%.4f, Sim=%.4f\n', (b-a)^2/12, var(X));
```

MATLAB

3.8 Exponential Distribution

■ Physical Interpretation

Models the **time between events** in a Poisson process, or the **lifetime** of a memoryless component.

■ Engineering Context

- ▶ Time between packet arrivals
- ▶ Time until component failure (constant failure rate)
- ▶ Time between cosmic ray events in a detector
- ▶ Inter-arrival time of calls at a switch
- ▶ Duration of a fading dip in a wireless channel

Mathematical Definition

$X \sim \text{Exponential}(\lambda)$ where λ = rate parameter (events per unit time).

Alternative parameterization: $X \sim \text{Exp}(\beta)$ where $\beta = 1/\lambda$ = mean.

■ PDF

$$f_X(x) = \lambda e^{-\lambda x}, \quad x \geq 0$$

Or with mean β : $f_X(x) = (1/\beta)e^{-x/\beta}$

■ CDF

$$F_X(x) = 1 - e^{-\lambda x}, \quad x \geq 0$$

■ Mean

$$E[X] = \frac{1}{\lambda} = \beta$$

■ Variance

$$\text{Var}(X) = \frac{1}{\lambda^2} = \beta^2$$

Note: Standard deviation = mean for exponential distribution.

■ Memoryless Property

$$P(X > s + t \mid X > s) = P(X > t)$$

The exponential is the **ONLY** continuous distribution with this property.

Engineering meaning: A component that has been working for s hours is statistically “as good as new.” Its remaining lifetime has the same distribution as a brand-new component.

■ Relationship to Poisson

If events arrive as a Poisson process with rate λ :

- ▶ Number of events in time $T \sim \text{Poisson}(\lambda T)$
- ▶ Time between events $\sim \text{Exponential}(\lambda)$

■ When to Use

- ✓ Modeling time between events in a Poisson process
- ✓ Component lifetime with constant failure rate
- ✓ Memoryless assumption is reasonable

■ When NOT to Use

- ✗ Failure rate increases with age (wear-out) — use Weibull or Gamma
- ✗ Failure rate decreases with age (burn-in) — use Weibull
- ✗ Data shows increasing hazard — NOT memoryless

■ MATLAB Implementation

```
% Exponential Distribution: Component Lifetime
lambda = 0.001;           % Failure rate: 0.001 per hour
beta = 1/lambda;          % Mean lifetime: 1000 hours

x = linspace(0, 5000, 1000);
pdf_exp = exppdf(x, beta);
cdf_exp = expcdf(x, beta);

figure;
subplot(2,1,1);
plot(x, pdf_exp, 'b-', 'LineWidth', 2);
xlabel('Time (hours)'); ylabel('f_X(x)');
title(sprintf('Exponential PDF: Mean Lifetime = %d hours', beta));
grid on;

subplot(2,1,2);
plot(x, cdf_exp, 'r-', 'LineWidth', 2);
xlabel('Time (hours)'); ylabel('F_X(x) = P(failure by time x)');
title('Exponential CDF');
grid on;

% P(component lasts more than 2000 hours)
p_survive_2000 = 1 - expcdf(2000, beta);
fprintf('P(T > 2000 hours) = %.4f\n', p_survive_2000);
```

MATLAB

■ Simulation Example: Memoryless Property Verification

MATLAB

```
% Verify Memoryless Property
beta = 100; % Mean lifetime = 100 hours
N = 100000;

X = exprnd(beta, 1, N);

% Conditional: given X > 50, what is distribution of (X - 50)?
survivors = X(X > 50);
remaining_life = survivors - 50;

fprintf('Mean of X: %.1f (should be %.1f)\n', mean(X), beta);
fprintf('Mean of remaining life given survival past 50: %.1f\n', mean(remaining_life));
fprintf('Should also be %.1f (memoryless!)\n', beta);
```

3.9 Gaussian (Normal) Distribution

■ Physical Interpretation

Models quantities that result from the **sum of many small, independent random effects**. The most important distribution in engineering due to the Central Limit Theorem.

■ Engineering Context

- ▶ Thermal noise voltage (sum of many electron contributions)
- ▶ Measurement errors (sum of many small error sources)
- ▶ Manufacturing variations (many independent process variations)
- ▶ Aggregate interference in communications
- ▶ Any quantity well-modeled by CLT

Mathematical Definition

$X \sim N(\mu, \sigma^2)$ where μ = mean, σ^2 = variance.

■ PDF

$$f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad -\infty < x < \infty$$

■ CDF

$$F_X(x) = \Phi\left(\frac{x-\mu}{\sigma}\right) = \frac{1}{2} \left[1 + \operatorname{erf}\left(\frac{x-\mu}{\sigma\sqrt{2}}\right) \right]$$

No closed-form — computed numerically or via tables.

■ Mean

$$E[X] = \mu$$

■ Variance

$$\operatorname{Var}(X) = \sigma^2$$

■ Standard Normal

$Z \sim N(0, 1)$: standardized form.

Any Normal can be standardized: $Z = (X - \mu)/\sigma$

■ Key Probability Rules (σ -rules)

Range	Probability
$\mu \pm 1\sigma$	68.27%
$\mu \pm 2\sigma$	95.45%

Range	Probability
$\mu \pm 3\sigma$	99.73%
$\mu \pm 4\sigma$	99.9937%

■ Properties

1. **Symmetric** about μ
2. **Linear transformation:** If $X \sim N(\mu, \sigma^2)$, then $aX + b \sim N(a\mu + b, a^2\sigma^2)$
3. **Sum of Gaussians:** If $X \sim N(\mu_1, \sigma_1^2)$ and $Y \sim N(\mu_2, \sigma_2^2)$ are independent, then $X + Y \sim N(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$
4. **Fully characterized** by mean and variance (all higher moments determined)

■ When to Use

- ✓ Thermal noise in circuits
- ✓ Measurement errors
- ✓ Any sum of many independent random effects (CLT)
- ✓ Manufacturing variations
- ✓ When data histogram is bell-shaped and symmetric

■ When NOT to Use

- ✗ Strictly positive quantities (Gaussian allows negative values)
- ✗ Heavily skewed data
- ✗ Bounded quantities (Gaussian has infinite support)
- ✗ Discrete counts
- ✗ Heavy-tailed phenomena (use t-distribution or Cauchy)

■ MATLAB Implementation

MATLAB

```

%% Gaussian Distribution: Thermal Noise
mu = 0;           % Zero-mean noise
sigma = 0.1;      % RMS noise = 100 mV

x = linspace(mu - 4*sigma, mu + 4*sigma, 1000);
pdf_norm = normpdf(x, mu, sigma);

figure;
plot(x, pdf_norm, 'b-', 'LineWidth', 2);
hold on;
% Shade the  $\pm 2\sigma$  region
x_fill = linspace(mu-2*sigma, mu+2*sigma, 200);
fill([x_fill, fliplr(x_fill)], [normpdf(x_fill,mu,sigma), zeros(size(x_fill))], ...
    'b', 'FaceAlpha', 0.3);
xlabel('Noise Voltage (V)');
ylabel('f_X(x)');
title(sprintf('Gaussian PDF:  $\mu$ =%.1f,  $\sigma$ =%.3f V', mu, sigma));
legend('PDF', ' $\pm 2\sigma$  region (95.45%)');
grid on;

% Probability computations
fprintf('P(|noise| > 0.2V) = %.4f\n', 2*(1-normcdf(0.2, 0, sigma)));
fprintf('P(|noise| > 0.3V) = %.4f\n', 2*(1-normcdf(0.3, 0, sigma)));

```

■ Simulation Example: Q-Function and BER

MATLAB

```

%% Engineering Application: Binary Communication BER
% In BPSK with AWGN: BER = Q(sqrt(2*Eb/N0))
% Q(x) = 1 - Phi(x) = probability that N(0,1) > x

EbN0_dB = 0:0.5:12;
EbN0 = 10.^(EbN0_dB/10);

% Theoretical BER
BER_theory = qfunc(sqrt(2*EbN0));

% Simulated BER
N_bits = 1e6;
BER_sim = zeros(size(EbN0_dB));
for idx = 1:length(EbN0)
    % BPSK: transmit ±1
    bits = 2*randi([0,1], 1, N_bits) - 1;
    noise_std = 1/sqrt(2*EbN0(idx));
    received = bits + noise_std * randn(1, N_bits);
    decisions = sign(received);
    BER_sim(idx) = mean(decisions ~= bits);
end

figure;
semilogy(EbN0_dB, BER_theory, 'b-', 'LineWidth', 2);
hold on;
semilogy(EbN0_dB, BER_sim, 'ro', 'MarkerSize', 6);
xlabel('E_b/N_0 (dB)'); ylabel('Bit Error Rate');
title('BPSK BER: Theory vs Simulation');
legend('Theory: Q(sqrt(2E_b/N_0))', 'Simulation');
grid on;

```

3.10 Gamma Distribution

■ Physical Interpretation

Models the **total waiting time until the k-th event** in a Poisson process, or any sum of k independent exponential random variables.

■ Engineering Context

- ▶ Total repair time for k components (each with exponential repair time)
- ▶ Time until k-th packet arrival
- ▶ Aggregate service time for k queued jobs
- ▶ Rainfall amount modeling

Mathematical Definition

$X \sim \text{Gamma}(\alpha, \beta)$ where α = shape parameter, β = scale parameter (or rate $\lambda = 1/\beta$).

■ PDF

$$f_X(x) = \frac{x^{\alpha-1} e^{-x/\beta}}{\beta^\alpha \Gamma(\alpha)}, \quad x \geq 0$$

where $\Gamma(\alpha) = (\alpha-1)!$ for integer α , and $\Gamma(\alpha) = \int_0^\infty t^{(\alpha-1)} e^{-t} dt$ in general.

■ CDF

No closed form for general α . Available via incomplete gamma function.

■ Mean

$$E[X] = \alpha\beta$$

■ Variance

$$\text{Var}(X) = \alpha\beta^2$$

■ Special Cases

Parameters	Distribution
$\alpha = 1$	Exponential(β)
$\alpha = n/2, \beta = 2$	Chi-square(n)
$\alpha = \text{integer } k$	Erlang(k, β)

■ When to Use

- ✓ Sum of independent exponential lifetimes
- ✓ Waiting time for k-th Poisson event

- ✓ Skewed, non-negative data
- ✓ Flexible shape (adjustable skewness)

■ When NOT to Use

- ✗ Data can be negative
- ✗ Data is symmetric (use Gaussian)
- ✗ Simpler distribution fits (exponential for $k=1$)

■ MATLAB Implementation

```
%% Gamma Distribution: Total Repair Time
% 3 components must be repaired sequentially
% Each repair time ~ Exp(mean = 2 hours)
alpha = 3;           % shape (number of stages)
beta = 2;            % scale (mean per stage)

x = linspace(0, 20, 500);
pdf_gamma = gampdf(x, alpha, beta);

figure;
plot(x, pdf_gamma, 'b-', 'LineWidth', 2);
xlabel('Total Repair Time (hours)');
ylabel('f_X(x)');
title(sprintf('Gamma PDF:  $\alpha$ =%d,  $\beta$ =%d', alpha, beta));
grid on;

% Verify by summing exponentials
N = 100000;
X_sim = sum(exprnd(beta, alpha, N), 1); % Sum of 3 exponentials
fprintf('Mean: Theory=%.2f, Sim=%.2f\n', alpha*beta, mean(X_sim));
fprintf('Var: Theory=%.2f, Sim=%.2f\n', alpha*beta^2, var(X_sim));
```

MATLAB

3.11 Rayleigh Distribution

■ Physical Interpretation

Models the **magnitude (envelope)** of a 2D Gaussian random vector. When X and Y are independent $N(0, \sigma^2)$, then $R = \sqrt{X^2 + Y^2}$ follows a Rayleigh distribution.

■ Engineering Context

- ▶ Envelope of narrowband noise (in-phase + quadrature)
- ▶ Magnitude of wireless fading channel coefficient (Rayleigh fading)
- ▶ Wind speed modeling
- ▶ Distance error in 2D positioning (when errors are Gaussian in x and y)

Mathematical Definition

$R \sim \text{Rayleigh}(\sigma)$, where σ is the parameter (NOT the standard deviation of R).

■ PDF

$$f_R(r) = \frac{r}{\sigma^2} e^{-r^2/(2\sigma^2)}, \quad r \geq 0$$

■ CDF

$$F_R(r) = 1 - e^{-r^2/(2\sigma^2)}, \quad r \geq 0$$

■ Mean

$$E[R] = \sigma \sqrt{\frac{\pi}{2}} \approx 1.2533\sigma$$

■ Variance

$$\text{Var}(R) = \frac{4 - \pi}{2} \sigma^2 \approx 0.4292\sigma^2$$

■ Relationship to Other Distributions

- ▶ $R^2 \sim \text{Exponential}(2\sigma^2)$ — the power is exponentially distributed
- ▶ If $X, Y \sim N(0, \sigma^2)$ independent, then $\sqrt{X^2 + Y^2} \sim \text{Rayleigh}(\sigma)$

■ When to Use

- ✓ Envelope/magnitude of complex Gaussian signal
- ✓ Rayleigh fading channel modeling (no line-of-sight)
- ✓ 2D distance with Gaussian errors in each dimension
- ✓ RSS (root-sum-square) of two independent Gaussian components

■ When NOT to Use

- ✗ Strong line-of-sight component present (use Rician)
- ✗ Not modeling a magnitude/envelope
- ✗ Components are not Gaussian or not equal-variance

■ MATLAB Implementation

```

%% Rayleigh Distribution: Wireless Fading Channel
sigma = 1; % Rayleigh parameter
r = linspace(0, 5, 500);

pdf_ray = raylpdf(r, sigma);
cdf_ray = raylcdf(r, sigma);

figure;
subplot(2,1,1);
plot(r, pdf_ray, 'b-', 'LineWidth', 2);
xlabel('Channel Gain |h|'); ylabel('f_R(r)');
title(sprintf('Rayleigh PDF ( $\sigma = %d$ )', sigma));
grid on;

subplot(2,1,2);
plot(r, cdf_ray, 'r-', 'LineWidth', 2);
xlabel('r'); ylabel('P(R ≤ r)');
title('Rayleigh CDF = Outage Probability');
grid on;

% Outage probability: P(|h| < threshold)
threshold = 0.5;
p_outage = raylcdf(threshold, sigma);
fprintf('P(outage, |h| < %.1f) = %.4f\n', threshold, p_outage);

```

MATLAB

■ Simulation Example: Rayleigh Fading

```

%% Generate Rayleigh Fading from Complex Gaussian
sigma = 1;
N = 100000;

% Complex Gaussian channel coefficient
h = sigma/sqrt(2) * (randn(1,N) + 1j*randn(1,N));

% Magnitude is Rayleigh distributed
R = abs(h);

figure;
histogram(R, 100, 'Normalization', 'pdf');
hold on;
r = linspace(0, 5, 300);
plot(r, raylpdf(r, sigma/sqrt(2)), 'r-', 'LineWidth', 2);
xlabel('|h|'); ylabel('PDF');
title('Rayleigh Fading: Simulated vs Theory');
legend('Simulation', 'Theory');

```

MATLAB

3.12 Chi-Square Distribution

■ Physical Interpretation

Models the **sum of squares of independent standard normal** random variables. Arises naturally in variance estimation and goodness-of-fit testing.

■ Engineering Context

- ▶ Sample variance distribution (for Gaussian data)
- ▶ Power of Gaussian noise (sum of squared components)
- ▶ Goodness-of-fit test statistic
- ▶ Confidence intervals for variance
- ▶ Energy detection in spectrum sensing

Mathematical Definition

If $Z_1, Z_2, \dots, Z_n$ are independent $N(0,1)$, then:

$$\chi^2 = Z_1^2 + Z_2^2 + \dots + Z_n^2 \sim \chi^2(n)$$

n = degrees of freedom.

■ PDF

$$f_X(x) = \frac{x^{n/2-1} e^{-x/2}}{2^{n/2} \Gamma(n/2)}, \quad x \geq 0$$

■ CDF

No closed form for general n . Computed via incomplete gamma function.

■ Mean

$$E[X] = n$$

■ Variance

$$\text{Var}(X) = 2n$$

■ Relationship to Other Distributions

- ▶ Chi-square(n) = Gamma($n/2, 2$)
- ▶ Chi-square(1) = square of $N(0,1)$
- ▶ Chi-square(2) = Exponential(1/2)
- ▶ For large n : $\chi^2(n) \approx N(n, 2n)$ by CLT

■ When to Use

- ✓ Sum of squared Gaussian random variables
- ✓ Variance testing and confidence intervals
- ✓ Goodness-of-fit tests
- ✓ Energy detection in N dimensions

■ When NOT to Use

- ✗ Variables are not Gaussian
- ✗ Variables are not independent
- ✗ Variables are not standard normal (need scaling)

■ MATLAB Implementation

MATLAB

```

%% Chi-Square Distribution: Noise Power
% Noise power from N complex dimensions = sum of 2N squared Gaussians

n_dof = 10;           % degrees of freedom
x = linspace(0, 30, 500);

pdf_chi2 = chi2pdf(x, n_dof);
cdf_chi2 = chi2cdf(x, n_dof);

figure;
plot(x, pdf_chi2, 'b-', 'LineWidth', 2);
xlabel('χ² Value (Noise Power)');
ylabel('f_X(x)');
title(sprintf('Chi-Square PDF: %d degrees of freedom', n_dof));
grid on;

% Simulation: sum of squared standard normals
N = 100000;
Z = randn(n_dof, N);
X_sim = sum(Z.^2, 1);

fprintf('Mean: Theory=%d, Sim=%.2f\n', n_dof, mean(X_sim));
fprintf('Var: Theory=%d, Sim=%.2f\n', 2*n_dof, var(X_sim));

```

3.13 Distribution Selection Guide

■ Decision Tree: Discrete Distributions

Is the random variable discrete (countable outcomes)?

- ├ Only two outcomes (success/failure)?
 - └ YES → Bernoulli(p)
- ├ Fixed n trials, counting successes?
 - └ YES → Binomial(n, p)
- ├ Counting trials until FIRST success?
 - └ YES → Geometric(p)
- ├ Counting trials until r -th success?
 - └ YES → Negative Binomial(r, p)
- └ Counting events in a fixed interval?
 - └ Events independent and rare? → Poisson(λ)

■ Decision Tree: Continuous Distributions

Is the random variable continuous?

- ├ Equally likely over a bounded range?
 - └ YES → Uniform(a, b)
- ├ Time between events (memoryless)?
 - └ YES → Exponential(λ)
- ├ Sum of many small independent effects?
 - └ YES → Gaussian(μ, σ^2)
- ├ Total time for k events (sum of exponentials)?
 - └ YES → Gamma(k, β)
- ├ Magnitude of 2D Gaussian vector?
 - └ YES → Rayleigh(σ)
- └ Sum of squared Gaussians?
 - └ YES → Chi-square(n)

■ Quick Recognition Patterns

If you see...	Think...
Binary outcome	Bernoulli
" n trials, k successes"	Binomial
"How many until first..."	Geometric
"Events per interval"	Poisson
"Equal chance anywhere in $[a, b]$ "	Uniform

If you see...	Think...
“Time until next event”	Exponential
“Sum of many effects” or “noise”	Gaussian
“Total waiting time for k events”	Gamma
“Signal envelope” or “fading”	Rayleigh
“Sum of squared normals”	Chi-square

3.14 Summary Table

Distribution	Type	Parameters	Mean	Variance	MATLAB
Bernoulli	Discrete	p	p	$p(1-p)$	<code>binornd(1,p)</code>
Binomial	Discrete	n, p	np	$np(1-p)$	<code>binornd(n,p)</code>
Geometric	Discrete	p	$1/p$	$(1-p)/p^2$	<code>geornd(p)+1</code>
Neg. Binomial	Discrete	r, p	r/p	$r(1-p)/p^2$	<code>nbinrnd(r,p)+r</code>
Poisson	Discrete	λ	λ	λ	<code>poissrnd(λ)</code>
Uniform	Continuous	a, b	$(a+b)/2$	$(b-a)^2/12$	<code>unifrnd(a,b)</code>
Exponential	Continuous	λ (or $\beta=1/\lambda$)	$1/\lambda$	$1/\lambda^2$	<code>exprnd(β)</code>
Gaussian	Continuous	μ, σ^2	μ	σ^2	<code>normrnd(μ, σ)</code>
Gamma	Continuous	α, β	$\alpha\beta$	$\alpha\beta^2$	<code>gamrnd(α, β)</code>
Rayleigh	Continuous	σ	$\sigma\sqrt{\pi/2}$	$(4-\pi)\sigma^2/2$	<code>raylrnd(σ)</code>
Chi-square	Continuous	n	n	$2n$	<code>chi2rnd(n)</code>

3.15 Practice Problems

Problem 1: Distribution Identification

Identify the appropriate distribution for each scenario:

- (a) Number of dropped calls in a 10-minute window (average: 2 per 10 min)
- (b) Voltage of thermal noise across a resistor
- (c) Time until a server crashes (constant failure rate)
- (d) Number of defective ICs out of a batch of 100 (defect rate 3%)
- (e) Envelope of received signal through multipath with no LOS

Solution (a) Poisson($\lambda=2$), (b) Gaussian($0, \sigma^2$), (c) Exponential(λ), (d) Binomial(100, 0.03), (e) Rayleigh(σ)

Problem 2: Exponential vs Poisson

Packets arrive at a router as a Poisson process with rate $\lambda = 5$ packets/second.

- (a) What is $P(\text{more than 8 packets in 1 second})$?
- (b) What is $P(\text{no packet arrives in the next 0.5 seconds})$?
- (c) What distribution describes the inter-arrival time?
- (d) What is the mean inter-arrival time?

Solution

- (a) $X \sim \text{Poisson}(5)$: $P(X > 8) = 1 - \text{poisscdf}(8, 5) = 0.0681$
- (b) $T \sim \text{Exp}(5)$: $P(T > 0.5) = e^{-5 \times 0.5} = e^{-2.5} = 0.0821$
- (c) Exponential with rate $\lambda = 5$
- (d) Mean = $1/\lambda = 0.2$ seconds

Problem 3: Gaussian Probability

A resistor has nominal value 1000Ω with manufacturing variation modeled as $N(1000, 25)$ ($\sigma = 5\Omega$).

- (a) What fraction of resistors are within $\pm 10\Omega$ of nominal (spec: 990-1010 Ω)?
- (b) What fraction fail the $\pm 1\%$ tolerance test?
- (c) What tolerance range contains 99.7% of resistors?

Solution

- (a) $P(990 \leq X \leq 1010) = P(-2 \leq Z \leq 2) = 0.9545 \rightarrow 95.45\%$
- (b) $\pm 1\% = \pm 10\Omega \rightarrow$ same as (a), fail rate = $1 - 0.9545 = 4.55\%$
- (c) $\pm 3\sigma = \pm 15\Omega \rightarrow$ range $[985, 1015]\Omega$

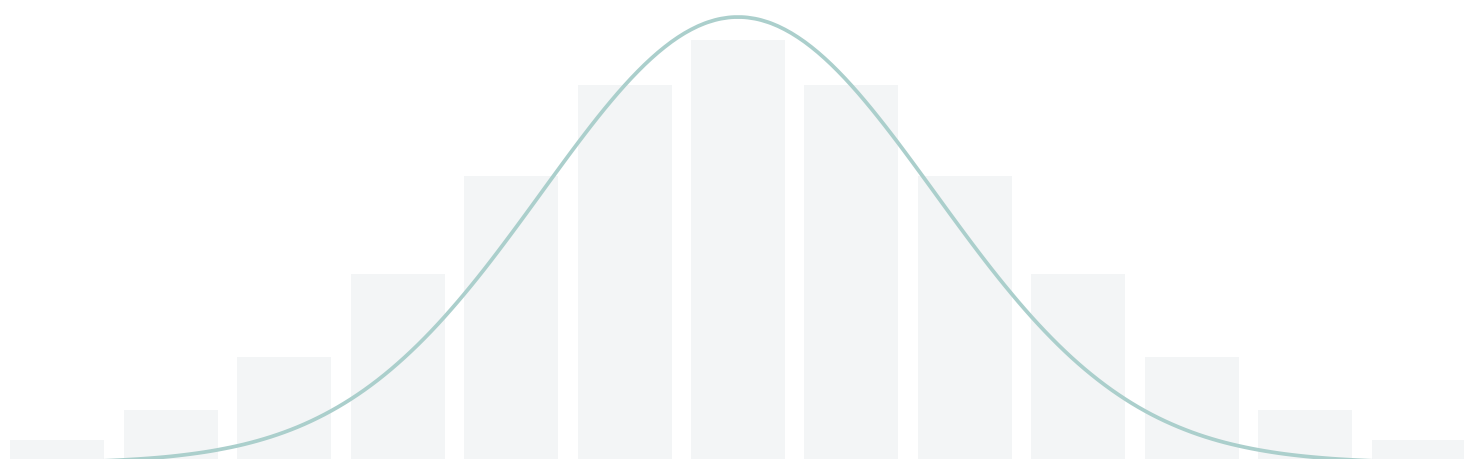
Next Module: Module 4 — Expectation and Moments

Module 4

Expectation and Moments

امید ریاضی و گشتاورها

4



In This Module

4.1	Introduction: Why Summary Statistics?	96
4.2	Expected Value (Mean)	97
4.3	Expected Value of a Function: $E[g(X)]$	98
4.4	Variance	99
4.5	Standard Deviation	100
4.6	Properties of Expectation	101
4.7	Properties of Variance	102
4.8	Higher-Order Moments	103
4.9	Moment Generating Function (MGF)	104
4.10	Engineering Applications of Moments	106
4.11	MATLAB Examples	107
4.12	Practice Problems	110
4.13	Key Takeaways	112

Learning Objectives

After completing this module, students will be able to:

1. Compute expected values for discrete and continuous random variables
2. Calculate variance and standard deviation and interpret their engineering meaning
3. Apply properties of expectation and variance in engineering analysis
4. Compute and interpret higher-order moments
5. Use the moment generating function (MGF)
6. Relate statistical moments to physical engineering quantities

4.1 Introduction: Why Summary Statistics?

■ The Engineering Need

A probability distribution contains ALL information about a random variable. But for engineering decisions, we often need concise summaries:

- ▶ **Signal processing:** What is the average power? → Mean of X^2
- ▶ **Quality control:** How much do components vary? → Variance
- ▶ **Communication:** What is the noise power? → $E[N^2]$
- ▶ **Reliability:** What is the average lifetime? → Mean

Moments (mean, variance, etc.) are the essential **engineering-relevant summaries** of a distribution.

4.2 Expected Value (Mean)

Definition — Discrete RV

$$E[X] = \mu_X = \sum_x x \cdot p_X(x)$$

Sum of (each value $\times$ its probability).

Definition — Continuous RV

$$E[X] = \mu_X = \int_{-\infty}^{\infty} x \cdot f_X(x) dx$$

■ Engineering Interpretation

The expected value is the **long-run average**: if you repeat the experiment many times and average the results, you get approximately $E[X]$.

Engineering Quantity	Expected Value Meaning
Noise voltage $E[N]$	DC bias/offset in noise
Signal amplitude $E[A]$	Average signal level
Component lifetime $E[T]$	Mean time to failure (MTTF)
Packet delay $E[D]$	Average delay
Bit errors $E[X]$	Average error count

Example: Average Noise Power

For zero-mean Gaussian noise $N \sim N(0, \sigma^2)$:

- $E[N] = 0$ (no DC component)
- Average power = $E[N^2] = \sigma^2$ (this is the variance!)

Example: Expected Number of Retransmissions

ARQ protocol: each transmission succeeds with probability $p = 0.8$.

Number of attempts $X \sim \text{Geometric}(0.8)$.

$E[X] = 1/p = 1/0.8 = 1.25$ attempts on average.

4.3 Expected Value of a Function: $E[g(X)]$

■ The LOTUS Theorem (Law of the Unconscious Statistician)

To find $E[g(X)]$, you do NOT need the distribution of $Y = g(X)$:

Discrete:

$$E[g(X)] = \sum_x g(x) \cdot p_X(x)$$

Continuous:

$$E[g(X)] = \int_{-\infty}^{\infty} g(x) \cdot f_X(x) dx$$

■ Engineering Applications

Function $g(X)$	Engineering Meaning
$g(X) = X^2$	Mean square value (power)
$g(X) = (X - \mu)^2$	Variance (AC power)
$g(X) =$	X
$g(X) = e^{j\omega X}$	Characteristic function

Example: Mean Power of a Signal

Signal X has PDF $f_X(x)$. Power $P = X^2$.

$$E[P] = E[X^2] = \int x^2 f_X(x) dx$$

For $X \sim N(0, \sigma^2)$: $E[X^2] = \sigma^2$ (the noise power equals the variance).

For $X \sim N(\mu, \sigma^2)$: $E[X^2] = \mu^2 + \sigma^2$ (total power = DC power + AC power).

4.4 Variance

Definition

$$\text{Var}(X) = E[(X - \mu_X)^2] = E[X^2] - (E[X])^2$$

The second formula (computational formula) is often easier to use.

■ Derivation of Computational Formula

$$\begin{aligned}\text{Var}(X) &= E[(X - \mu)^2] \\ &= E[X^2 - 2\mu X + \mu^2] \\ &= E[X^2] - 2\mu E[X] + \mu^2 \\ &= E[X^2] - 2\mu^2 + \mu^2 \\ &= E[X^2] - \mu^2\end{aligned}$$

■ Engineering Interpretation

The variance measures **spread/dispersion/uncertainty**:

Context	Variance Meaning
Noise signal	Noise power (for zero-mean noise)
Manufacturing	How much components deviate from nominal
Measurements	Measurement precision (smaller = more precise)
Signal	AC power component
Lifetime	How predictable is the lifetime?

■ Key Insight: Variance = Power in Engineering

For a zero-mean signal X :

- ▶ $E[X^2] = \text{Var}(X) = \sigma^2 = \text{total power}$
- ▶ For $X \sim N(0, \sigma^2)$: noise power = $\sigma^2 = \text{Var}(X)$

For a signal with DC offset ($\mu \neq 0$):

- ▶ $E[X^2] = \mu^2 + \sigma^2 = \text{total power} = \text{DC power} + \text{AC power}$
- ▶ $\text{Var}(X) = \sigma^2 = \text{AC power only}$

4.5 Standard Deviation

Definition

$$\sigma_X = \sqrt{\text{Var}(X)}$$

■ Engineering Significance

The standard deviation has the **same units** as X , making it directly interpretable:

Random Variable	Standard Deviation Unit	Interpretation
Voltage noise	Volts	RMS noise voltage
Timing jitter	Seconds	RMS jitter
Distance error	Meters	RMS position error
Temperature	°C	RMS temperature fluctuation

The standard deviation is the **RMS (root-mean-square)** value for zero-mean signals.

4.6 Properties of Expectation

■ Linearity (FUNDAMENTAL)

$$E[aX + b] = aE[X] + b$$

$$E[X + Y] = E[X] + E[Y] \quad (\text{ALWAYS, even if dependent!})$$

$$E\left[\sum_{i=1}^n a_i X_i\right] = \sum_{i=1}^n a_i E[X_i]$$

■ Monotonicity

If $X \geq 0$ always, then $E[X] \geq 0$.

If $X \geq Y$ always, then $E[X] \geq E[Y]$.

■ For Independent RVs

If X and Y are independent:

$$E[XY] = E[X] \cdot E[Y]$$

▲ This does NOT hold for dependent variables!

Engineering Example: Total Noise

A received signal: $R = S + N_1 + N_2$ (signal plus two noise sources)

$$E[R] = E[S] + E[N_1] + E[N_2] = E[S] + 0 + 0 = E[S]$$

(Zero-mean noise doesn't bias the expected received signal.)

4.7 Properties of Variance

■ Scaling and Shift

$$\text{Var}(aX + b) = a^2 \text{Var}(X)$$

- ▶ Adding a constant b does NOT change variance (shifting doesn't change spread)
- ▶ Scaling by a multiplies variance by a^2

■ Sum of Independent RVs

If $X_1, X_2, \dots, X_n$ are **independent**:

$$\text{Var}(X_1 + X_2 + \dots + X_n) = \text{Var}(X_1) + \text{Var}(X_2) + \dots + \text{Var}(X_n)$$

▲ Variances add for INDEPENDENT variables only!

Engineering Example: Noise Power Addition

Two independent noise sources ($\sigma_1^2 = 0.01 \text{ V}^2$, $\sigma_2^2 = 0.04 \text{ V}^2$):

Total noise power = $\sigma_1^2 + \sigma_2^2 = 0.05 \text{ V}^2$

Total RMS noise = $\sqrt{0.05} = 0.224 \text{ V}$

Note: RMS values do NOT add directly! ($0.1 + 0.2 \neq 0.224$)

■ Variance of Sample Mean

For n i.i.d. observations $X_1, \dots, X_n$ with variance σ^2 :

$$\text{Var}(\bar{X}) = \text{Var}\left(\frac{1}{n} \sum X_i\right) = \frac{\sigma^2}{n}$$

More measurements $\rightarrow$ lower uncertainty in the average.

4.8 Higher-Order Moments

■ n-th Moment

$$\mu'_n = E[X^n] = \int_{-\infty}^{\infty} x^n f_X(x) dx$$

■ n-th Central Moment

$$\mu_n = E[(X - \mu)^n]$$

- ▶ $\mu_1 = 0$ (always)
- ▶ $\mu_2 = \text{Var}(X)$
- ▶ μ_3 relates to **skewness**
- ▶ μ_4 relates to **kurtosis**

■ Skewness (3rd Central Moment, Normalized)

$$\gamma_1 = \frac{E[(X - \mu)^3]}{\sigma^3}$$

Skewness	Shape	Example
$\gamma_1 = 0$	Symmetric	Gaussian
$\gamma_1 > 0$	Right-skewed (long right tail)	Exponential
$\gamma_1 < 0$	Left-skewed (long left tail)	Negated exponential

Engineering significance: Skewness tells us if extreme values are more likely in one direction. Important for reliability (failure time distributions are right-skewed).

■ Kurtosis (4th Central Moment, Normalized)

$$\gamma_2 = \frac{E[(X - \mu)^4]}{\sigma^4} - 3$$

(Subtracting 3 makes Gaussian have kurtosis 0: “excess kurtosis.”)

Kurtosis	Interpretation	Meaning
$\gamma_2 = 0$	Mesokurtic	Gaussian-like tails
$\gamma_2 > 0$	Leptokurtic	Heavier tails than Gaussian
$\gamma_2 < 0$	Platykurtic	Lighter tails than Gaussian

Engineering significance: High kurtosis means more outliers — important for impulsive noise modeling.

4.9 Moment Generating Function (MGF)

Definition

$$M_X(t) = E[e^{tX}]$$

Discrete: $M_X(t) = \sum e^{tx} p_X(x)$

Continuous: $M_X(t) = \int e^{tx} f_X(x) dx$

■ Why “Moment Generating”?

The n-th moment is obtained by differentiating n times at $t = 0$:

$$E[X^n] = \left. \frac{d^n M_X(t)}{dt^n} \right|_{t=0}$$

- ▶ $M_X'(0) = E[X]$
- ▶ $M_X''(0) = E[X^2]$
- ▶ $\text{Var}(X) = M_X''(0) - [M_X'(0)]^2$

■ Properties

1. **Uniqueness:** If $M_X(t) = M_Y(t)$ for all t in a neighborhood of 0, then X and Y have the same distribution.
2. **Sum of Independent RVs:** If X and Y are independent:
 $M_{X+Y}(t) = M_X(t) \cdot M_Y(t)$
3. **Linear transformation:** $M_{aX+b}(t) = e^{bt} \cdot M_X(at)$

■ MGFs of Common Distributions

Distribution	MGF $M_X(t)$
Bernoulli(p)	$1 - p + pe^t$
Binomial(n, p)	$(1 - p + pe^t)^n$
Poisson(λ)	$\exp(\lambda(e^t - 1))$
Exponential(λ)	$\lambda/(\lambda - t), t < \lambda$
Gaussian(μ, σ^2)	$\exp(\mu t + \sigma^2 t^2/2)$

■ Engineering Application: Sum of Independent Signals

If signal components $X_1 \sim N(\mu_1, \sigma_1^2)$ and $X_2 \sim N(\mu_2, \sigma_2^2)$ are independent:

$$M_{X_1 + X_2}(t) = M_{X_1}(t) \cdot M_{X_2}(t)$$

$$\begin{aligned}
 &= \exp(\mu_1 t + \sigma_1^2 t^2/2) \cdot \exp(\mu_2 t + \sigma_2^2 t^2/2) \\
 &= \exp((\mu_1 + \mu_2)t + (\sigma_1^2 + \sigma_2^2)t^2/2)
 \end{aligned}$$

This is the MGF of $N(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$. Therefore $X_1 + X_2 \sim N(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$.

4.10 Engineering Applications of Moments

■ Application 1: Noise Power Characterization

A noise signal $N(t)$ is sampled. The samples have mean μ_N and variance σ_N^2 .

- ▶ **DC component (bias):** μ_N — should be zero for ideal noise
- ▶ **AC noise power:** σ_N^2 — determines SNR
- ▶ **Total power:** $E[N^2] = \mu_N^2 + \sigma_N^2$

SNR (Signal-to-Noise Ratio):

$$\text{SNR} = \frac{E[S^2]}{\sigma_N^2} = \frac{\text{Signal Power}}{\text{Noise Power}}$$

■ Application 2: Measurement Error

A sensor measures true value θ with error E :

- ▶ Measurement: $X = \theta + E$
- ▶ $E[X] = \theta + E[E]$ — bias in measurement
- ▶ $\text{Var}(X) = \text{Var}(E)$ — measurement precision

Accuracy: $|E[E]|$ (how close the average is to truth)

Precision: σ_E (how repeatable measurements are)

■ Application 3: Component Lifetime Statistics

Components have lifetime $T \sim \text{Exponential}(\lambda)$:

- ▶ Mean Time To Failure: $\text{MTTF} = E[T] = 1/\lambda$
- ▶ Standard Deviation: $\sigma_T = 1/\lambda = \text{MTTF}$
- ▶ For exponential: coefficient of variation $\text{CV} = \sigma/\mu = 1$ (always!)

■ Application 4: Communication Signal Amplitude

Received signal $R = A + N$ where A = signal amplitude, $N \sim N(0, \sigma^2)$:

- ▶ $E[R] = A$ (signal amplitude preserved on average)
- ▶ $\text{Var}(R) = \sigma^2$ (noise power determines variance)
- ▶ $\text{SNR} = A^2/\sigma^2$

4.11 MATLAB Examples

Example 1: Computing Moments from a Distribution

MATLAB

```
%% Moments of Common Distributions

% Exponential: failure time, lambda = 0.01 (per hour)
lambda = 0.01;
beta = 1/lambda; % mean = 100 hours

fprintf('=== Exponential Distribution ( $\lambda = %.3f$ ) ===\n', lambda);
fprintf('Mean (MTTF) = %.1f hours\n', beta);
fprintf('Variance = %.1f hours2\n', beta^2);
fprintf('Std Dev = %.1f hours\n', beta);
fprintf('Coefficient of Variation = %.2f\n', beta/beta);

% Gaussian: noise voltage, sigma = 0.1V
mu = 0; sigma = 0.1;
fprintf('\n=== Gaussian Distribution ( $\mu=%.1f$ ,  $\sigma=%.2f$  V) ===\n', mu, sigma);
fprintf('Mean = %.2f V\n', mu);
fprintf('Variance (noise power) = %.4f V2\n', sigma^2);
fprintf('RMS noise = %.3f V\n', sigma);
fprintf('E[X2] (total power) = %.4f V2\n', mu^2 + sigma^2);
```

Example 2: Verifying Variance Properties

MATLAB

```
%% Variance of Sum of Independent RVs
N = 100000;

% Two independent noise sources
sigma1 = 0.1; % RMS of noise 1
sigma2 = 0.2; % RMS of noise 2

X1 = sigma1 * randn(1, N);
X2 = sigma2 * randn(1, N);
Y = X1 + X2; % Combined noise

fprintf('Var(X1) = %.4f (theory: %.4f)\n', var(X1), sigma1^2);
fprintf('Var(X2) = %.4f (theory: %.4f)\n', var(X2), sigma2^2);
fprintf('Var(X1+X2) = %.4f (theory: %.4f)\n', var(Y), sigma1^2 + sigma2^2);
fprintf('Std(X1+X2) = %.4f (theory: %.4f)\n', std(Y), sqrt(sigma1^2+sigma2^2));
fprintf('\nNote: std devs do NOT add: %.3f + %.3f = %.3f  $\neq$  %.3f\n', ...
    sigma1, sigma2, sigma1+sigma2, sqrt(sigma1^2+sigma2^2));
```

Example 3: Mean and Variance of Transformed RV

MATLAB

```

%% E[g(X)] using LOTUS: Power from Voltage
% X = voltage ~ N(0, 0.5V)
% P = X^2 / R, with R = 50 ohms

sigma = 0.5;    % noise voltage RMS
R = 50;         % resistance
N = 100000;

X = sigma * randn(1, N);
P = X.^2 / R;   % instantaneous power

% Using LOTUS: E[X^2/R] = E[X^2]/R = sigma^2/R
E_P_theory = sigma^2 / R;
E_P_sim = mean(P);

fprintf('Mean power: Theory = %.6f W, Sim = %.6f W\n', E_P_theory, E_P_sim);
fprintf('Mean power = %.3f mW\n', E_P_theory * 1000);

```

Example 4: Higher Moments and Skewness

MATLAB

```

%% Skewness and Kurtosis Comparison
N = 100000;

% Gaussian (symmetric)
X_gauss = randn(1, N);

% Exponential (right-skewed)
X_exp = exprnd(1, 1, N);

% Uniform (symmetric, light tails)
X_unif = rand(1, N);

fprintf('Distribution      | Skewness | Kurtosis\n');
fprintf('-----+-----+-----\n');
fprintf('Gaussian      | %+.3f   | %+.3f\n', skewness(X_gauss),
    ↪ kurtosis(X_gauss)-3);
fprintf('Exponential   | %+.3f   | %+.3f\n', skewness(X_exp), kurtosis(X_exp)-3);
fprintf('Uniform       | %+.3f   | %+.3f\n', skewness(X_unif), kurtosis(X_unif)-3);
fprintf('\nTheoretical:\n');
fprintf('Gaussian:     skew=0, kurt=0\n');
fprintf('Exponential:  skew=2, kurt=6\n');
fprintf('Uniform:      skew=0, kurt=-1.2\n');

```

Example 5: MGF Verification

MATLAB

```

%% Verify MGF: Compute moments by differentiation (numerical)
% For  $X \sim \text{Exponential}(\lambda)$ 
lambda_rate = 2;
beta_scale = 1/lambda_rate;

% MGF:  $M(t) = \lambda/(\lambda - t)$ 
%  $M'(0) = E[X]$ ,  $M''(0) = E[X^2]$ 

t = 0;
dt = 1e-6;

% Numerical derivatives
M = @(t) lambda_rate ./ (lambda_rate - t);
M_prime = (M(t+dt) - M(t-dt)) / (2*dt);
M_double_prime = (M(t+dt) - 2*M(t) + M(t-dt)) / dt^2;

fprintf('From MGF derivatives:\n');
fprintf('   $E[X] = M'(0) = %.4f$  (theory:  $%.4f$ )\n', M_prime, 1/lambda_rate);
fprintf('   $E[X^2] = M''(0) = %.4f$  (theory:  $%.4f$ )\n', M_double_prime,
    ↪ 2/lambda_rate^2);
fprintf('   $\text{Var}(X) = E[X^2] - (E[X])^2 = %.4f$  (theory:  $%.4f$ )\n', ...
    M_double_prime - M_prime^2, 1/lambda_rate^2);

```

4.12 Practice Problems

Problem 1: Basic Computation

A discrete RV X has PMF: $P(X=-1) = 0.2$, $P(X=0) = 0.5$, $P(X=1) = 0.2$, $P(X=2) = 0.1$.

- (a) Find $E[X]$.
- (b) Find $E[X^2]$.
- (c) Find $\text{Var}(X)$.
- (d) Find $E[3X + 2]$.
- (e) Find $\text{Var}(3X + 2)$.

Solution

- (a) $E[X] = (-1)(0.2) + (0)(0.5) + (1)(0.2) + (2)(0.1) = -0.2 + 0 + 0.2 + 0.2 = 0.2$
- (b) $E[X^2] = (1)(0.2) + (0)(0.5) + (1)(0.2) + (4)(0.1) = 0.2 + 0 + 0.2 + 0.4 = 0.8$
- (c) $\text{Var}(X) = E[X^2] - (E[X])^2 = 0.8 - 0.04 = 0.76$
- (d) $E[3X + 2] = 3E[X] + 2 = 3(0.2) + 2 = 2.6$
- (e) $\text{Var}(3X + 2) = 9 \cdot \text{Var}(X) = 9(0.76) = 6.84$

Problem 2: Signal and Noise

A received signal $R = 2.5 + N$, where $N \sim N(0, 0.04)$.

- (a) Find $E[R]$ and $\text{Var}(R)$.
- (b) Find the SNR in dB.
- (c) Find $P(R < 0)$ — probability the signal goes negative.

Solution

- (a) $E[R] = 2.5$, $\text{Var}(R) = \text{Var}(N) = 0.04$, $\sigma_R = 0.2$
- (b) Signal power = $2.5^2 = 6.25$, Noise power = 0.04
 $\text{SNR} = 6.25/0.04 = 156.25 \rightarrow \text{SNR}_{\text{dB}} = 10 \cdot \log_{10}(156.25) = 21.94 \text{ dB}$
- (c) $P(R < 0) = P(N < -2.5) = \Phi(-2.5/0.2) = \Phi(-12.5) \approx 0$ (extremely unlikely)

Problem 3: Averaging Measurements

A sensor makes n independent measurements of a constant voltage $V = 3.3\text{V}$.
Each measurement: $X_i = 3.3 + N_i$ where $N_i \sim N(0, 0.01)$.

- (a) What is $E[\bar{X}]$ and $\text{Var}(\bar{X})$ for $n = 1$?
- (b) How many measurements n needed so that $\sigma_{\bar{X}} < 0.01\text{V}$?
- (c) With $n = 100$, what is $P(|\bar{X} - 3.3| > 0.02)$?

Solution

- (a) $E[\bar{X}] = 3.3$, $\text{Var}(\bar{X}) = 0.01/1 = 0.01$, $\sigma = 0.1\text{V}$
- (b) $\sigma_{\bar{X}} = \sigma/\sqrt{n} < 0.01 \rightarrow \sqrt{n} > 10 \rightarrow n > 100 \rightarrow \text{need } n \geq 101$
- (c) $\sigma_{\bar{X}} = 0.1/\sqrt{100} = 0.01\text{V}$
 $P(|\bar{X} - 3.3| > 0.02) = P(|Z| > 2) = 2(1 - \Phi(2)) = 0.0455$

Problem 4: Component Lifetime

Components have lifetime $T \sim \text{Exp}(\lambda = 0.002 \text{ per hour})$.

- (a) Find MTTF, σ_T , and coefficient of variation.
- (b) Find $P(T > 1000 \text{ hours})$.
- (c) Find $E[T^2]$ and interpret.

Solution

- (a) $\text{MTTF} = 1/\lambda = 500 \text{ hours}$, $\sigma_T = 500 \text{ hours}$, $\text{CV} = \sigma/\mu = 1$
- (b) $P(T > 1000) = e^{-0.002 \times 1000} = e^{-2} = 0.1353$
- (c) $E[T^2] = \text{Var}(T) + (E[T])^2 = 500^2 + 500^2 = 500000 \text{ hours}^2$
 $\sqrt{E[T^2]} = 707 \text{ hours (RMS lifetime)}$

Problem 5: MGF Application

X has MGF $M_X(t) = (1/3)e^t + (2/3)e^{2t}$.

- (a) What type of RV is X ? Find its PMF.
- (b) Find $E[X]$ using the MGF.
- (c) Find $\text{Var}(X)$ using the MGF.

Solution

- (a) X is discrete. Comparing with $\sum p_X(x)e^{tx}$:
 $P(X=1) = 1/3$, $P(X=2) = 2/3$
- (b) $M'(t) = (1/3)e^t + (4/3)e^{2t}$
 $E[X] = M'(0) = 1/3 + 4/3 = 5/3$
- (c) $M''(t) = (1/3)e^t + (8/3)e^{2t}$
 $E[X^2] = M''(0) = 1/3 + 8/3 = 3$
 $\text{Var}(X) = 3 - (5/3)^2 = 3 - 25/9 = 2/9$

4.13 Key Takeaways

1. **$E[X]$ = long-run average** — the center of the distribution, DC component
2. **$\text{Var}(X)$ = spread/power** — for zero-mean signals, variance IS power
3. **σ has same units as X** — directly interpretable as RMS value
4. **Linearity of expectation** always holds; variance addition requires independence
5. **MGF uniquely identifies** a distribution and generates all moments
6. **In engineering:** mean $\rightarrow$ signal level, variance $\rightarrow$ noise power, std dev $\rightarrow$ RMS noise

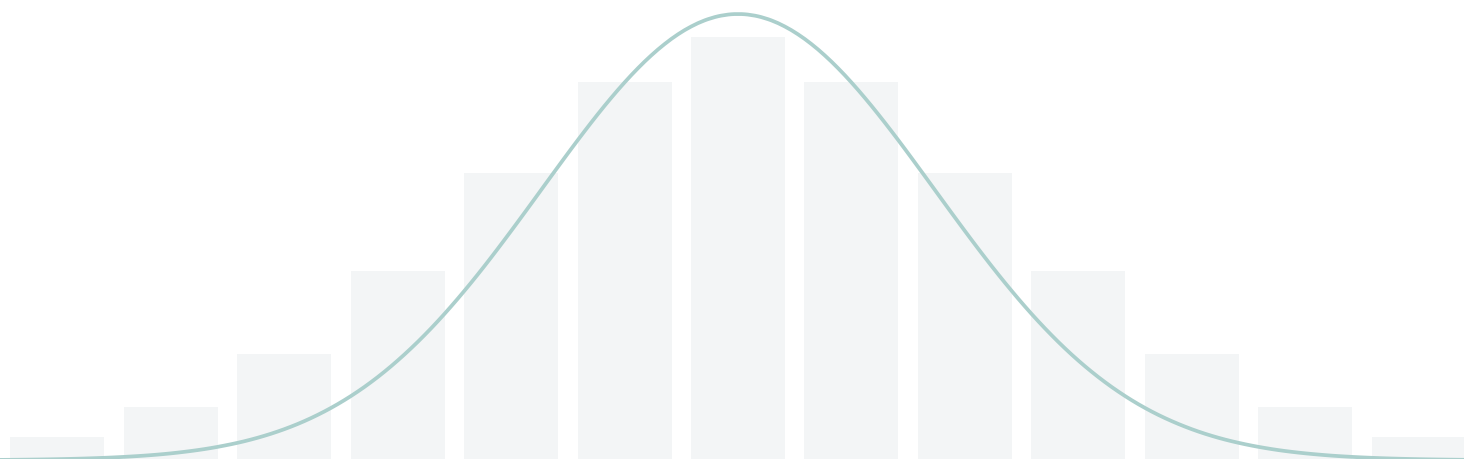
Next Module: Module 5 — Important Random-Variable Functions

Module 5

Important Random-Variable Functions

توابع مهم متغیر تصادفی

5



In This Module

5.1	Introduction: Why Transform Random Variables?	116
5.2	Linear Transformations: $Y = aX + b$	117
5.3	General Monotonic Transformations	118
5.4	Nonlinear Transformations: Non-Monotonic Case	119
5.5	The Jacobian Method	120
5.6	Engineering Application: Rayleigh from Gaussian	121
5.7	MATLAB Examples	122
5.8	Practice Problems	125

Learning Objectives

After completing this module, students will be able to:

1. Apply linear transformations to random variables
2. Derive PDFs of nonlinear transformations using the CDF and Jacobian methods
3. Apply transformations to engineering problems (power, SNR, envelope)
4. Verify analytical results with MATLAB simulation

5.1 Introduction: Why Transform Random Variables?

In engineering, we often know the statistics of one quantity but need the statistics of a **derived** quantity:

Known	Needed	Transformation
Voltage V	Power P	$P = V^2/R$
Linear SNR	SNR in dB	$Y = 10 \cdot \log_{10}(X)$
Gaussian I, Q	Envelope	$R = \sqrt{I^2 + Q^2}$
Current I	Power	$P = I^2 R$
Distance d	Path loss	$L = d^\alpha$

5.2 Linear Transformations: $Y = aX + b$

■ Result

If X has PDF $f_X(x)$, then $Y = aX + b$ has:

$$f_Y(y) = \frac{1}{|a|} f_X\left(\frac{y-b}{a}\right)$$

■ Mean and Variance

- ▶ $E[Y] = aE[X] + b$
- ▶ $\text{Var}(Y) = a^2 \text{Var}(X)$

Engineering Example: Amplifier

Input signal $X \sim N(0, \sigma^2)$ passes through amplifier with gain $G = 10$ and DC offset $V_0 = 1.5\text{V}$.

Output: $Y = 10X + 1.5$

- ▶ $Y \sim N(10 \cdot 0 + 1.5, 10^2 \cdot \sigma^2) = N(1.5, 100\sigma^2)$
- ▶ Output noise power increased by $G^2 = 100$

Engineering Example: Temperature Sensor

Sensor output $V = 0.01 \cdot T + 0.5$ (V in volts, T in $^\circ\text{C}$)

If $T \sim N(25, 4)$: $V \sim N(0.01 \times 25 + 0.5, 0.01^2 \times 4) = N(0.75, 0.0004)$

5.3 General Monotonic Transformations

■ CDF Method (Most General)

For $Y = g(X)$, find $f_Y(y)$ by:

1. Write $F_Y(y) = P(Y \leq y) = P(g(X) \leq y)$
2. Solve the inequality for X
3. Express in terms of F_X
4. Differentiate to get $f_Y(y)$

■ Formula for Monotonic g

If g is strictly increasing with inverse g^{-1} :

$$f_Y(y) = f_X(g^{-1}(y)) \cdot \frac{d}{dy}[g^{-1}(y)]$$

If g is strictly decreasing:

$$f_Y(y) = f_X(g^{-1}(y)) \cdot \left| \frac{d}{dy}[g^{-1}(y)] \right|$$

Combined (works for both):

$$f_Y(y) = \frac{f_X(x)}{|g'(x)|} \Big|_{x=g^{-1}(y)}$$

Engineering Example: SNR in dB

Linear SNR: $X \sim \text{Exponential}(1)$ (normalized Rayleigh fading power).

dB SNR: $Y = 10 \cdot \log_{10}(X)$

- $g(x) = 10 \cdot \log_{10}(x)$, $g'(x) = 10/(x \cdot \ln 10)$
- $g^{-1}(y) = 10^{y/10}$

$$f_Y(y) = \frac{f_X(10^{y/10})}{10/(10^{y/10} \cdot \ln 10)} = \frac{\ln 10}{10} \cdot 10^{y/10} \cdot e^{-10^{y/10}}$$

This is NOT Gaussian — it has a longer left tail (deep fades).

5.4 Nonlinear Transformations: Non-Monotonic Case

■ When $Y = g(X)$ is Not One-to-One

If multiple x -values map to the same y , sum over all solutions:

$$f_Y(y) = \sum_i \frac{f_X(x_i)}{|g'(x_i)|}$$

where $x_1, x_2, \dots$ are all roots of $g(x) = y$.

Engineering Example: Power from Voltage ($Y = X^2$)

Voltage $X \sim N(0, \sigma^2)$. Power: $Y = X^2$.

For $y > 0$, solutions are $x = +\sqrt{y}$ and $x = -\sqrt{y}$. $g'(x) = 2x$.

$$f_Y(y) = \frac{f_X(\sqrt{y})}{2\sqrt{y}} + \frac{f_X(-\sqrt{y})}{2\sqrt{y}} = \frac{1}{\sqrt{2\pi}\sigma} \cdot \frac{e^{-y/(2\sigma^2)}}{\sqrt{y}}$$

This is a **Chi-squared distribution with 1 degree of freedom** (scaled by σ^2).

Engineering Example: Full-Wave Rectifier ($Y = |X|$)

Input $X \sim N(0, \sigma^2)$. Output $Y = |X|$.

For $y > 0$: $x = +y$ and $x = -y$ are solutions. $|g'(x)| = 1$.

$$f_Y(y) = f_X(y) + f_X(-y) = \frac{2}{\sigma\sqrt{2\pi}} e^{-y^2/(2\sigma^2)}, \quad y \geq 0$$

This is the **folded normal** (half-normal) distribution.

5.5 The Jacobian Method

■ Procedure for $Y = g(X)$

1. Identify the transformation $y = g(x)$
2. Find the inverse: $x = g^{-1}(y)$
3. Compute the Jacobian: $J = |dx/dy| = |d[g^{-1}(y)]/dy|$
4. Apply: $f_Y(y) = f_X(g^{-1}(y)) \cdot |J|$

■ Step-by-Step Example: Exponential of Gaussian

$X \sim N(\mu, \sigma^2)$. $Y = e^X$ (log-normal transformation).

1. $g(x) = e^x$ (monotonically increasing)
2. $x = \ln(y)$, valid for $y > 0$
3. $J = |dx/dy| = 1/y$
4. $f_Y(y) = f_X(\ln y) \cdot (1/y) = (1/(y\sigma\sqrt{2\pi})) \cdot \exp(-(\ln y - \mu)^2/(2\sigma^2))$

This is the **log-normal distribution** — models many engineering quantities (shadowing in wireless, stock prices, component lifetimes with wear).

5.6 Engineering Application: Rayleigh from Gaussian

■ Derivation of Rayleigh Distribution

In communications, the received signal has in-phase (I) and quadrature (Q) components:

- $I \sim N(0, \sigma^2)$, $Q \sim N(0, \sigma^2)$, independent

Envelope: $R = \sqrt{I^2 + Q^2}$

Using the transformation from $(I, Q) \rightarrow (R, \theta)$ with polar coordinates:

- $I = R \cdot \cos(\theta)$, $Q = R \cdot \sin(\theta)$
- Jacobian: $|\partial(I, Q) / \partial(R, \theta)| = R$

Joint PDF of (R, θ) :

$$f_{R, \Theta}(r, \theta) = f_{I, Q}(r \cdot \cos \theta, r \cdot \sin \theta) \cdot r = (1/(2\pi\sigma^2)) \cdot e^{-r^2/(2\sigma^2)}$$

Marginalizing over $\theta \in [0, 2\pi)$:

$$f_R(r) = \frac{r}{\sigma^2} e^{-r^2/(2\sigma^2)}, \quad r \geq 0$$

This is the **Rayleigh distribution** — fundamental to wireless fading channel modeling.

5.7 MATLAB Examples

Example 1: Linear Transformation Verification

MATLAB

```

%% Linear Transformation: Amplifier Output
sigma_in = 0.1; G = 5; offset = 2;
N = 100000;

X = sigma_in * randn(1, N); % Input noise
Y = G*X + offset;           % Output

figure;
subplot(1,2,1);
histogram(X, 80, 'Normalization', 'pdf'); hold on;
x = linspace(-0.5, 0.5, 200);
plot(x, normpdf(x, 0, sigma_in), 'r', 'LineWidth', 2);
title('Input X ~ N(0, 0.01)'); xlabel('Voltage (V)');

subplot(1,2,2);
histogram(Y, 80, 'Normalization', 'pdf'); hold on;
y = linspace(0, 4, 200);
plot(y, normpdf(y, offset, G*sigma_in), 'r', 'LineWidth', 2);
title('Output Y = 5X + 2'); xlabel('Voltage (V)');

```

Example 2: Power from Gaussian Voltage

MATLAB

```

%% Nonlinear: Y = X^2 (instantaneous power)
sigma = 1; N = 200000;
X = sigma * randn(1, N);
Y = X.^2;

figure;
histogram(Y, 150, 'Normalization', 'pdf', 'BinLimits', [0, 8]);
hold on;
y = linspace(0.01, 8, 300);
f_theory = chi2pdf(y/sigma^2, 1) / sigma^2;
plot(y, f_theory, 'r', 'LineWidth', 2);
xlabel('Power (V^2)'); ylabel('PDF');
title('Distribution of Power Y = X^2');
legend('Simulation', 'Theory (scaled  $\chi^2_1$ )');

```

Example 3: SNR in dB from Linear SNR

```

%% SNR transformation:  $Y = 10 \log_{10}(X)$ 
%  $X \sim \text{Exponential}(1)$  (Rayleigh fading power, normalized)
N = 200000;
X = exprnd(1, 1, N);
Y = 10*log10(X); % SNR in dB

figure;
histogram(Y, 150, 'Normalization', 'pdf');
hold on;
y = linspace(-30, 15, 300);
% Theoretical PDF of  $Y = 10 \log_{10}(X)$  where  $X \sim \text{Exp}(1)$ 
f_Y = (log(10)/10) .* 10.^(y/10) .* exp(-10.^(y/10));
plot(y, f_Y, 'r', 'LineWidth', 2);
xlabel('SNR (dB)'); ylabel('PDF');
title('Distribution of SNR in dB (Rayleigh Fading)');
legend('Simulation', 'Theory');

```

MATLAB

Example 4: Rayleigh Envelope from Gaussian Components

```

%% Envelope of Complex Gaussian
sigma = 1; N = 100000;
I = sigma * randn(1, N);
Q = sigma * randn(1, N);
R = sqrt(I.^2 + Q.^2);

figure;
histogram(R, 100, 'Normalization', 'pdf');
hold on;
r = linspace(0, 5, 200);
plot(r, raylpdf(r, sigma), 'r', 'LineWidth', 2);
xlabel('Envelope |h|'); ylabel('PDF');
title('Rayleigh Distribution from I/Q Components');
legend('Simulation', 'Theory');
fprintf('Mean: Sim=%.3f, Theory=%.3f\n', mean(R), sigma*sqrt(pi/2));

```

MATLAB

Example 5: Log-Normal from Gaussian

```
%% Log-Normal:  $Y = \exp(X)$  where  $X \sim N(\mu, \sigma^2)$ 
mu = 0; sigma_x = 0.5; N = 100000;
X = mu + sigma_x*randn(1, N);
Y = exp(X);

figure;
histogram(Y, 150, 'Normalization', 'pdf', 'BinLimits', [0, 5]);
hold on;
y = linspace(0.01, 5, 300);
f_lognorm = lognpdf(y, mu, sigma_x);
plot(y, f_lognorm, 'r', 'LineWidth', 2);
xlabel('Y = e^X'); ylabel('PDF');
title('Log-Normal Distribution');
legend('Simulation', 'Theory');
```

MATLAB

5.8 Practice Problems

Problem 1

Signal $X \sim \text{Uniform}[0, 1]$. Output $Y = -2 \cdot \ln(X)$. Find the PDF of Y and identify the distribution.

Solution CDF method: $F_Y(y) = P(-2 \ln(X) \leq y) = P(X \geq e^{-y/2}) = 1 - e^{-y/2}$ for $y \geq 0$.

$f_Y(y) = (1/2)e^{-y/2} \rightarrow Y \sim \text{Exponential}(\beta = 2)$. (This is the inverse CDF method for generating exponentials!)

Problem 2

Noise voltage $X \sim N(0, \sigma^2)$ with $\sigma = 2V$. Power dissipated in $R = 50\Omega$: $P = X^2/R$.

(a) Find $E[P]$. (b) Find the PDF of P . (c) Find $P(P > 0.2W)$.

Solution

(a) $E[P] = E[X^2]/R = \sigma^2/R = 4/50 = 0.08W$

(b) $P = X^2/50$. Let $W = X^2 \sim \sigma^2 \cdot \chi^2(1)$. Then $P = W/50$. $f_P(p) = 50 \cdot f_W(50p) = (50/(2\sigma^2)) \cdot (50p/\sigma^2)^{(-1/2)} \cdot e^{-(50p/(2\sigma^2))}$ for $p > 0$.

(c) Use MATLAB: `1 - chi2cdf(0.2*50/4, 1) = 1 - chi2cdf(2.5, 1)  $\approx$  0.114`

Problem 3

$X \sim \text{Exponential}(\lambda = 1)$. $Y = \sqrt{X}$. Find $f_Y(y)$.

Solution $g(x) = \sqrt{x} \rightarrow x = y^2, dx/dy = 2y$.

$f_Y(y) = f_X(y^2) \cdot |2y| = e^{-y^2} \cdot 2y$ for $y \geq 0$. This is a Rayleigh distribution with $\sigma^2 = 1/2$.

Problem 4

Distance $D \sim \text{Uniform}[1, 10]$ km. Path loss: $L = 20 \cdot \log_{10}(D)$ dB. Find $E[L]$ and the PDF of L .

Solution $E[L] = E[20 \cdot \log_{10}(D)] = \int_1^{10} 20 \cdot \log_{10}(d) \cdot (1/9) dd = (20/9) \cdot \int_1^{10} \log_{10}(d) dd$
 $= (20/9) \cdot [d \cdot \log_{10}(d) - d/\ln(10)]_1^{10} = (20/9) \cdot [10 - 10/\ln(10) + 1/\ln(10)] = (20/9) \cdot (10 - 9/\ln(10)) \approx 13.55$ dB

PDF: L ranges from 0 to 20 dB. $g^{-1}(l) = 10^{l/20}$, $|dg^{-1}/dl| = (\ln(10)/20) \cdot 10^{l/20}$.

$f_L(l) = (1/9) \cdot (\ln(10)/20) \cdot 10^{l/20}$ for $0 \leq l \leq 20$.

Problem 5

Write MATLAB code to verify Problem 2 by simulation.

Solution

```
sigma = 2; R = 50; N = 500000;
X = sigma*randn(1,N);
P = X.^2 / R;
fprintf('E[P]: Theory=%.4f W, Sim=%.4f W\n', sigma^2/R, mean(P));
fprintf('P(P>0.2): Theory=%.4f, Sim=%.4f\n', 1-chi2cdf(0.2*R/sigma^2,1),
↳ mean(P>0.2));
```

MATLAB

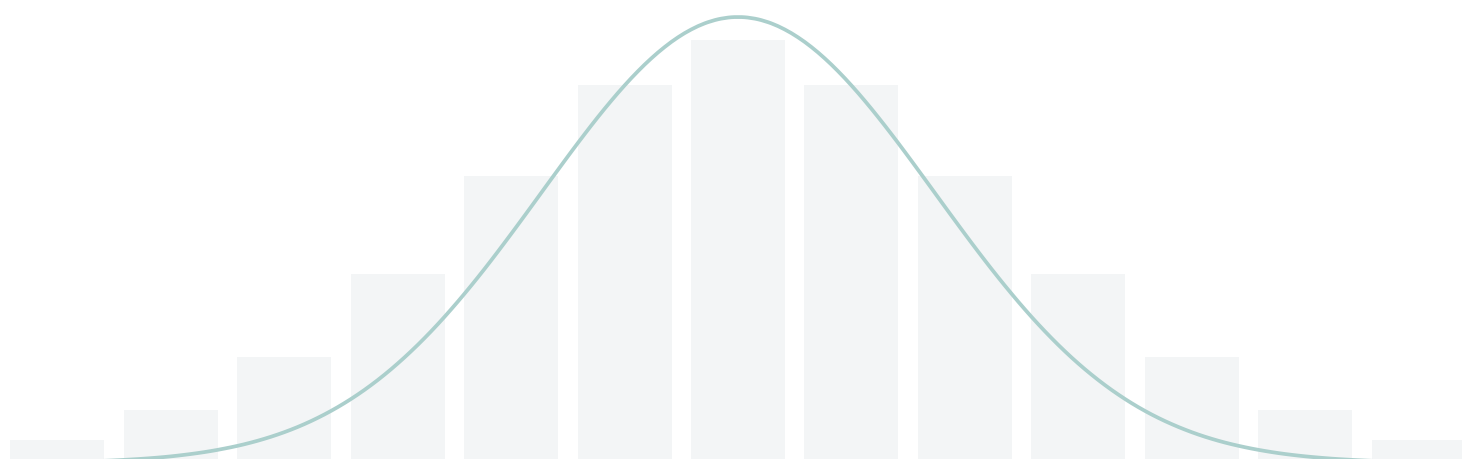
Next Module: Module 6 — Two Random Variables

Module 6

Two Random Variables

دو متغیر تصادفی

6



In This Module

6.1	Introduction: Why Study Joint Distributions?	130
6.2	Joint PMF (Discrete Case)	131
6.3	Joint PDF (Continuous Case)	132
6.4	Joint CDF	133
6.5	Marginal Distributions	134
6.6	Conditional Distributions	135
6.7	Independence of Two Random Variables	136
6.8	Conditional Expectation	137
6.9	MATLAB Examples	138
6.10	Practice Problems	140

Learning Objectives

After completing this module, students will be able to:

1. Define and compute joint PMF, PDF, and CDF
2. Derive marginal distributions from joint distributions
3. Compute conditional distributions
4. Test independence of two random variables
5. Compute conditional expectations
6. Visualize joint distributions in MATLAB

6.1 Introduction: Why Study Joint Distributions?

Real engineering systems involve **multiple interacting uncertain quantities**:

- ▶ Voltage AND current in a noisy circuit
- ▶ Signal AND noise at a receiver
- ▶ Temperature AND resistance of a sensor
- ▶ I-channel AND Q-channel of a communication signal

Understanding the **joint behavior** tells us things marginals cannot — like correlation, dependence, and conditional behavior.

6.2 Joint PMF (Discrete Case)

Definition

For discrete RVs X, Y :

$$p_{X,Y}(x, y) = P(X = x, Y = y)$$

■ Properties

1. $p_{X,Y}(x, y) \geq 0$
2. $\sum_x \sum_y p_{X,Y}(x, y) = 1$

■ Visualization: Probability Table

Example: X = number of retransmissions (0,1,2), Y = acknowledgment delay category (1=fast, 2=slow)

	Y=1	Y=2	$p_X(x)$
X=0	0.30	0.10	0.40
X=1	0.15	0.20	0.35
X=2	0.05	0.20	0.25
$p_Y(y)$	0.50	0.50	1.00

6.3 Joint PDF (Continuous Case)

Definition

For continuous RVs X, Y , the joint PDF $f_{X,Y}(x,y)$ satisfies:

$$P((X, Y) \in A) = \iint_A f_{X,Y}(x, y) dx dy$$

■ Properties

1. $f_{X,Y}(x,y) \geq 0$
2. $\iint f_{X,Y}(x,y) dx dy = 1$

■ Visualization

The joint PDF is a **surface** over the (x,y) plane. Probability = volume under surface over a region.

Engineering Example: Bivariate Gaussian

Two correlated noise voltages (X, Y) with correlation ρ :

$$f_{X,Y}(x, y) = \frac{1}{2\pi\sigma_X\sigma_Y\sqrt{1-\rho^2}} \exp\left(-\frac{1}{2(1-\rho^2)} \left[\frac{x^2}{\sigma_X^2} - \frac{2\rho xy}{\sigma_X\sigma_Y} + \frac{y^2}{\sigma_Y^2}\right]\right)$$

6.4 Joint CDF

Definition

$$F_{X,Y}(x, y) = P(X \leq x, Y \leq y)$$

■ Properties

1. $F_{X,Y}(-\infty, y) = 0, F_{X,Y}(x, -\infty) = 0$
2. $F_{X,Y}(\infty, \infty) = 1$
3. Non-decreasing in both arguments
4. $f_{X,Y}(x,y) = \partial^2 F_{X,Y} / (\partial x \partial y)$

6.5 Marginal Distributions

■ Obtaining Marginals from Joint

Discrete:

$$p_X(x) = \sum_y p_{X,Y}(x, y), \quad p_Y(y) = \sum_x p_{X,Y}(x, y)$$

Continuous:

$$f_X(x) = \int_{-\infty}^{\infty} f_{X,Y}(x, y) dy, \quad f_Y(y) = \int_{-\infty}^{\infty} f_{X,Y}(x, y) dx$$

■ Visual Interpretation

The marginal PDF $f_X(x)$ is the **projection** (integral) of the joint surface onto the x-axis. Similarly $f_Y(y)$ projects onto the y-axis.

Engineering Example

Joint PDF: $f_{X,Y}(x,y) = 2e^{(-x)e}(-2y)$, $x \geq 0, y \geq 0$

Marginals:

- ▶ $f_X(x) = \int_0^{\infty} 2e^{(-x)e}(-2y) dy = 2e^{-x} \cdot [1/2] = e^{-x} \rightarrow X \sim \text{Exp}(1)$
- ▶ $f_Y(y) = \int_0^{\infty} 2e^{(-x)e}(-2y) dx = 2e^{-2y} \cdot [1] = 2e^{-2y} \rightarrow Y \sim \text{Exp}(2)$

6.6 Conditional Distributions

■ Discrete

$$p_{X|Y}(x|y) = \frac{p_{X,Y}(x,y)}{p_Y(y)}$$

■ Continuous

$$f_{X|Y}(x|y) = \frac{f_{X,Y}(x,y)}{f_Y(y)}$$

■ Interpretation

The conditional distribution describes X when we **know** Y = y. It's a “slice” of the joint distribution at a fixed y value, renormalized to integrate to 1.

Engineering Example: Signal Given Channel State

Received power Y given channel state X:

- ▶ If channel is good (X=1): $Y \sim N(10, 1)$
- ▶ If channel is poor (X=0): $Y \sim N(2, 4)$
- ▶ $P(X=1) = 0.7$, $P(X=0) = 0.3$

Conditional PDFs describe receiver behavior in each state.

6.7 Independence of Two Random Variables

Definition

X and Y are independent if and only if:

$$f_{X,Y}(x,y) = f_X(x) \cdot f_Y(y) \quad \text{for all } x, y$$

(Or equivalently for PMFs: $p_{X,Y}(x,y) = p_X(x) \cdot p_Y(y)$)

■ Consequences of Independence

- ▶ $f_{X|Y}(x|y) = f_X(x)$ (knowing Y doesn't change X)
- ▶ $E[XY] = E[X]E[Y]$
- ▶ $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$

■ Testing Independence

Check: Does the joint factor into a product of a function of x alone and y alone?

- ▶ $f_{X,Y}(x,y) = 2e^{(-x)} \cdot e^{(-2y)} = [e^{-x}] \cdot [2e^{-2y}]$ ✓ Independent!
- ▶ $f_{X,Y}(x,y) = x + y$ for $0 \leq x, y \leq 1 \rightarrow$ cannot factor $\rightarrow$ NOT independent

Engineering Example: Signal and Independent Noise

Transmitted signal S and channel noise N are independent by physical assumption:

$$f_{S,N}(s,n) = f_S(s) \cdot f_N(n)$$

This independence assumption is fundamental to communication system design.

6.8 Conditional Expectation

Definition

$$E[X|Y = y] = \begin{cases} \sum_x x \cdot p_{X|Y}(x|y) & \text{discrete} \\ \int_{-\infty}^{\infty} x \cdot f_{X|Y}(x|y) dx & \text{continuous} \end{cases}$$

■ Interpretation

$E[X|Y=y]$ is the **best prediction** of X given that we observed $Y = y$ (in the mean-square sense).

Engineering Example: Temperature-Dependent Resistance

A resistor's value R depends on temperature T . If $R|T \sim N(1000 + 0.5T, 4)$:

$$E[R|T=50^\circ\text{C}] = 1000 + 0.5(50) = 1025\Omega$$

Given temperature, we can predict resistance.

6.9 MATLAB Examples

Example 1: Joint PMF Visualization

```
%% Joint PMF: Retransmissions vs Delay Category
pXY = [0.30 0.10; 0.15 0.20; 0.05 0.20];
x = 0:2; y = 1:2;

figure;
bar3(pXY);
xlabel('Delay Category Y'); ylabel('Retransmissions X');
zlabel('P(X=x, Y=y)');
title('Joint PMF');

% Marginals
pX = sum(pXY, 2)'; % Sum over Y
pY = sum(pXY, 1); % Sum over X
fprintf('Marginal of X: [%.2f %.2f %.2f]\n', pX);
fprintf('Marginal of Y: [%.2f %.2f]\n', pY);
```

MATLAB

Example 2: Bivariate Gaussian Visualization

```
%% Joint Gaussian PDF with Correlation
mu = [0 0]; sigma_x = 1; sigma_y = 1.5; rho = 0.7;
Sigma = [sigma_x^2, rho*sigma_x*sigma_y; rho*sigma_x*sigma_y, sigma_y^2];

[X, Y] = meshgrid(linspace(-4, 4, 100), linspace(-5, 5, 100));
XY = [X(:) Y(:)];
Z = mvnpdf(XY, mu, Sigma);
Z = reshape(Z, size(X));

figure;
subplot(1,2,1);
surf(X, Y, Z, 'EdgeColor', 'none'); view(30, 40);
xlabel('X'); ylabel('Y'); zlabel('f_{X,Y}(x,y)');
title(sprintf('Joint Gaussian (ρ = %.1f)', rho));

subplot(1,2,2);
contour(X, Y, Z, 20);
xlabel('X'); ylabel('Y');
title('Contour Plot');
axis equal;
```

MATLAB

Example 3: Testing Independence

```

%% Independence Check via Simulation
N = 100000;

% Case 1: Independent (signal and noise)
S = randn(1, N);           % Signal
Noise = 0.5*randn(1, N);    % Independent noise
R = S + Noise;              % Received

% Case 2: Dependent (voltage and current through same resistor)
V = randn(1, N);           % Voltage
I = V / 50 + 0.01*randn(1,N); % Current (dependent on V!)

fprintf('Independent (S, Noise): corr = %.4f\n', corr(S', Noise'));
fprintf('Dependent (V, I): corr = %.4f\n', corr(V', I'));

```

MATLAB

Example 4: Conditional Distribution

```

%% Conditional PDF: Slice of Bivariate Gaussian
rho = 0.8; N = 100000;
X = randn(1, N);
Y = rho*X + sqrt(1-rho^2)*randn(1, N); % Y|X ~ N(rho*x, 1-rho^2)

% Conditional: Y given X ≈ 1 (take samples where 0.9 < X < 1.1)
mask = (X > 0.9) & (X < 1.1);
Y_cond = Y(mask);

figure;
histogram(Y_cond, 50, 'Normalization', 'pdf');
hold on;
y = linspace(-3, 4, 200);
plot(y, normpdf(y, rho*1, sqrt(1-rho^2)), 'r', 'LineWidth', 2);
xlabel('Y'); ylabel('PDF');
title('f_{Y|X}(y|X≈1) for Bivariate Gaussian');
legend('Empirical', sprintf('N(%.1f, %.2f)', rho, 1-rho^2));

```

MATLAB

6.10 Practice Problems

Problem 1

Joint PDF: $f_{X,Y}(x,y) = 6(1-y)$ for $0 \leq x \leq y \leq 1$, zero elsewhere.

(a) Verify normalization. (b) Find marginals. (c) Are X,Y independent? (d) Find $P(X < 0.5, Y < 0.5)$.

Solution

$$(a) \int_0^1 \int_0^y 6(1-y) dx dy = \int_0^1 6y(1-y) dy = 6[y^2/2 - y^3/3]_0^1 = 6(1/2 - 1/3) = 1 \quad \checkmark$$

$$(b) f_X(x) = \int_x^1 6(1-y) dy = 6[(1-y^2/2) - (y)]_x^1 = 3(1-x)^2; f_Y(y) = 6y(1-y)$$

(c) $f_{X,Y} \neq f_X \cdot f_Y \rightarrow$ NOT independent (also, the support is triangular, not rectangular)

$$(d) P(X < 0.5, Y < 0.5) = \int_0^{0.5} \int_0^y 6(1-y) dx dy = \int_0^{0.5} 6y(1-y) dy = 6[y^2/2 - y^3/3]_0^{0.5} = 0.500$$

Problem 2

Two sensors measure the same signal with independent noise. $X = S + N_1$, $Y = S + N_2$ where $S=5V$, $N_1 \sim N(0,1)$, $N_2 \sim N(0,4)$. Are X and Y independent? Find $E[X]$, $E[Y]$, and explain why $\text{Cov}(X,Y) \neq 0$.

Solution X and Y are NOT independent because they share S (though noise is independent). $E[X] = E[Y] = 5$. $\text{Cov}(X,Y) = \text{Cov}(S+N_1, S+N_2) = \text{Var}(S) + \text{Cov}(S,N_1) + \text{Cov}(N_2,S) + \text{Cov}(N_1,N_2)$. Since S is constant, $\text{Var}(S)=0$, so if S were random they'd be correlated. With S fixed, X and Y are actually independent in this case. If S is random with $\text{Var}(S) = \sigma_S^2$, then $\text{Cov}(X,Y) = \sigma_S^2$.

Problem 3

Given joint PMF in the table of Section 6.2, find (a) $P(X \leq 1, Y=2)$, (b) $f_{X|Y}(x|Y=1)$, (c) $E[X|Y=1]$.

Solution

$$(a) P(X \leq 1, Y=2) = p(0,2) + p(1,2) = 0.10 + 0.20 = 0.30$$

$$(b) p_{X|Y}(x|1) = p(x,1)/p_Y(1): p(0|1)=0.30/0.50=0.60, p(1|1)=0.15/0.50=0.30, p(2|1)=0.05/0.50=0.10$$

$$(c) E[X|Y=1] = 0(0.60) + 1(0.30) + 2(0.10) = 0.50$$

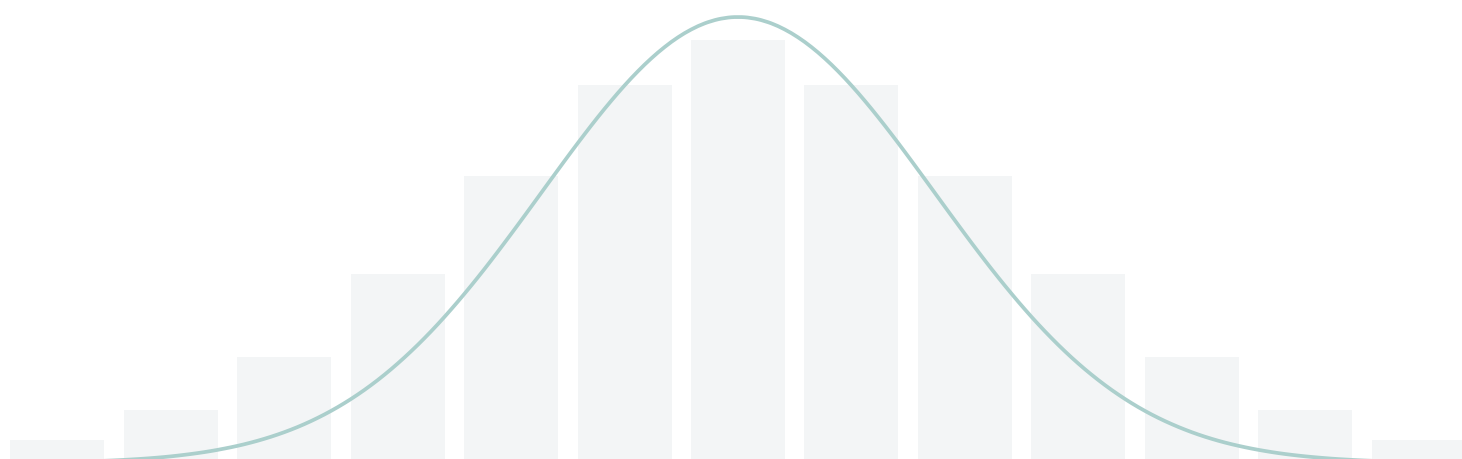
Next Module: Module 7 — Functions of Two Random Variables

Module 7

Functions of Two Random Variables

توابع دو متغیر تصادفی

7



In This Module

7.1	Introduction	144
7.2	Sum of Two Random Variables: $Z = X + Y$	145
7.3	Difference: $Z = X - Y$	146
7.4	Product: $Z = XY$	147
7.5	Ratio: $Z = X/Y$	148
7.6	Maximum: $Z = \max(X, Y)$	149
7.7	Minimum: $Z = \min(X, Y)$	150
7.8	General Bivariate Transformation (Jacobian Method)	151
7.9	MATLAB Examples	152
7.10	Practice Problems	154

Learning Objectives

After completing this module, students will be able to:

1. Derive the distribution of $Z = X + Y$ using convolution
2. Find distributions of differences, products, ratios, max, and min
3. Apply the bivariate Jacobian transformation method
4. Connect these results to engineering applications (signal+noise, reliability)

7.1 Introduction

Engineering systems combine random variables:

- ▶ **Received signal** = transmitted + noise: $R = S + N$
- ▶ **Total interference** = sum of multiple sources: $I = I_1 + I_2 + \dots + I_n$
- ▶ **System lifetime** = minimum of component lifetimes: $T = \min(T_1, T_2)$
- ▶ **SNR** = signal power / noise power: ratio of RVs

We need methods to find distributions of **functions of two (or more) RVs**.

7.2 Sum of Two Random Variables: $Z = X + Y$

■ CDF Method

$$F_Z(z) = P(X + Y \leq z) = \int \int \{x+y \leq z\} f\{X,Y\}(x,y) dx dy$$

■ Convolution Formula (Independent Case)

If X and Y are **independent**:

$$f_Z(z) = \int_{-\infty}^{\infty} f_X(x) \cdot f_Y(z-x) dx = (f_X * f_Y)(z)$$

This is the **convolution** of f_X and f_Y .

■ Derivation

$$\begin{aligned} F_Z(z) &= P(X + Y \leq z) = \int \int_{-\infty}^{\infty} \int_{-\infty}^{z-x} f_X(x)f_Y(y) dy dx \\ &= \int_{-\infty}^{\infty} f_X(x) F_Y(z-x) dx \end{aligned}$$

$$\text{Differentiating: } f_Z(z) = \int_{-\infty}^{\infty} f_X(x) f_Y(z-x) dx \quad \checkmark$$

■ Important Special Cases

Sum of two Gaussians (independent):

$$X \sim N(\mu_1, \sigma_1^2), Y \sim N(\mu_2, \sigma_2^2) \rightarrow Z = X+Y \sim N(\mu_1+\mu_2, \sigma_1^2+\sigma_2^2)$$

Sum of two exponentials (same rate):

$$X, Y \sim \text{Exp}(\lambda) \text{ independent} \rightarrow Z \sim \text{Gamma}(2, 1/\lambda) = \text{Erlang}(2, \lambda)$$

Sum of two Poissons:

$$X \sim \text{Poisson}(\lambda_1), Y \sim \text{Poisson}(\lambda_2) \text{ independent} \rightarrow Z \sim \text{Poisson}(\lambda_1+\lambda_2)$$

Sum of two uniforms:

$$X, Y \sim \text{Uniform}(0,1) \text{ independent} \rightarrow Z \text{ has triangular distribution on } [0,2]$$

Engineering Example: Signal + Noise

Transmitted signal $S \sim N(A, \sigma_s^2)$, noise $N \sim N(0, \sigma_n^2)$, independent.

Received: $R = S + N \sim N(A, \sigma_s^2 + \sigma_n^2)$

The received signal is still Gaussian — with increased variance (reduced SNR).

7.3 Difference: $Z = X - Y$

For independent X, Y :

$$f_Z(z) = \int_{-\infty}^{\infty} f_X(x) \cdot f_Y(x - z) dx$$

Gaussian case: $X \sim N(\mu_1, \sigma_1^2), Y \sim N(\mu_2, \sigma_2^2) \rightarrow Z = X - Y \sim N(\mu_1 - \mu_2, \sigma_1^2 + \sigma_2^2)$

Note: variances still ADD (not subtract) because $\text{Var}(-Y) = \text{Var}(Y)$.

Engineering Example: Differential Measurement

Two sensors measure same quantity: $X_1 = \theta + N_1, X_2 = \theta + N_2$.

Difference: $X_1 - X_2 = N_1 - N_2 \sim N(0, \sigma_1^2 + \sigma_2^2)$

The difference eliminates the common signal but doubles the noise variance.

7.4 Product: $Z = XY$

For independent X, Y :

$$f_Z(z) = \int_{-\infty}^{\infty} \frac{1}{|x|} f_X(x) \cdot f_Y(z/x) dx$$

Engineering Example: Power

Voltage V and current I in a linear circuit: $P = V \cdot I$.

If V and I have known joint distribution, we can find the power distribution.

7.5 Ratio: $Z = X/Y$

For independent X, Y :

$$f_Z(z) = \int_{-\infty}^{\infty} |y| \cdot f_X(zy) \cdot f_Y(y) dy$$

Engineering Example: SNR

SNR = Signal Power / Noise Power = P_s / P_n

If both are chi-squared distributed (sum of squared Gaussians), the ratio follows an **F-distribution**.

7.6 Maximum: $Z = \max(X, Y)$

■ CDF Approach

$$F_Z(z) = P(\max(X, Y) \leq z) = P(X \leq z \text{ AND } Y \leq z)$$

$$\text{If independent: } F_Z(z) = F_X(z) \cdot F_Y(z)$$

$$\text{PDF: } f_Z(z) = f_X(z)F_Y(z) + F_X(z)f_Y(z)$$

Engineering Example: Parallel Redundancy

System works if AT LEAST ONE component works. System fails only when ALL fail.

System lifetime = $\max(T_1, T_2)$ for parallel redundant components.

For $T_1, T_2 \sim \text{Exp}(\lambda)$ independent:

$$F_Z(z) = (1 - e^{-\lambda z})^2 \text{ for } z \geq 0$$

$$f_Z(z) = 2\lambda e^{-\lambda z}(1 - e^{-\lambda z})$$

Mean lifetime: $E[\max] = 3/(2\lambda) > 1/\lambda = E[\text{single component}]$ (reliability improved!)

7.7 Minimum: $Z = \min(X, Y)$

■ Survival Function Approach

$$P(Z > z) = P(\min(X, Y) > z) = P(X > z \text{ AND } Y > z)$$

$$\text{If independent: } P(Z > z) = P(X > z) \cdot P(Y > z) = [1 - F_X(z)][1 - F_Y(z)]$$

$$\text{CDF: } F_Z(z) = 1 - [1 - F_X(z)][1 - F_Y(z)]$$

Engineering Example: Series System

System fails when FIRST component fails. System lifetime = $\min(T_1, T_2)$.

For $T_1 \sim \text{Exp}(\lambda_1)$, $T_2 \sim \text{Exp}(\lambda_2)$ independent:

$$P(Z > z) = e^{-\lambda_1 z} \cdot e^{-\lambda_2 z} = e^{-(\lambda_1 + \lambda_2)z}$$

Therefore: $\min(T_1, T_2) \sim \text{Exp}(\lambda_1 + \lambda_2)$

Key result: Failure rates ADD in series systems. Mean lifetime = $1/(\lambda_1 + \lambda_2) < \min(1/\lambda_1, 1/\lambda_2)$.

7.8 General Bivariate Transformation (Jacobian Method)

■ Setup

Given (X, Y) with known joint PDF, find the joint PDF of (U, V) where:

- ▶ $U = g_1(X, Y)$
- ▶ $V = g_2(X, Y)$

■ Procedure

1. Solve for the inverse: $X = h_1(U, V)$, $Y = h_2(U, V)$
2. Compute the Jacobian determinant:

$$J = \begin{vmatrix} \frac{\partial x}{\partial u} & \frac{\partial x}{\partial v} \\ \frac{\partial y}{\partial u} & \frac{\partial y}{\partial v} \end{vmatrix}$$

3. Apply:

$$f_{U,V}(u, v) = f_{X,Y}(h_1(u, v), h_2(u, v)) \cdot |J|$$

4. If only U is needed, marginalize out V .

Example: Sum and Difference

$$U = X + Y, V = X - Y \rightarrow X = (U+V)/2, Y = (U-V)/2$$

$$J = |\partial x / \partial u \cdot \partial y / \partial v - \partial x / \partial v \cdot \partial y / \partial u| = |(1/2)(-1/2) - (1/2)(1/2)| = 1/2$$

$$f_{U,V}(u,v) = (1/2) \cdot f_{X,Y}((u+v)/2, (u-v)/2)$$

7.9 MATLAB Examples

Example 1: Convolution — Sum of Two Uniforms

```

%% Z = X + Y, both Uniform(0,1)
N = 200000;
X = rand(1, N); Y = rand(1, N);
Z = X + Y;

figure;
histogram(Z, 100, 'Normalization', 'pdf');
hold on;
z = linspace(0, 2, 200);
f_theory = max(0, 1 - abs(z - 1)); % Triangular PDF
plot(z, f_theory, 'r', 'LineWidth', 2);
xlabel('Z = X + Y'); ylabel('PDF');
title('Sum of Two Uniforms → Triangular');
legend('Simulation', 'Theory');

```

MATLAB

Example 2: Sum of Gaussians (Signal + Noise)

```

%% Received = Signal + Noise
mu_s = 3; sigma_s = 0.5; % Signal
sigma_n = 1; % Noise (zero-mean)
N = 100000;

S = mu_s + sigma_s*randn(1,N);
Noise = sigma_n*randn(1,N);
R = S + Noise;

fprintf('E[R]: Theory=%.2f, Sim=%.2f\n', mu_s, mean(R));
fprintf('Var(R): Theory=%.2f, Sim=%.2f\n', sigma_s^2+sigma_n^2, var(R));
fprintf('SNR = %.2f dB\n', 10*log10(mu_s^2/(sigma_s^2+sigma_n^2)));

```

MATLAB

Example 3: Series System Reliability (Minimum)

```

%% Series System: T_sys = min(T1, T2)
lambda1 = 0.001; lambda2 = 0.002; % Failure rates per hour
N = 100000;

T1 = exprnd(1/lambda1, 1, N);
T2 = exprnd(1/lambda2, 1, N);
T_sys = min(T1, T2);

fprintf('MTTF component 1: %.0f hours\n', 1/lambda1);
fprintf('MTTF component 2: %.0f hours\n', 1/lambda2);
fprintf('MTTF system (theory): %.0f hours\n', 1/(lambda1+lambda2));
fprintf('MTTF system (sim): %.0f hours\n', mean(T_sys));

```

MATLAB

Example 4: Parallel System (Maximum)

```
%% Parallel System: T_sys = max(T1, T2)  
lambda = 0.01; N = 100000;  
T1 = exprnd(1/lambda, 1, N);  
T2 = exprnd(1/lambda, 1, N);  
T_parallel = max(T1, T2);  
  
fprintf('MTTF single: %.0f hours\n', 1/lambda);  
fprintf('MTTF parallel (theory): %.0f hours\n', 3/(2*lambda));  
fprintf('MTTF parallel (sim): %.0f hours\n', mean(T_parallel));
```

MATLAB

7.10 Practice Problems**Problem 1**

$X \sim \text{Exp}(1)$, $Y \sim \text{Exp}(1)$, independent. Find the PDF of $Z = X + Y$.

Solution Convolution: $f_Z(z) = \int_0^z e^{(-x)} \cdot e^{-(z-x)} dx = \int_0^z e^{-z} dx = ze^{-z}$, $z \geq 0$.

This is $\text{Gamma}(2,1) = \text{Erlang}(2,1)$. $E[Z] = 2$, $\text{Var}(Z) = 2$.

Problem 2

Three components in series with failure rates $\lambda_1=0.001$, $\lambda_2=0.002$, $\lambda_3=0.003$ per hour.

(a) Find system MTTF. (b) Find $P(\text{system survives 100 hours})$.

Solution

(a) $\text{MTTF} = 1/(0.001+0.002+0.003) = 1/0.006 = 166.7$ hours

(b) $P(T_{\text{sys}} > 100) = e^{-0.006 \times 100} = e^{-0.6} = 0.549$

Problem 3

$X \sim N(5, 4)$, $Y \sim N(3, 9)$, independent. Find distribution and $P(X + Y > 12)$.

Solution $Z = X+Y \sim N(8, 13)$. $P(Z>12) = P(Z' > (12-8)/\sqrt{13}) = Q(1.109) = 0.1337$.

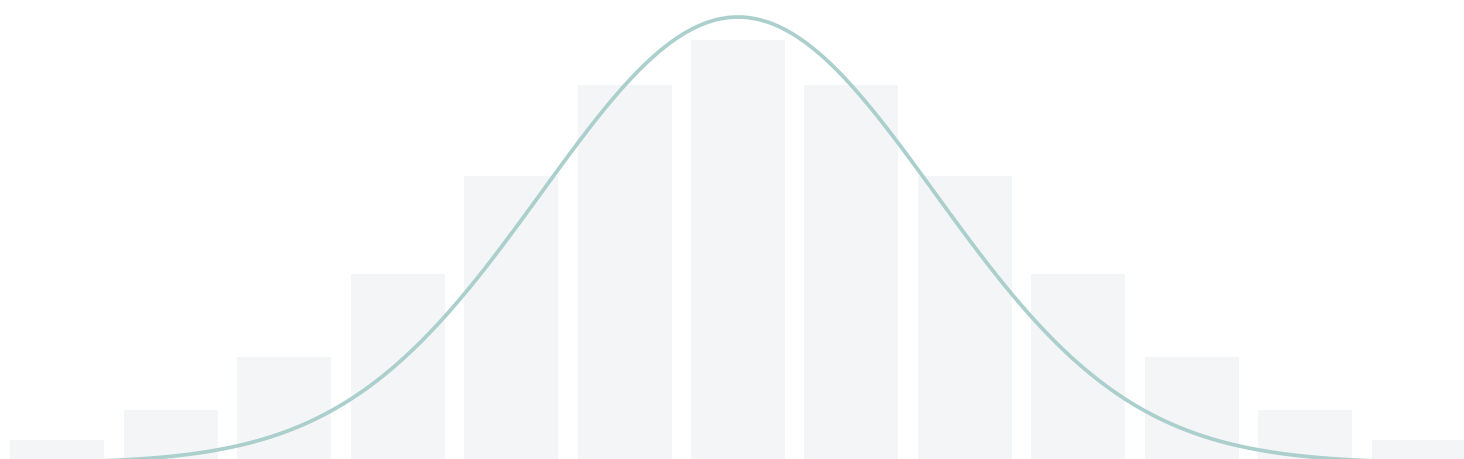
Next Module: Module 8 — Conditional Expectation and Variance

Module 8

Conditional Expectation and Variance

امید ریاضی و واریانس شرطی

8



In This Module

8.1	Introduction	158
8.2	Conditional Expectation $E[X Y=y]$	159
8.3	Conditional Variance $\text{Var}(X Y=y)$	160
8.4	Law of Total Expectation (Tower Property)	161
8.5	Law of Total Variance	162
8.6	MATLAB Examples	163
8.7	Practice Problems	165

Learning Objectives

After completing this module, students will be able to:

1. Compute conditional expectations and variances
2. Apply the Law of Total Expectation
3. Apply the Law of Total Variance
4. Use these tools for engineering analysis across multiple operating conditions

8.1 Introduction

Many engineering systems operate under **different conditions** (channel states, modes, environments). We often need the **overall average performance** computed from conditional performance in each state.

- ▶ What is the average BER when the channel randomly switches between good and bad?
- ▶ What is the overall measurement variance when the sensor operates in different environments?

8.2 Conditional Expectation $E[X|Y=y]$

Definition

Continuous:

$$E[X|Y = y] = \int_{-\infty}^{\infty} x \cdot f_{X|Y}(x|y) dx$$

Discrete:

$$E[X|Y = y] = \sum_x x \cdot p_{X|Y}(x|y)$$

■ Interpretation

$E[X|Y=y]$ is the **average value of X** when we know $Y = y$. It is a function of y .

■ $E[X|Y]$ as a Random Variable

When Y is itself random, $E[X|Y]$ is a **random variable** — it takes the value $E[X|Y=y]$ when $Y = y$.

Engineering Example

Average received power given channel state:

- ▶ Good channel ($Y=1$): $E[P|Y=1] = 10$ mW
- ▶ Poor channel ($Y=0$): $E[P|Y=0] = 1$ mW
- ▶ $P(Y=1) = 0.7$, $P(Y=0) = 0.3$

8.3 Conditional Variance $\text{Var}(X|Y=y)$

Definition

$$\text{Var}(X|Y = y) = E[(X - E[X|Y = y])^2|Y = y] = E[X^2|Y = y] - (E[X|Y = y])^2$$

■ Interpretation

The conditional variance measures the **remaining uncertainty in X** after Y is known.

Engineering Example

Measurement precision depends on temperature:

- ▶ At $T=20^\circ\text{C}$: $\text{Var}(X|T=20) = 0.01$ (high precision)
- ▶ At $T=80^\circ\text{C}$: $\text{Var}(X|T=80) = 0.09$ (low precision due to thermal effects)

8.4 Law of Total Expectation (Tower Property)

■ Statement

$$E[X] = E[E[X|Y]]$$

Discrete version:

$$E[X] = \sum_y E[X|Y=y] \cdot P(Y=y)$$

Continuous version:

$$E[X] = \int_{-\infty}^{\infty} E[X|Y=y] \cdot f_Y(y) dy$$

■ Proof (Discrete)

$$\begin{aligned} E[E[X|Y]] &= \sum_y E[X|Y=y] \cdot P(Y=y) \\ &= \sum_y \left[\sum_x x \cdot P(X=x|Y=y) \right] \cdot P(Y=y) \\ &= \sum_x \sum_y x \cdot P(X=x, Y=y) \\ &= \sum_x x \cdot P(X=x) = E[X] \quad \checkmark \end{aligned}$$

■ Interpretation

The **overall average** equals the **weighted average of conditional averages**, weighted by the probability of each condition.

Engineering Example: Average BER Across Channel States

A wireless system switches between three modes:

- ▶ Mode 1 (60% of time): $\text{BER} = 10^{-6}$
- ▶ Mode 2 (30% of time): $\text{BER} = 10^{-4}$
- ▶ Mode 3 (10% of time): $\text{BER} = 10^{-2}$

$$\begin{aligned} \text{Overall BER} &= E[\text{BER}] = 0.6(10^{-6}) + 0.3(10^{-4}) + 0.1(10^{-2}) \\ &= 0.6 \times 10^{-6} + 30 \times 10^{-6} + 10000 \times 10^{-6} \approx 1.003 \times 10^{-3} \end{aligned}$$

Engineering insight: The overall BER is dominated by the worst mode, even though it occurs only 10% of the time.

Engineering Example: Average Lifetime with Defect Types

Components are defect-free (90%) or have a hidden defect (10%):

- ▶ Defect-free: $E[T|\text{good}] = 10000$ hours
- ▶ With defect: $E[T|\text{defect}] = 500$ hours

$$E[T] = 0.9(10000) + 0.1(500) = 9050 \text{ hours}$$

8.5 Law of Total Variance

■ Statement

$$\text{Var}(X) = E[\text{Var}(X|Y)] + \text{Var}(E[X|Y])$$

Term	Name	Interpretation
$\text{Var}(X)$	Total variance	Overall uncertainty in X
$E[\text{Var}(X Y)]$	Expected conditional variance	Average “within-group” variability
$\text{Var}(E[X Y])$	Variance of conditional means	“Between-group” variability

■ Proof

$$\text{Var}(X) = E[X^2] - (E[X])^2$$

$$E[X^2] = E[E[X^2|Y]] \text{ (tower property)}$$

$$= E[\text{Var}(X|Y) + (E[X|Y])^2]$$

$$= E[\text{Var}(X|Y)] + E[(E[X|Y])^2]$$

$$(E[X])^2 = (E[E[X|Y]])^2$$

$$\text{Var}(X) = E[\text{Var}(X|Y)] + E[(E[X|Y])^2] - (E[E[X|Y]])^2$$

$$= E[\text{Var}(X|Y)] + \text{Var}(E[X|Y]) \quad \checkmark$$

■ Interpretation: Decomposition of Uncertainty

Total uncertainty = Average uncertainty within each condition + Variability of the mean across conditions

Engineering Example: Sensor in Two Environments

A sensor operates indoors ($Y=1$, prob 0.6) and outdoors ($Y=2$, prob 0.4):

- Indoor: $E[X|Y=1] = 25^\circ\text{C}$, $\text{Var}(X|Y=1) = 1$
- Outdoor: $E[X|Y=2] = 15^\circ\text{C}$, $\text{Var}(X|Y=2) = 16$

$$E[\text{Var}(X|Y)] = 0.6(1) + 0.4(16) = 7.0 \text{ (average within-condition variance)}$$

$$\text{Var}(E[X|Y]) = E[(E[X|Y])^2] - (E[E[X|Y]])^2$$

$$= 0.6(25^2) + 0.4(15^2) - [0.6(25) + 0.4(15)]^2 = 375 + 90 - (21)^2 = 465 - 441 = 24$$

$$\text{Var}(X) = 7.0 + 24.0 = 31.0$$

Most variance comes from the **between-condition** difference (24 out of 31).

8.6 MATLAB Examples

Example 1: Law of Total Expectation

```

%% Average BER across channel states
% State probabilities and conditional BERs
p_state = [0.6, 0.3, 0.1];
BER_cond = [1e-6, 1e-4, 1e-2];

% Total expectation
BER_overall = sum(p_state .* BER_cond);
fprintf('Overall BER = %.4e\n', BER_overall);

% Simulation verification
N = 1000000;
states = randsample(1:3, N, true, p_state);
errors = rand(1,N) < BER_cond(states);
BER_sim = mean(errors);
fprintf('Simulated BER = %.4e\n', BER_sim);

```

MATLAB

Example 2: Law of Total Variance

```

%% Total Variance Decomposition: Sensor Example
N = 100000;
p_indoor = 0.6;

% Generate environment (1=indoor, 2=outdoor)
env = (rand(1,N) > p_indoor) + 1;

% Generate measurements conditional on environment
X = zeros(1, N);
X(env==1) = 25 + 1*randn(1, sum(env==1)); % Indoor: N(25, 1)
X(env==2) = 15 + 4*randn(1, sum(env==2)); % Outdoor: N(15, 16)

% Total variance
total_var = var(X);

% E[Var(X|Y)]
within_var = p_indoor*var(X(env==1)) + (1-p_indoor)*var(X(env==2));

% Var(E[X|Y])
cond_means = [mean(X(env==1)), mean(X(env==2))];
between_var = p_indoor*cond_means(1)^2 + (1-p_indoor)*cond_means(2)^2 ...
    - (p_indoor*cond_means(1) + (1-p_indoor)*cond_means(2))^2;

fprintf('Total Var(X) = %.2f (theory: 31.0)\n', total_var);
fprintf('E[Var(X|Y)] = %.2f (theory: 7.0)\n', within_var);
fprintf('Var(E[X|Y]) = %.2f (theory: 24.0)\n', between_var);
fprintf('Sum = %.2f\n', within_var + between_var);

```

MATLAB

Example 3: Conditional Expectation as Best Predictor

MATLAB

```

%% E[X|Y] as a function of Y
% Joint:  $X = Y + N$ ,  $Y \sim \text{Uniform}(0,10)$ ,  $N \sim N(0, 1)$ 
N_samp = 50000;
Y = 10*rand(1, N_samp);
X = Y + randn(1, N_samp);

% Estimate  $E[X|Y=y]$  for various y
y_bins = 0:0.5:10;
E_X_given_Y = zeros(size(y_bins)-[0 1]);
for i = 1:length(y_bins)-1
    mask = Y >= y_bins(i) & Y < y_bins(i+1);
    E_X_given_Y(i) = mean(X(mask));
end
y_centers = (y_bins(1:end-1) + y_bins(2:end))/2;

figure;
scatter(Y(1:5000), X(1:5000), 1, 'b', '.');
hold on;
plot(y_centers, E_X_given_Y, 'r-', 'LineWidth', 3);
plot(y_centers, y_centers, 'g--', 'LineWidth', 2);
xlabel('Y'); ylabel('X');
legend('Data (X,Y)', 'E[X|Y] (estimated)', 'E[X|Y] = Y (theory)');
title('Conditional Expectation');

```

8.7 Practice Problems

Problem 1

A digital link operates over two channel types:

- Type A (prob 0.7): $E[\text{delay}|A] = 5\text{ms}$, $\text{Var}(\text{delay}|A) = 2$
- Type B (prob 0.3): $E[\text{delay}|B] = 20\text{ms}$, $\text{Var}(\text{delay}|B) = 25$

Find (a) $E[\text{delay}]$, (b) $\text{Var}(\text{delay})$.

Solution

- (a) $E[D] = 0.7(5) + 0.3(20) = 3.5 + 6 = 9.5 \text{ ms}$
 (b) $E[\text{Var}(D|\text{type})] = 0.7(2) + 0.3(25) = 1.4 + 7.5 = 8.9$
 $\text{Var}(E[D|\text{type}]) = 0.7(5^2) + 0.3(20^2) - 9.5^2 = 17.5 + 120 - 90.25 = 47.25$
 $\text{Var}(D) = 8.9 + 47.25 = 56.15$

Problem 2

A random number N of components are tested. $N \sim \text{Poisson}(10)$. Each has independent failure probability $p=0.05$. Let X = number of failures. Find $E[X]$ and $\text{Var}(X)$.

Solution $X|N \sim \text{Binomial}(N, 0.05)$. $E[X|N] = 0.05N$. $\text{Var}(X|N) = N(0.05)(0.95) = 0.0475N$.

$E[X] = E[E[X|N]] = E[0.05N] = 0.05(10) = 0.5$

$\text{Var}(X) = E[\text{Var}(X|N)] + \text{Var}(E[X|N]) = E[0.0475N] + \text{Var}(0.05N) = 0.0475(10) + 0.05^2(10) = 0.475 + 0.025 = 0.5$

Problem 3

Write MATLAB code to verify Problem 2 by simulation.

Solution

```
N_trials = 100000; p = 0.05; lambda = 10;
N = poissrnd(lambda, 1, N_trials);
X = binornd(N, p);
fprintf('E[X]: theory=0.5, sim=%.3f\n', mean(X));
fprintf('Var(X): theory=0.5, sim=%.3f\n', var(X));
```

MATLAB

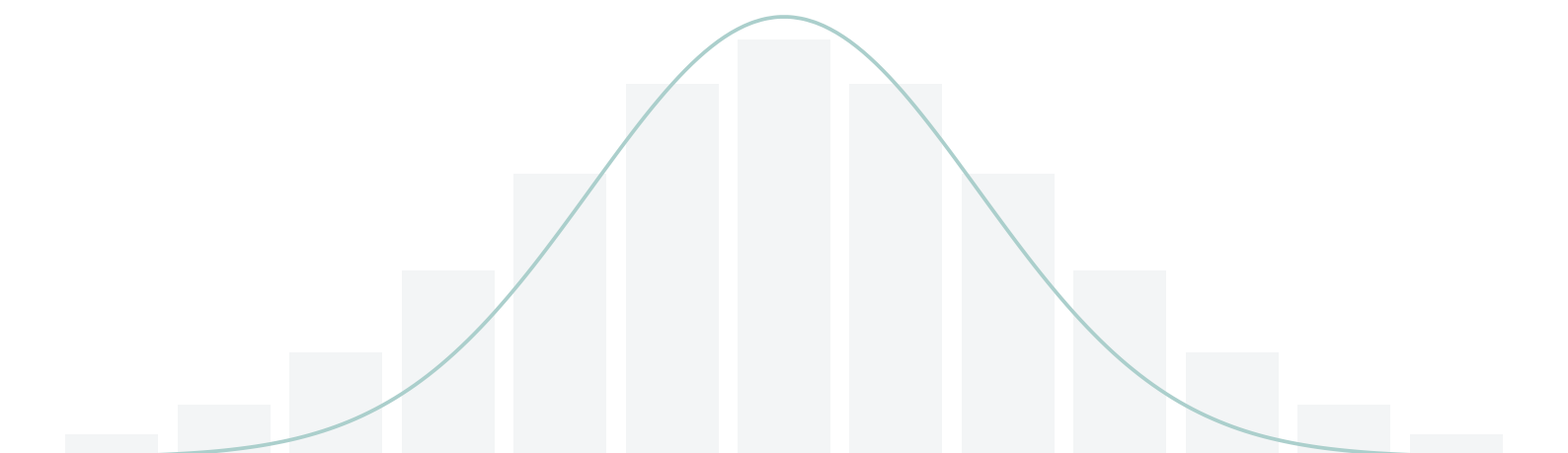
Next Module: Module 9 — Covariance and Correlation

Module 9

Covariance and Correlation

کوواریانس و همبستگی

9



In This Module

9.1	Introduction: Measuring Linear Relationships	169
9.2	Covariance	170
9.3	Correlation Coefficient	171
9.4	Cross-Correlation	172
9.5	Independence vs. Uncorrelatedness	173
9.6	Correlation Matrix	174
9.7	Variance of Linear Combinations	175
9.8	MATLAB Examples	176
9.9	Practice Problems	178

Learning Objectives

After completing this module, students will be able to:

1. Compute and interpret covariance
2. Compute and interpret the correlation coefficient
3. Distinguish between independence and uncorrelatedness
4. Construct and interpret correlation matrices
5. Apply correlation analysis to engineering problems

9.1 Introduction: Measuring Linear Relationships

When two random variables are not independent, we need to quantify **how they relate**. Covariance and correlation measure the **linear relationship** between two RVs.

Engineering examples:

- ▶ Correlated noise in adjacent sensor channels
- ▶ Signal measurements at two antennas (MIMO correlation)
- ▶ Voltage and current in a circuit (related through impedance)
- ▶ Temperature and device performance (thermal dependence)

9.2 Covariance

Definition

$$\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)] = E[XY] - E[X]E[Y]$$

■ Properties

1. $\text{Cov}(X, X) = \text{Var}(X)$
2. $\text{Cov}(X, Y) = \text{Cov}(Y, X)$ (symmetric)
3. $\text{Cov}(aX + b, cY + d) = ac \cdot \text{Cov}(X, Y)$
4. If X, Y independent $\rightarrow \text{Cov}(X, Y) = 0$
5. $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2\text{Cov}(X, Y)$
6. $\text{Var}(X - Y) = \text{Var}(X) + \text{Var}(Y) - 2\text{Cov}(X, Y)$

■ Sign Interpretation

Cov(X,Y)	Meaning
> 0	X and Y tend to increase together
< 0	When X increases, Y tends to decrease
= 0	No linear relationship (uncorrelated)

Engineering Example: Amplifier Gain and Output

If gain G varies randomly and input is fixed at V_{in} :

- Output: $V_{out} = G \cdot V_{in}$
- $\text{Cov}(G, V_{out}) = \text{Cov}(G, G \cdot V_{in}) = V_{in} \cdot \text{Var}(G) > 0$

Higher gain $\rightarrow$ higher output (positive covariance).

9.3 Correlation Coefficient

Definition

$$\rho_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

■ Properties

1. **Bounded:** $-1 \leq \rho \leq 1$
2. $\rho = 1$: Perfect positive linear relationship ($Y = aX + b$, $a > 0$)
3. $\rho = -1$: Perfect negative linear relationship ($Y = aX + b$, $a < 0$)
4. $\rho = 0$: Uncorrelated (no linear relationship)
5. **Dimensionless:** Unlike covariance, ρ doesn't depend on units

■ Interpretation Scale

ρ	
0.0 – 0.2	Very weak/negligible
0.2 – 0.4	Weak
0.4 – 0.6	Moderate
0.6 – 0.8	Strong
0.8 – 1.0	Very strong

Engineering Example: Correlated Fading

Two antenna paths with correlation $\rho = 0.3$:

- Low correlation $\rightarrow$ diversity works well (independent fading)
- High correlation $\rightarrow$ diversity gain is reduced

MIMO system design requires: $\rho < 0.5$ for effective spatial multiplexing.

9.4 Cross-Correlation

Definition

$$R_{XY} = E[XY]$$

■ Relationship to Covariance

$$\text{Cov}(X, Y) = R_{XY} - \mu_X \mu_Y$$

If either X or Y has zero mean: $\text{Cov}(X, Y) = E[XY] = R_{XY}$

■ Engineering Significance

Cross-correlation is fundamental in:

- ▶ Matched filters (correlation receivers in communications)
- ▶ Radar signal processing (detecting time delay)
- ▶ Signal detection (correlating received with known template)

9.5 Independence vs. Uncorrelatedness

■ The Critical Distinction

Statement	Always True?
Independent $\rightarrow$ Uncorrelated	✓ YES
Uncorrelated $\rightarrow$ Independent	✗ NO (in general)
Uncorrelated $\rightarrow$ Independent (jointly Gaussian)	✓ YES

■ Why Independent $\rightarrow$ Uncorrelated

If X, Y independent: $E[XY] = E[X]E[Y]$, so $\text{Cov}(X, Y) = E[XY] - E[X]E[Y] = 0$.

■ Counterexample: Uncorrelated but Dependent

Let $X \sim N(0, 1)$ and $Y = X^2$. Then:

- $\text{Cov}(X, Y) = E[XY] - E[X]E[Y] = E[X^3] - 0 = 0$ (since X is symmetric)
- But Y is completely determined by $X \rightarrow$ maximally dependent!

$\rho = 0$ does NOT mean independent (except for jointly Gaussian RVs).

■ The Gaussian Exception

For **jointly Gaussian** random variables:

$$\text{Uncorrelated} \Leftrightarrow \text{Independent}$$

This is why the Gaussian distribution is so convenient in engineering — checking $\rho = 0$ suffices to establish independence.

■ Engineering Implication

In communications with Gaussian noise:

- If noise samples are uncorrelated $\rightarrow$ they are independent
- This simplifies analysis enormously
- For non-Gaussian interference: uncorrelated does NOT guarantee independence

9.6 Correlation Matrix

Definition for Random Vector $\mathbf{X} = [X_1, X_2, \dots, X_n]^T$

The **covariance matrix** Σ :

$$\Sigma_{ij} = \text{Cov}(X_i, X_j)$$

$$\Sigma = \begin{bmatrix} \text{Var}(X_1) & \text{Cov}(X_1, X_2) & \cdots \\ \text{Cov}(X_2, X_1) & \text{Var}(X_2) & \cdots \\ \vdots & & \ddots \end{bmatrix}$$

The **correlation matrix** $\mathbf{R}$ (normalized):

$$R_{ij} = \rho_{X_i X_j} = \frac{\Sigma_{ij}}{\sqrt{\Sigma_{ii} \Sigma_{jj}}}$$

■ Properties of Covariance Matrix

1. Symmetric: $\Sigma = \Sigma^T$
2. Positive semi-definite: $\mathbf{x}^T \Sigma \mathbf{x} \geq 0$ for all $\mathbf{x}$
3. Diagonal entries = variances
4. Off-diagonal entries = covariances

Engineering Example: Three Sensors

Three temperature sensors with:

- ▶ $\sigma_1 = \sigma_2 = \sigma_3 = 1^\circ\text{C}$
- ▶ $\rho_{12} = 0.8$ (sensors 1,2 are nearby)
- ▶ $\rho_{13} = 0.3$ (sensor 3 is farther)
- ▶ $\rho_{23} = 0.4$

Correlation matrix:

$$\mathbf{R} = \begin{bmatrix} 1.0 & 0.8 & 0.3 \\ 0.8 & 1.0 & 0.4 \\ 0.3 & 0.4 & 1.0 \end{bmatrix}$$

9.7 Variance of Linear Combinations

■ General Formula

$$\text{Var}\left(\sum_{i=1}^n a_i X_i\right) = \sum_{i=1}^n a_i^2 \text{Var}(X_i) + 2 \sum_{i < j} a_i a_j \text{Cov}(X_i, X_j)$$

In matrix form: $\text{Var}(\mathbf{a}^T \mathbf{X}) = \mathbf{a}^T \Sigma \mathbf{a}$

■ Special Case: Sum of Two RVs

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2\text{Cov}(X, Y) = \sigma_X^2 + \sigma_Y^2 + 2\rho\sigma_X\sigma_Y$$

Correlation	Var(X+Y)	Effect
$\rho = 1$	$(\sigma_X + \sigma_Y)^2$	Maximum — variances add constructively
$\rho = 0$	$\sigma_X^2 + \sigma_Y^2$	Independent addition
$\rho = -1$	$(\sigma_X - \sigma_Y)^2$	Minimum — cancellation

■ Engineering Application: Averaging Correlated Sensors

If n sensors have equal variance σ^2 and pairwise correlation ρ :

$$\text{Var}(\bar{X}) = \frac{\sigma^2}{n} [1 + (n-1)\rho]$$

- If $\rho = 0$ (independent): $\text{Var}(\bar{X}) = \sigma^2/n$ (full averaging benefit)
- If $\rho = 1$ (identical): $\text{Var}(\bar{X}) = \sigma^2$ (no benefit from averaging!)
- If $\rho = 0.5$, $n = 4$: $\text{Var}(\bar{X}) = \sigma^2(1 + 3 \times 0.5)/4 = 0.625\sigma^2$ (reduced benefit)

9.8 MATLAB Examples

Example 1: Computing Correlation

MATLAB

```

%% Sample Correlation: Correlated Sensor Data
N = 10000;
rho = 0.7; sigma1 = 2; sigma2 = 3;

% Generate correlated Gaussians
Z1 = randn(1, N);
Z2 = randn(1, N);
X = sigma1 * Z1;
Y = sigma2 * (rho * Z1 + sqrt(1-rho^2) * Z2);

% Compute sample statistics
cov_xy = mean(X.*Y) - mean(X)*mean(Y);
rho_hat = cov_xy / (std(X)*std(Y));

fprintf('True  $\rho$  = %.2f, Estimated  $\rho$  = %.4f\n', rho, rho_hat);
fprintf('Cov(X,Y): Theory=%.2f, Estimated=%.4f\n', rho*sigma1*sigma2, cov_xy);

% Scatter plot
figure;
scatter(X, Y, 1, 'b', '.');
xlabel('X (Sensor 1)'); ylabel('Y (Sensor 2)');
title(sprintf('Correlated Sensors ( $\rho$  = %.2f)', rho));
grid on;

```

Example 2: Uncorrelated but Dependent

MATLAB

```

%% Demonstration:  $\rho = 0$  does NOT mean NOT independent
N = 100000;
X = randn(1, N);
Y = X.^2; % Y completely depends on X

rho_hat = corr(X', Y');
fprintf('Correlation  $\rho(X, X^2)$  = %.4f  $\approx 0$  (UNCORRELATED)\n', rho_hat);
fprintf('But  $Y = X^2$  is perfectly determined by X (DEPENDENT!)\n');

figure;
scatter(X, Y, 1, '.'); xlabel('X'); ylabel('Y = X^2');
title('Uncorrelated but Dependent!');

```

Example 3: Correlation Matrix Visualization

MATLAB

```
%% Correlation Matrix for Multiple Sensors
n_sensors = 5;
% Create correlation matrix with decaying correlation
R = zeros(n_sensors);
for i = 1:n_sensors
    for j = 1:n_sensors
        R(i,j) = 0.9^abs(i-j); % Correlation decays with distance
    end
end

% Generate correlated samples
L = chol(R, 'lower'); % Cholesky decomposition
N = 10000;
Z = randn(n_sensors, N);
X = L * Z; % Correlated samples

% Estimated correlation matrix
R_hat = corrcoef(X');

figure;
subplot(1,2,1); imagesc(R); colorbar;
title('Theoretical R'); xlabel('Sensor'); ylabel('Sensor');
subplot(1,2,2); imagesc(R_hat); colorbar;
title('Estimated R'); xlabel('Sensor'); ylabel('Sensor');
```

9.9 Practice Problems

Problem 1

X and Y have: $E[X]=2$, $E[Y]=3$, $\text{Var}(X)=4$, $\text{Var}(Y)=9$, $E[XY]=8$.

(a) Find $\text{Cov}(X,Y)$. (b) Find ρ . (c) Find $\text{Var}(X+Y)$. (d) Find $\text{Var}(2X-3Y)$.

Solution

(a) $\text{Cov}(X,Y) = E[XY] - E[X]E[Y] = 8 - 2(3) = 2$

(b) $\rho = 2/(\sqrt{4} \cdot \sqrt{9}) = 2/6 = 1/3$

(c) $\text{Var}(X+Y) = 4 + 9 + 2(2) = 17$

(d) $\text{Var}(2X-3Y) = 4(4) + 9(9) + 2(2)(-3)(2) = 16 + 81 - 24 = 73$

Problem 2

Four identical sensors ($\sigma^2=1$) have pairwise correlation $\rho=0.5$. What is $\text{Var}(\bar{X})$?

Solution $\text{Var}(\bar{X}) = \sigma^2/n \cdot [1 + (n-1)\rho] = 1/4 \cdot [1 + 3(0.5)] = 0.25 \times 2.5 = 0.625$

Compare: if independent ($\rho=0$): $\text{Var}(\bar{X}) = 0.25$. Correlation reduces the effectiveness of averaging.

Problem 3

Given $X \sim N(0,1)$, define $Y = X$ when $|X| < 1$ and $Y = -X$ when $|X| \geq 1$. Show that $\text{Cov}(X,Y) \neq 0$ in this case but that $E[X \cdot Y]$ can be computed.

Solution $E[XY] = E[X^2 \cdot 1(|X| < 1)] + E[-X^2 \cdot 1(|X| \geq 1)] = E[X^2 \cdot 1(|X| < 1)] - E[X^2 \cdot 1(|X| \geq 1)]$
 $= P(|X| < 1)E[X^2 | |X| < 1] - P(|X| \geq 1)E[X^2 | |X| \geq 1]$

This is nonzero since $E[X^2 | |X| < 1] \neq E[X^2 | |X| \geq 1]$, so $\text{Cov}(X,Y) = E[XY] - 0 \neq 0$.

X and Y are clearly dependent (Y is defined from X), and in this case also correlated.

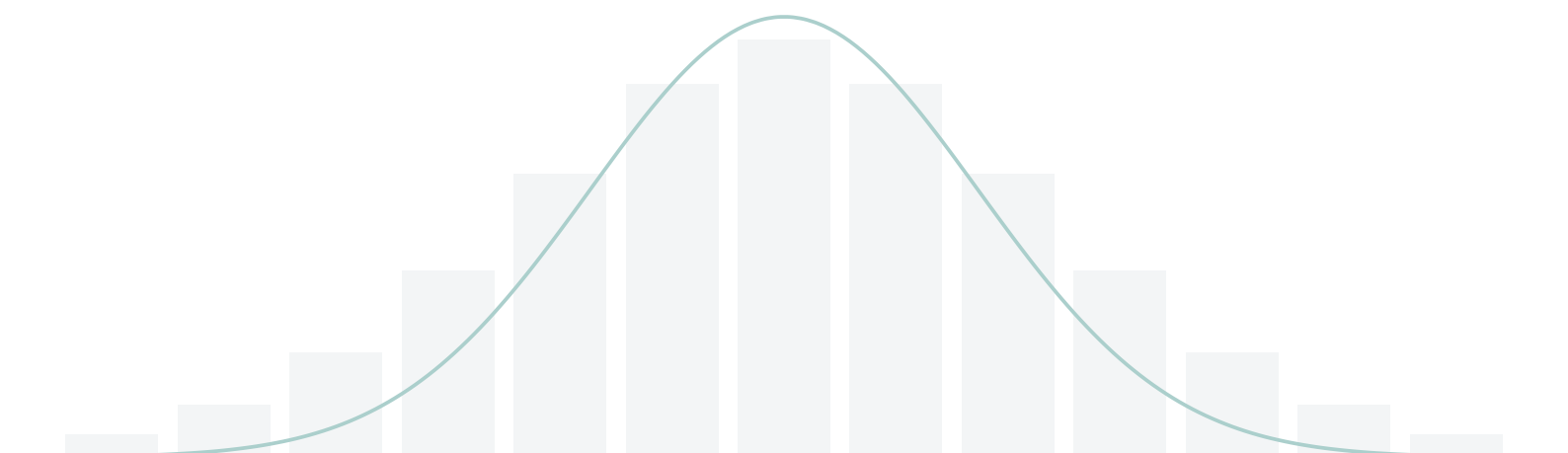
Next Module: Module 10 — Multiple Random Variables

Module 10

Multiple Random Variables

چند متغیر تصادفی

10



In This Module

10.1	Introduction: From 2 to N Variables	182
10.2	Joint Distributions for N Random Variables	183
10.3	Random Vectors	184
10.4	Mean Vector	185
10.5	Covariance Matrix	186
10.6	Multivariate Gaussian Distribution	187
10.7	Linear Transformations of Random Vectors	188
10.8	MATLAB Examples	189
10.9	Practice Problems	191

Learning Objectives

After completing this module, students will be able to:

1. Extend joint distributions to N random variables
2. Define and work with random vectors
3. Compute mean vectors and covariance matrices
4. Apply the multivariate Gaussian distribution
5. Perform linear transformations of random vectors

10.1 Introduction: From 2 to N Variables

Real systems often involve many uncertain quantities simultaneously:

- ▶ **Sensor array:** N sensors each producing a noisy measurement
- ▶ **MIMO system:** $M \times N$ channel coefficients
- ▶ **Parameter estimation:** p unknown parameters
- ▶ **Signal samples:** N consecutive noise samples

We need a framework for **N-dimensional** random vectors.

10.2 Joint Distributions for N Random Variables

■ Joint PDF

For $X_1, X_2, \dots, X_n$:

$$f_{X_1, \dots, X_n}(x_1, \dots, x_n) \geq 0, \quad \int \cdots \int f_{X_1, \dots, X_n} dx_1 \cdots dx_n = 1$$

■ Marginal Distributions

Integrate (or sum) out unwanted variables:

$$f_{X_1}(x_1) = \int \cdots \int f_{X_1, \dots, X_n}(x_1, \dots, x_n) dx_2 \cdots dx_n$$

■ Conditional Distributions

$$f_{X_1|X_2, \dots, X_n}(x_1|x_2, \dots, x_n) = \frac{f_{X_1, \dots, X_n}(x_1, \dots, x_n)}{f_{X_2, \dots, X_n}(x_2, \dots, x_n)}$$

■ Mutual Independence

$X_1, \dots, X_n$ are mutually independent if:

$$f_{X_1, \dots, X_n}(x_1, \dots, x_n) = \prod_{i=1}^n f_{X_i}(x_i)$$

10.3 Random Vectors

Definition

A **random vector** is an ordered collection of random variables:

$$\mathbf{X} = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_n \end{bmatrix}$$

■ Engineering Examples

Random Vector	Components	Context
Sensor array output	$X_1, \dots, X_n =$ sensor readings	Array processing
MIMO channel	$h_1, \dots, h_n =$ channel gains	Wireless communications
Noise vector	$N_1, \dots, N_n =$ noise samples	Signal processing
State vector	Position, velocity, acceleration	Kalman filtering
Feature vector	Extracted measurements	Pattern recognition

10.4 Mean Vector

Definition

$$\boldsymbol{\mu} = E[\mathbf{X}] = \begin{bmatrix} E[X_1] \\ E[X_2] \\ \vdots \\ E[X_n] \end{bmatrix} = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_n \end{bmatrix}$$

■ Properties

- ▶ $E[\mathbf{A}\mathbf{X} + \mathbf{b}] = \mathbf{A}E[\mathbf{X}] + \mathbf{b} = \mathbf{A}\boldsymbol{\mu} + \mathbf{b}$
- ▶ Linearity extends to vectors and matrices

10.5 Covariance Matrix

Definition

$$\Sigma = E[(\mathbf{X} - \boldsymbol{\mu})(\mathbf{X} - \boldsymbol{\mu})^T]$$

Element (i,j): $\Sigma_{ij} = \text{Cov}(X_i, X_j)$

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \text{Cov}(X_1, X_2) & \cdots & \text{Cov}(X_1, X_n) \\ \text{Cov}(X_2, X_1) & \sigma_2^2 & \cdots & \text{Cov}(X_2, X_n) \\ \vdots & & \ddots & \vdots \\ \text{Cov}(X_n, X_1) & \cdots & \cdots & \sigma_n^2 \end{bmatrix}$$

■ Properties

1. **Symmetric:** $\Sigma = \Sigma^T$
2. **Positive semi-definite:** $\mathbf{v}^T \Sigma \mathbf{v} \geq 0$ for all $\mathbf{v}$
3. **Diagonal = variances:** $\Sigma_{ii} = \text{Var}(X_i)$
4. **Off-diagonal = covariances:** $\Sigma_{ij} = \text{Cov}(X_i, X_j)$
5. **Independent components** $\rightarrow$ **diagonal** Σ

■ Alternative Formula

$$\Sigma = E[\mathbf{X}\mathbf{X}^T] - \boldsymbol{\mu}\boldsymbol{\mu}^T$$

10.6 Multivariate Gaussian Distribution

Definition

$\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ has PDF:

$$f_{\mathbf{X}}(\mathbf{x}) = \frac{1}{(2\pi)^{n/2} |\boldsymbol{\Sigma}|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right)$$

where $|\boldsymbol{\Sigma}|$ = determinant of $\boldsymbol{\Sigma}$.

■ Key Properties

1. **Marginals are Gaussian:** Any subset of components is jointly Gaussian
2. **Conditionals are Gaussian:** $X_1|X_2=x_2$ is Gaussian
3. **Linear transformations:** $\mathbf{Y} = \mathbf{A}\mathbf{X} + \mathbf{b} \sim \mathcal{N}(\mathbf{A}\boldsymbol{\mu} + \mathbf{b}, \mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^T)$
4. **Uncorrelated = Independent** (unique to Gaussian!)
5. **Fully specified** by $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ (only 2 parameters needed per variable plus correlations)

■ Why So Important?

- Central Limit Theorem: sums of many RVs $\rightarrow$ multivariate Gaussian
- Thermal noise vectors are multivariate Gaussian
- Analytically tractable (closed-form marginals, conditionals)
- Optimal filters (Kalman, Wiener) assume Gaussian

■ 2D Case (Bivariate Gaussian)

For $n=2$ with $\boldsymbol{\mu} = [0,0]^T$, $\sigma_1 = \sigma_2 = 1$:

$$f(x, y) = \frac{1}{2\pi\sqrt{1-\rho^2}} \exp \left(-\frac{x^2 - 2\rho xy + y^2}{2(1-\rho^2)} \right)$$

Contours are **ellipses** (tilted when $\rho \neq 0$).

10.7 Linear Transformations of Random Vectors

■ Transformation $\mathbf{Y} = \mathbf{A}\mathbf{X} + \mathbf{b}$

If $\mathbf{X}$ is a random vector with mean μ_X and covariance Σ_X :

$$E[\mathbf{Y}] = \mathbf{A}\mu_X + \mathbf{b}$$

$$\Sigma_Y = \mathbf{A}\Sigma_X\mathbf{A}^T$$

■ Engineering Applications

Beamforming: Output $y = \mathbf{w}^T \mathbf{X}$ (weight vector applied to sensor array)

- ▶ $E[y] = \mathbf{w}^T \mu$
- ▶ $\text{Var}(y) = \mathbf{w}^T \Sigma \mathbf{w}$

Whitening: Find matrix $\mathbf{W}$ such that $\mathbf{W}\Sigma\mathbf{W}^T = \mathbf{I}$ (decorrelates data)

- ▶ $\mathbf{W} = \Sigma^{-1/2}$

Principal Component Analysis: Rotate to diagonalize Σ

10.8 MATLAB Examples

Example 1: Generating Multivariate Gaussian

MATLAB

```

%% Generate and Visualize 2D Gaussian Random Vector

mu = [1; 2];
Sigma = [4, 2; 2, 3]; % Covariance matrix

N = 5000;
X = mvnrnd(mu', Sigma, N);

figure;
scatter(X(:,1), X(:,2), 5, 'b', '.');
hold on;
plot(mu(1), mu(2), 'r+', 'MarkerSize', 15, 'LineWidth', 3);
xlabel('X_1'); ylabel('X_2');
title('Bivariate Gaussian Samples');
axis equal; grid on;

% Verify statistics
fprintf('Mean: [%.2f, %.2f] (theory: [%.1f, %.1f])\n', mean(X), mu');
fprintf('Cov matrix:\n'); disp(cov(X));

```

Example 2: Linear Transformation

MATLAB

```

%% Transform: Y = AX + b
mu_x = [0; 0]; Sigma_x = eye(2); % Standard normal
A = [2, 1; -1, 3]; b = [1; -2];

N = 10000;
X = mvnrnd(mu_x', Sigma_x, N);
Y = A*X + b;

% Theoretical
mu_y_theory = A*mu_x + b;
Sigma_y_theory = A*Sigma_x*A';

fprintf('E[Y] theory: [%.1f; %.1f], sim: [%.2f; %.2f]\n', ...
        mu_y_theory, mean(Y,2));
fprintf('Σ_Y theory:\n'); disp(Sigma_y_theory);
fprintf('Σ_Y simulated:\n'); disp(cov(Y));

```

Example 3: Sensor Array with Correlated Noise

MATLAB

```

%% 4-Element Sensor Array with Spatially Correlated Noise
n_sensors = 4;
sigma2 = 1;      % Noise power per sensor
rho = 0.6;       % Adjacent sensor correlation

% Build covariance matrix (exponential decay model)
Sigma = zeros(n_sensors);
for i = 1:n_sensors
    for j = 1:n_sensors
        Sigma(i,j) = sigma2 * rho^abs(i-j);
    end
end

fprintf('Noise covariance matrix:\n'); disp(Sigma);

% Generate noise samples
N_samp = 10000;
L = chol(Sigma, 'lower');
noise = L * randn(n_sensors, N_samp);

% Signal present in direction: s = [1; 1; 1; 1] (broadside)
signal = 3;
s = ones(n_sensors, 1) / sqrt(n_sensors);
received = signal * s + noise; % N_sensors x N_samp

% Beamformer output: y = w'*received (w = s for matched filter)
w = s;
y = w' * received;
SNR_out = signal^2 / (w'*Sigma*w);
fprintf('Output SNR = %.2f (%.1f dB)\n', SNR_out, 10*log10(SNR_out));

```

10.9 Practice Problems

Problem 1

Random vector $X = [X_1, X_2, X_3]^T$ with $\mu = [1, 0, -1]^T$, and covariance matrix $\Sigma = \begin{bmatrix} 4 & 1 & 0 \\ 1 & 2 & -1 \\ 0 & -1 & 3 \end{bmatrix}$.

(a) Find $\text{Var}(X_1 + X_2 + X_3)$. (b) Find $\text{Cov}(X_1, X_2 + X_3)$. (c) Are X_1 and X_3 uncorrelated?

Solution

(a) $\text{Var}(X_1 + X_2 + X_3) = 1^T \Sigma 1 = 4 + 2 + 3 + 2(1) + 2(0) + 2(-1) = 9 + 0 = 9$

Wait: $= \Sigma_{ij}$ all entries $= 4 + 1 + 0 + 1 + 2 + (-1) + 0 + (-1) + 3 = 9$

(b) $\text{Cov}(X_1, X_2 + X_3) = \text{Cov}(X_1, X_2) + \text{Cov}(X_1, X_3) = 1 + 0 = 1$

(c) $\text{Cov}(X_1, X_3) = 0$, so yes, X_1 and X_3 are uncorrelated.

Problem 2

$X \sim N(0, \Sigma)$ with $\Sigma = \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}$. Find the conditional distribution of $X_1 | X_2 = x_2$.

Solution For bivariate Gaussian:

$X_1 | X_2 = x_2 \sim N(\mu_1 + \rho(\sigma_1/\sigma_2)(x_2 - \mu_2), \sigma_1^2(1 - \rho^2))$

$= N(\rho x_2, 1 - \rho^2)$

The conditional mean is a linear function of x_2 , and conditional variance is reduced by factor $(1 - \rho^2)$.

Problem 3

Write MATLAB code to generate a 3D Gaussian vector with given μ and Σ , compute the sample covariance, and verify.

Solution

```
mu = [1; 0; -1];
Sigma = [4 1 0; 1 2 -1; 0 -1 3];
N = 50000;
X = mvnrnd(mu', Sigma, N);
fprintf('Sample mean: '); disp(mean(X));
fprintf('Sample cov:\n'); disp(cov(X));
```

MATLAB

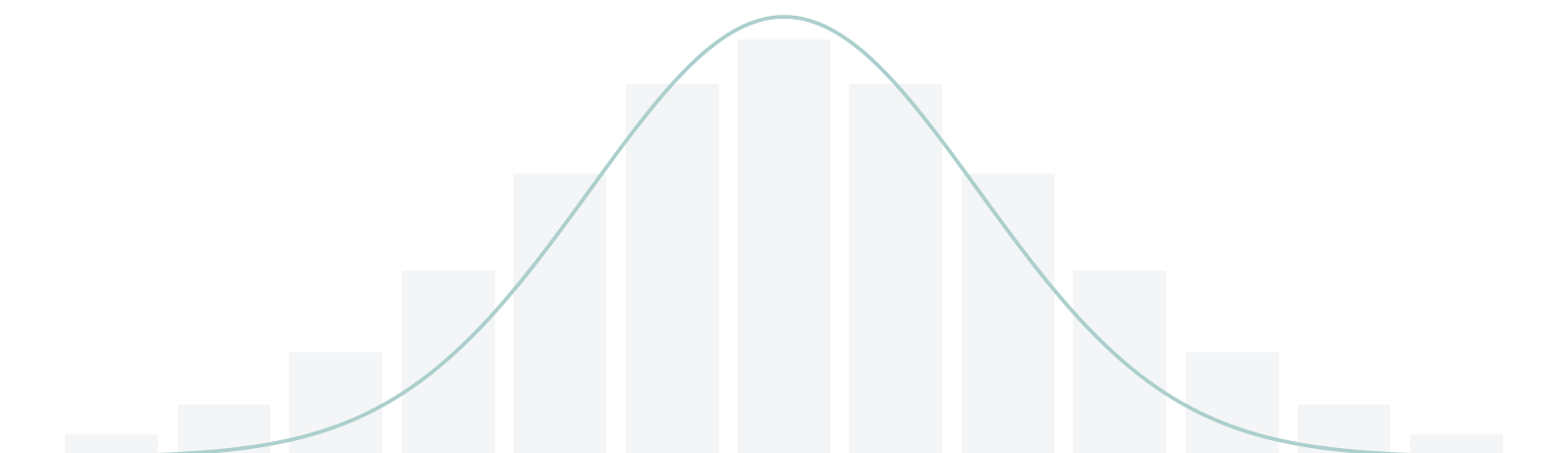
Next Module: Module 11 — Central Limit Theorem

Module 11

Central Limit Theorem

قضیه حد مرکزی

11



In This Module

11.1	Introduction: Why Is the Gaussian Everywhere?	195
11.2	Law of Large Numbers (LLN)	196
11.3	Central Limit Theorem (CLT)	197
11.4	Rate of Convergence	198
11.5	Visual Demonstration	199
11.6	Engineering Significance	200
11.7	MATLAB Monte Carlo Demonstrations	201
11.8	Approximations Using CLT	204
11.9	Practice Problems	205

Learning Objectives

After completing this module, students will be able to:

1. State and explain the Law of Large Numbers
2. State and apply the Central Limit Theorem
3. Explain why Gaussian distributions appear so frequently in engineering
4. Use CLT for approximations
5. Demonstrate CLT visually using MATLAB Monte Carlo simulations

11.1 Introduction: Why Is the Gaussian Everywhere?

The Gaussian distribution appears in:

- Thermal noise, measurement errors, manufacturing tolerances, aggregate interference, stock fluctuations

The reason: All these quantities are sums (or averages) of many small, independent random effects. The CLT guarantees convergence to Gaussian regardless of the underlying distribution.

11.2 Law of Large Numbers (LLN)

■ Weak Law of Large Numbers

For $X_1, X_2, \dots, X_n$ i.i.d. with mean μ and finite variance σ^2 :

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i \xrightarrow{P} \mu \quad \text{as } n \rightarrow \infty$$

(“Convergence in probability”: $P(|\bar{X}_n - \mu| > \varepsilon) \rightarrow 0$ for any $\varepsilon > 0$)

■ Strong Law of Large Numbers

$$\bar{X}_n \xrightarrow{a.s.} \mu \quad \text{as } n \rightarrow \infty$$

(“Almost sure convergence”: $P(\bar{X}_n \rightarrow \mu) = 1$)

■ Engineering Significance

- ▶ **Sample average converges to true mean** as sample size grows
- ▶ Justifies estimating means from data
- ▶ Foundation of Monte Carlo simulation (simulate many times $\rightarrow$ average converges)
- ▶ Justifies: “If I measure enough times, my average will be close to the truth”

■ How Fast?

By Chebyshev: $P(|\bar{X}_n - \mu| > \varepsilon) \leq \sigma^2/(n\varepsilon^2)$

For 95% confidence that error $< \varepsilon$: need $n \geq \sigma^2/(0.05 \cdot \varepsilon^2)$

11.3 Central Limit Theorem (CLT)

■ Formal Statement

For $X_1, X_2, \dots, X_n$ i.i.d. with mean μ and finite variance σ^2 :

$$Z_n = \frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} = \frac{\sum_{i=1}^n X_i - n\mu}{\sigma\sqrt{n}} \xrightarrow{d} N(0, 1)$$

■ Practical Form

For large n :

$$\bar{X}_n \approx N\left(\mu, \frac{\sigma^2}{n}\right)$$

$$S_n = \sum_{i=1}^n X_i \approx N(n\mu, n\sigma^2)$$

■ Conditions

1. $X_1, \dots, X_n$ are independent
2. Identically distributed (can be relaxed with Lyapunov/Lindeberg CLT)
3. Finite mean μ and finite variance σ^2
4. n is “large enough” (rule of thumb: $n \geq 30$, but depends on distribution shape)

■ What CLT Does NOT Require

- Does NOT require X_i to be Gaussian
- Does NOT require X_i to be continuous
- Does NOT require any specific distribution

11.4 Rate of Convergence

How fast does the CLT “kick in”?

Original Distribution	n needed for good approximation
Symmetric (uniform)	$n \approx 5-10$
Mildly skewed	$n \approx 20-30$
Highly skewed (exponential)	$n \approx 30-50$
Extremely skewed	$n \approx 50-100+$

Rule: More symmetric distributions → faster convergence.

11.5 Visual Demonstration

■ Sum of Uniform RVs → Gaussian

- ▶ $n=1$: Uniform (flat)
- ▶ $n=2$: Triangular
- ▶ $n=3$: Starts looking bell-shaped
- ▶ $n=5$: Nearly Gaussian
- ▶ $n=12$: Practically indistinguishable from Gaussian

■ Sum of Exponential RVs → Gaussian

- ▶ $n=1$: Exponential (heavily right-skewed)
- ▶ $n=3$: Still skewed but less so (Gamma(3))
- ▶ $n=10$: Nearly symmetric, bell-shaped
- ▶ $n=30$: Essentially Gaussian

■ Binomial → Normal Approximation

$\text{Binomial}(n, p) \approx N(np, np(1-p))$ for large n

This is CLT applied to sum of Bernoulli trials!

- ▶ Continuity correction: $P(X \leq k) \approx \Phi((k + 0.5 - np)/(\sqrt{np(1-p)}))$

11.6 Engineering Significance

■ Why Thermal Noise is Gaussian

Thermal noise = aggregate effect of billions of electrons moving randomly. Each electron's contribution is tiny and independent $\rightarrow$ CLT applies $\rightarrow$ noise is Gaussian.

■ Why Aggregate Interference is Gaussian

In a cellular network, total interference = sum of signals from many independent users. CLT $\rightarrow$ interference $\approx$ Gaussian (for many users).

■ Why Measurement Errors are Gaussian

Total error = sum of many independent small error sources (calibration, quantization, thermal, vibration, etc.) $\rightarrow$ CLT $\rightarrow$ error $\approx$ Gaussian.

■ Why Manufacturing Variations are Gaussian

Resistor value = nominal + sum of many small process variations (doping, etching, temperature, etc.) $\rightarrow$ CLT $\rightarrow$ value $\approx$ Gaussian around nominal.

11.7 MATLAB Monte Carlo Demonstrations

Example 1: CLT with Uniform Distribution

MATLAB

```
% CLT: Sum of Uniform RVs approaches Gaussian
N_experiments = 50000;
n_values = [1, 2, 5, 12, 30];

figure;
for idx = 1:length(n_values)
    n = n_values(idx);

    % Generate sum of n Uniform(0,1) RVs
    X = sum(rand(n, N_experiments), 1);

    % Standardize: Z = (X - nμ)/(σ√n) where μ=0.5, σ²=1/12
    mu_sum = n * 0.5;
    sigma_sum = sqrt(n / 12);
    Z = (X - mu_sum) / sigma_sum;

    subplot(2, 3, idx);
    histogram(Z, 60, 'Normalization', 'pdf');
    hold on;
    z = linspace(-4, 4, 200);
    plot(z, normpdf(z), 'r', 'LineWidth', 2);
    title(sprintf('n = %d', n));
    xlabel('Standardized Sum'); xlim([-4 4]);
    if idx == 1, ylabel('PDF'); end
end
sgtitle('CLT: Sum of Uniform RVs → Gaussian');
```

Example 2: CLT with Exponential Distribution

MATLAB

```

%% CLT: Sample Mean of Exponential(1) converges to Gaussian
lambda = 1; mu = 1; sigma = 1;
N_experiments = 100000;
n_values = [1, 3, 10, 30, 100];

figure;
for idx = 1:length(n_values)
    n = n_values(idx);

    % Generate sample means
    X = exprnd(1, n, N_experiments);
    X_bar = mean(X, 1);

    subplot(2, 3, idx);
    histogram(X_bar, 80, 'Normalization', 'pdf');
    hold on;
    x = linspace(0, max(X_bar), 200);
    plot(x, normpdf(x, mu, sigma/sqrt(n)), 'r', 'LineWidth', 2);
    title(sprintf('n = %d', n));
    xlabel('Sample Mean');
end
sgtitle('CLT: Mean of Exponential → Gaussian');

```

Example 3: Binomial Normal Approximation

MATLAB

```

%% Binomial(n,p) approximated by Gaussian
n = 50; p = 0.3;
k = 0:n;
pmf_binom = binopdf(k, n, p);

% Normal approximation
mu_approx = n*p;
sigma_approx = sqrt(n*p*(1-p));
pdf_normal = normpdf(k, mu_approx, sigma_approx);

figure;
bar(k, pmf_binom, 'FaceAlpha', 0.5);
hold on;
plot(k, pdf_normal, 'r-', 'LineWidth', 2);
xlabel('k'); ylabel('Probability');
title(sprintf('Binomial(%d, %.1f) vs Normal(%d, %.2f)', n, p, mu_approx,
    sigma_approx^2));
legend('Binomial (exact)', 'Normal approximation');

```

■ Example 4: Complete Monte Carlo CLT Demonstration

MATLAB

```

%% Full CLT demonstration: non-Gaussian → Gaussian via averaging
% Generate N samples from a VERY non-Gaussian distribution
% (mixture: 80% from Exp(1), 20% from Exp(0.1))
N_experiments = 100000;
n_samples = 50;

% Generate raw data
raw = zeros(n_samples, N_experiments);
for i = 1:N_experiments
    mask = rand(n_samples, 1) < 0.8;
    raw(mask, i) = exprnd(1, sum(mask), 1);
    raw(~mask, i) = exprnd(10, sum(~mask), 1);
end

% True mean and variance of mixture
mu_true = 0.8*1 + 0.2*10; % = 2.8
var_true = 0.8*(1+1^2) + 0.2*(100+10^2) - mu_true^2; % Second moment - mean^2

% Compute sample means
X_bar = mean(raw, 1);

figure;
subplot(2,1,1);
histogram(raw(:,1), 100, 'Normalization', 'pdf');
title('Original Distribution (highly skewed mixture)');
xlabel('X'); ylabel('PDF');

subplot(2,1,2);
histogram(X_bar, 100, 'Normalization', 'pdf');
hold on;
x = linspace(min(X_bar), max(X_bar), 200);
plot(x, normpdf(x, mu_true, sqrt(var_true/n_samples)), 'r', 'LineWidth', 2);
title(sprintf('Sample Mean (n=%d) - Gaussian by CLT!', n_samples));
xlabel('Sample Mean'); ylabel('PDF');
legend('Empirical', 'Gaussian approximation');

```

11.8 Approximations Using CLT

■ General Procedure

To approximate $P(S_n \leq x)$ where $S_n = X_1 + \dots + X_n$:

1. Compute $\mu = E[X_i]$, $\sigma^2 = \text{Var}(X_i)$
2. $S_n \approx N(n\mu, n\sigma^2)$
3. $P(S_n \leq x) \approx \Phi((x - n\mu)/(\sigma\sqrt{n}))$

■ Continuity Correction (for Discrete RVs)

When approximating a discrete sum by Gaussian:

- ▶ $P(X \leq k) \approx \Phi((k + 0.5 - n\mu)/(\sigma\sqrt{n}))$
- ▶ $P(X = k) \approx \Phi((k + 0.5 - n\mu)/(\sigma\sqrt{n})) - \Phi((k - 0.5 - n\mu)/(\sigma\sqrt{n}))$

Example: Packet Errors

1000 packets, each error probability 0.02. X = total errors.

Exact: $X \sim \text{Binomial}(1000, 0.02)$

CLT approx: $X \approx N(20, 19.6)$

$P(X > 30) \approx 1 - \Phi((30.5 - 20)/\sqrt{19.6}) = 1 - \Phi(2.37) = 0.0089$

11.9 Practice Problems

Problem 1

A factory produces bolts. Length $X \sim$ distribution with $\mu = 10\text{cm}$, $\sigma = 0.2\text{cm}$. A batch of 100 bolts is measured.

- (a) What is the approximate distribution of $\bar{X}$? (b) $P(\bar{X} > 10.05)$? (c) $P(9.96 < \bar{X} < 10.04)$?

Solution

- (a) $\bar{X} \sim N(10, 0.04/100) = N(10, 0.0004)$, $\sigma_{\bar{X}} = 0.02$
 (b) $P(\bar{X} > 10.05) = P(Z > 2.5) = 0.0062$
 (c) $P(9.96 < \bar{X} < 10.04) = \Phi(2) - \Phi(-2) = 0.9545$

Problem 2

A communication link has 10000 bits with $\text{BER} = 0.001$. Approximate $P(\text{more than 15 errors})$ using CLT.

Solution $X \sim \text{Binomial}(10000, 0.001)$. $\mu=10$, $\sigma^2=9.99$.

$$P(X > 15) \approx 1 - \Phi((15.5-10)/\sqrt{9.99}) = 1 - \Phi(1.74) = 0.0409$$

Problem 3

Write MATLAB code to verify the CLT for chi-squared(1) distribution (very skewed) and show convergence for $n=2,5,10,50$.

Solution

```
N = 100000; ns = [2 5 10 50];
figure;
for i = 1:4
    X_bar = mean(chi2rnd(1, ns(i), N), 1);
    subplot(2,2,i);
    histogram(X_bar, 80, 'Normalization', 'pdf'); hold on;
    x = linspace(0, 3, 200);
    plot(x, normpdf(x, 1, sqrt(2/ns(i))), 'r', 'LineWidth', 2);
    title(sprintf('n=%d', ns(i)));
end
sgtitle('CLT for  $\chi^2(1)$ : Mean→Gaussian');
```

MATLAB

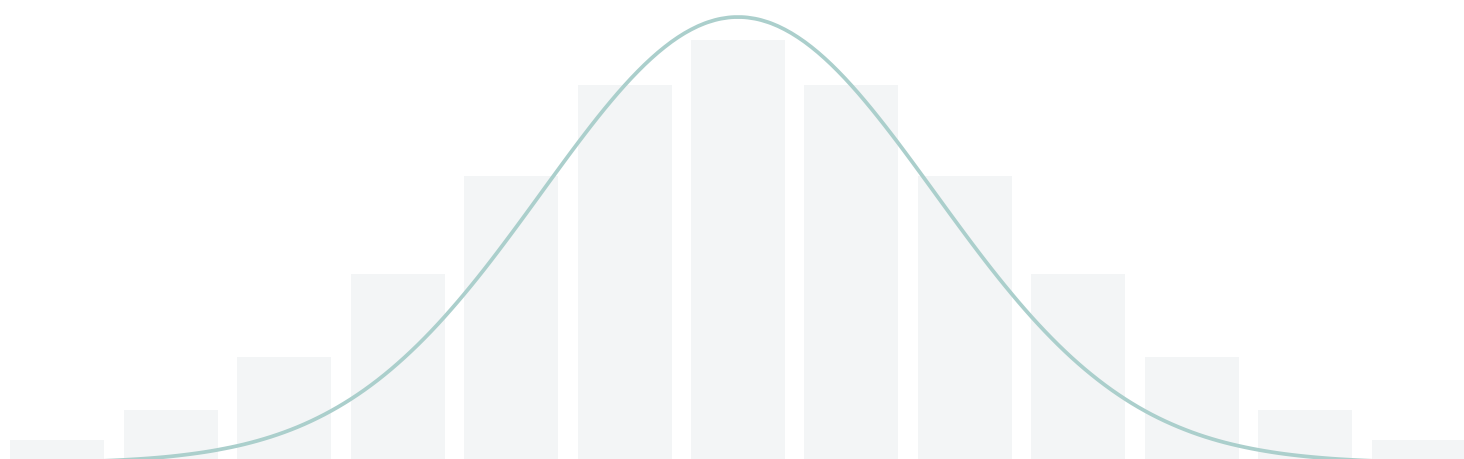
Next Module: Module 12 — Introduction to Statistics

Module 12

Introduction to Statistics

مقدمه‌ای بر آمار

12



In This Module

12.1	Introduction: From Probability to Statistics	209
12.2	Population vs. Sample	210
12.3	Parameter vs. Statistic	211
12.4	Random Sample	212
12.5	Sampling Distributions	213
12.6	Sample Mean $\bar{X}$	214
12.7	Sample Variance S^2	215
12.8	Sample Standard Deviation	216
12.9	Connection to CLT	217
12.10	MATLAB Examples	218
12.11	Practice Problems	220

Learning Objectives

After completing this module, students will be able to:

1. Distinguish between population and sample
2. Distinguish between parameter and statistic
3. Define and compute sample mean and sample variance
4. Explain sampling distributions and why they matter
5. Understand the transition from probability (model→predictions) to statistics (data→model)

12.1 Introduction: From Probability to Statistics

■ The Two Directions

Probability	Statistics
Model is KNOWN	Model is UNKNOWN
Predict outcomes	Infer model from data
Forward problem	Inverse problem
“Given distribution, what data?”	“Given data, what distribution?”

Probability (Modules 1–11): Given a model $\rightarrow$ predict observations.

Statistics (Modules 12–16): Given observations $\rightarrow$ infer the model.

■ Engineering Context

- **Probability:** “If noise is $N(0, 0.01)$, what is $P(\text{error})$?”
- **Statistics:** “I measured 1000 noise samples. What is σ^2 ?”

12.2 Population vs. Sample

■ Population

The **complete** set of ALL items/measurements of interest.

Engineering Context	Population
All chips ever produced by a fab	ALL chip delays
The noise process forever	ALL noise voltage values
All network packets ever sent	ALL packet delays

Populations are usually **infinite or impractically large** — we cannot measure everything.

■ Sample

A **subset** of the population that we actually observe/measure.

Engineering Context	Sample
50 chips tested from production line	Measured delays
10000 noise voltage samples recorded	Recorded values
500 packet delays measured	Observed delays

■ Why We Sample

- ▶ Cannot test every chip (destructive testing)
- ▶ Cannot record noise forever (finite memory)
- ▶ Cannot measure every packet (too many)

The goal: **Infer properties of the population from the sample.**

12.3 Parameter vs. Statistic

■ Parameter

A **fixed** (but unknown) number that describes the population:

- ▶ μ = true population mean
- ▶ σ^2 = true population variance
- ▶ p = true defect rate
- ▶ λ = true failure rate

Parameters are **constants** — they have ONE true value (we just don't know it).

■ Statistic

A **function of sample data** used to estimate or test parameters:

- ▶ $\bar{X}$ = sample mean (estimates μ)
- ▶ S^2 = sample variance (estimates σ^2)
- ▶ $\hat{p}$ = sample proportion (estimates p)

▲ **Key insight:** Statistics are RANDOM VARIABLES! Different samples give different values.

Example

True noise power: $\sigma^2 = 0.04 \text{ V}^2$ (parameter — fixed, unknown)

Experiment 1: Collect 100 samples → compute $S^2 = 0.038$ (one realization)

Experiment 2: Collect 100 samples → compute $S^2 = 0.042$ (different realization)

Experiment 3: Collect 100 samples → compute $S^2 = 0.041$ (another realization)

Each S^2 is different because of randomness in sampling.

12.4 Random Sample

Definition

A **random sample** $X_1, X_2, \dots, X_n$ is a set of n independent, identically distributed (i.i.d.) random variables from the population distribution.

■ Requirements

1. **Independent:** Each observation doesn't influence others
2. **Identically distributed:** All come from the same population
3. **Random selection:** No systematic bias in choosing which items to measure

■ When i.i.d. Applies

- ✓ Random selection of components from a large batch
- ✓ Independent measurements of a stable process
- ✓ Noise samples separated by more than the correlation time

■ When i.i.d. Fails

- ✗ Consecutive samples of a correlated signal
- ✗ Measurements during a drifting process
- ✗ Non-random selection (e.g., only testing “suspicious” components)

12.5 Sampling Distributions

Definition

The **sampling distribution** of a statistic is the probability distribution of that statistic across all possible samples of size n .

■ Why It Matters

A statistic (like $\bar{X}$) is a random variable. Its distribution tells us:

- ▶ How much will it vary from sample to sample?
- ▶ How close is it likely to be to the true parameter?
- ▶ How should we quantify our uncertainty?

This is the foundation for confidence intervals and hypothesis testing.

12.6 Sample Mean $\bar{X}$

Definition

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

■ Properties

Mean of $\bar{X}$:

$$E[\bar{X}] = \mu$$

$\bar{X}$ is an **unbiased** estimator of μ (on average, it equals the truth).

Variance of $\bar{X}$:

$$\text{Var}(\bar{X}) = \frac{\sigma^2}{n}$$

More measurements $\rightarrow$ less variability $\rightarrow$ more precise estimate.

Standard Error:

$$\text{SE}(\bar{X}) = \frac{\sigma}{\sqrt{n}}$$

■ Sampling Distribution of $\bar{X}$

- If population is Gaussian: $\bar{X} \sim N(\mu, \sigma^2/n)$ **exactly** for any n
- If population is NOT Gaussian: $\bar{X} \approx N(\mu, \sigma^2/n)$ **approximately** for large n (by CLT)

12.7 Sample Variance S^2

Definition

$$S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2$$

■ Why n-1? (Bessel's Correction)

Using n in the denominator gives a **biased** estimator (systematically underestimates σ^2):

$$E[1/n \cdot \sum (X_i - \bar{X})^2] = ((n-1)/n)\sigma^2 \neq \sigma^2$$

Using $n-1$ corrects this:

$$E[S^2] = \sigma^2$$

Intuition: $\bar{X}$ is computed from the same data, so deviations from $\bar{X}$ are smaller than deviations from μ . We “lose one degree of freedom” by estimating the mean.

■ Degrees of Freedom

With n data points and $\bar{X}$ estimated:

- ▶ The n deviations $(X_i - \bar{X})$ sum to zero: $\sum (X_i - \bar{X}) = 0$
- ▶ Only $n-1$ are “free” — the last is determined
- ▶ $df = n - 1$

■ Sampling Distribution (Normal Population)

If population is $N(\mu, \sigma^2)$:

$$(n-1)S^2/\sigma^2 \sim \chi^2(n-1)$$

12.8 Sample Standard Deviation

$$S = \sqrt{S^2} = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2}$$

Note: $E[S] \neq \sigma$ (S is slightly biased for σ , though S^2 is unbiased for σ^2).

12.9 Connection to CLT

From Module 11: $\bar{X} \rightarrow N(\mu, \sigma^2/n)$ as $n \rightarrow \infty$.

This means the sampling distribution of $\bar{X}$ is approximately Gaussian. This is what allows us to:

1. Build confidence intervals (Module 14)
2. Perform hypothesis tests (Module 15)
3. Quantify uncertainty in estimates

12.10 MATLAB Examples

Example 1: Sampling Distribution of $\bar{X}$

MATLAB

```

%% Demonstrate:  $\bar{X}$  is a random variable with its own distribution
% Population: Exponential( $\lambda=1$ ),  $\mu=1$ ,  $\sigma^2=1$ 
mu_pop = 1; sigma2_pop = 1;
n = 25; % Sample size
N_experiments = 10000;

% Repeat experiment N_experiments times
X_bar = zeros(1, N_experiments);
for i = 1:N_experiments
    sample = exprnd(1, 1, n);
    X_bar(i) = mean(sample);
end

figure;
histogram(X_bar, 60, 'Normalization', 'pdf');
hold on;
x = linspace(0, 3, 200);
plot(x, normpdf(x, mu_pop, sqrt(sigma2_pop/n)), 'r', 'LineWidth', 2);
xlabel('Sample Mean'); ylabel('PDF');
title(sprintf('Sampling Distribution of  $\bar{X}$  (n=%d)', n));
legend('Empirical', sprintf('N(%0.1f, %0.4f)', mu_pop, sigma2_pop/n));

fprintf('E[ $\bar{X}$ ] = %0.4f (theory: %0.4f)\n', mean(X_bar), mu_pop);
fprintf('Var( $\bar{X}$ ) = %0.4f (theory: %0.4f)\n', var(X_bar), sigma2_pop/n);

```

Example 2: Sample Variance Distribution

MATLAB

```

%% Sampling Distribution of  $S^2$ 
% Population:  $N(0, 4)$ ,  $\sigma^2 = 4$ 
sigma2 = 4; n = 10;
N_experiments = 10000;

S2 = zeros(1, N_experiments);
for i = 1:N_experiments
    sample = sqrt(sigma2) * randn(1, n);
    S2(i) = var(sample); % Uses  $n-1$  denominator
end

fprintf('E[S^2] = %.4f (theory: %.4f)\n', mean(S2), sigma2);
fprintf('Var(S^2) = %.4f (theory: %.4f)\n', var(S2), 2*sigma2^2/(n-1));

figure;
histogram(S2, 80, 'Normalization', 'pdf');
hold on;
x = linspace(0, 15, 200);
%  $(n-1)S^2/\sigma^2 \sim \chi^2(n-1)$ , so  $S^2 \sim \sigma^2/(n-1) * \chi^2(n-1)$ 
plot(x, chi2pdf(x*(n-1)/sigma2, n-1)*(n-1)/sigma2, 'r', 'LineWidth', 2);
xlabel('S^2'); ylabel('PDF');
title(sprintf('Sampling Distribution of  $S^2$  (n=%d,  $\sigma^2$ =%d)', n, sigma2));
legend('Empirical', 'Theoretical (scaled  $\chi^2$ )');

```

Example 3: Effect of Sample Size

MATLAB

```

%% Precision improves with  $n$ :  $\text{Var}(\bar{X}) = \sigma^2/n$ 
sigma = 2;
n_values = [5, 10, 25, 50, 100, 500];
N_exp = 10000;

fprintf('n\t| SE( $\bar{X}$ ) Theory\t| SE( $\bar{X}$ ) Sim\n');
fprintf('-----+-----+-----\n');
for n = n_values
    X_bar = mean(sigma*randn(n, N_exp), 1);
    fprintf('%d\t| %.4f\t| %.4f\n', n, sigma/sqrt(n), std(X_bar));
end

```

12.11 Practice Problems**Problem 1**

A population has $\mu = 50$ and $\sigma = 10$. A random sample of $n = 64$ is taken.

(a) What is $E[\bar{X}]$? (b) What is $\text{Var}(\bar{X})$? (c) What is $P(48 < \bar{X} < 52)$?

Solution

(a) $E[\bar{X}] = 50$

(b) $\text{Var}(\bar{X}) = 100/64 = 1.5625$, $\text{SE} = 1.25$

(c) $P(48 < \bar{X} < 52) = P(-1.6 < Z < 1.6) = 2\Phi(1.6) - 1 = 0.8904$

Problem 2

Why do we use $n-1$ in the sample variance formula instead of n ? Explain both mathematically and intuitively.

Solution Mathematically: $E[(1/n)\Sigma(X_i - \bar{X})^2] = (n-1)/n \cdot \sigma^2$, so dividing by $n-1$ gives $E[S^2] = \sigma^2$. Intuitively: we use $\bar{X}$ from the same data (not the true μ), so deviations from $\bar{X}$ are artificially small. One degree of freedom is “used up” estimating the mean.

Problem 3

An engineer measures noise power from 20 samples and gets $S^2 = 0.045 \text{ V}^2$. The true variance is $\sigma^2 = 0.04 \text{ V}^2$. Is this concerning? Use the sampling distribution to assess.

Solution Under H_0 : $\sigma^2=0.04$, $(n-1)S^2/\sigma^2 \sim \chi^2(19)$. Observed: $19(0.045)/0.04 = 21.375$. $P(\chi^2(19) > 21.375) \approx 0.31$. This is not unusual — about 31% of samples would give $S^2 \geq 0.045$ when $\sigma^2=0.04$. Not concerning.

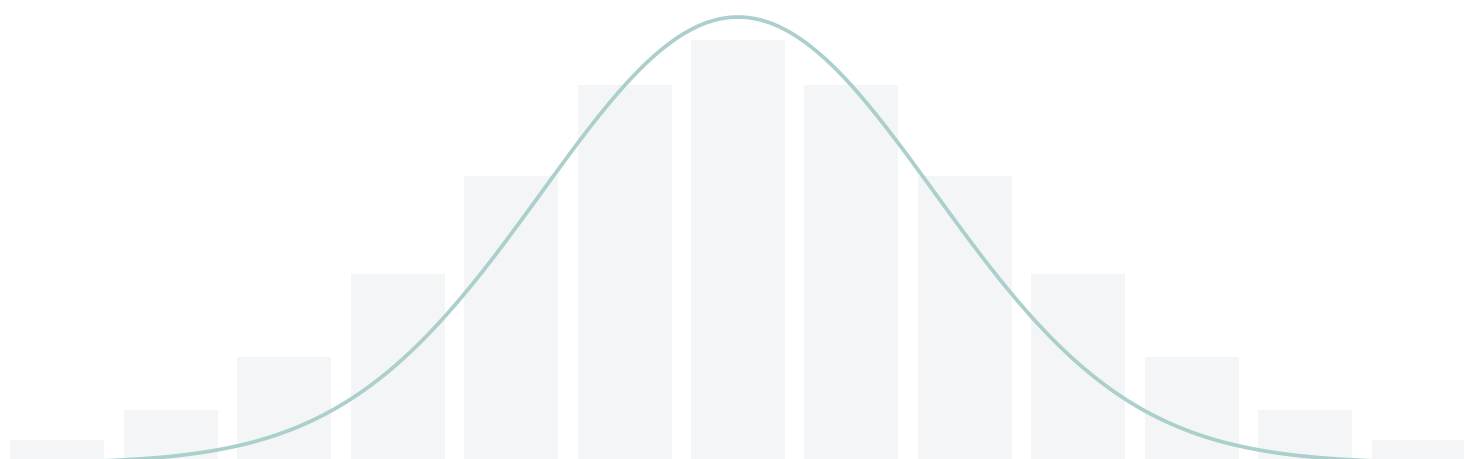
Next Module: Module 13 — Parameter Estimation

Module 13

Parameter Estimation

برآورد پارامتر

13



In This Module

13.1	Introduction	224
13.2	Estimator vs. Estimate	225
13.3	Properties of Estimators	226
13.4	Method of Moments (MoM)	227
13.5	Maximum Likelihood Estimation (MLE)	228
13.6	Engineering Examples	230
13.7	Comparing Estimators	231
13.8	MATLAB Examples	232
13.9	Practice Problems	234

Learning Objectives

After completing this module, students will be able to:

1. Distinguish between estimator (rule) and estimate (number)
2. Evaluate estimator properties: bias, consistency, efficiency, MSE
3. Apply the Method of Moments
4. Apply Maximum Likelihood Estimation (MLE)
5. Compare estimators using MSE and bias-variance tradeoff

13.1 Introduction

We observe data $X_1, \dots, X_n$ from a distribution with unknown parameter θ . **Parameter estimation** gives us a method to infer θ from data.

Engineering examples:

- ▶ Estimate noise variance σ^2 from recorded samples
- ▶ Estimate failure rate λ from component lifetimes
- ▶ Estimate channel gain from received signal strength
- ▶ Estimate mean signal level from measurements

13.2 Estimator vs. Estimate

■ Definitions

Term	Type	Definition	Example
Estimator $\hat{\theta}$	Random variable (function of data)	The RULE applied to data	$\hat{\theta} = \bar{X} = (1/n)\sum X_i$
Estimate	Number	The VALUE from a specific sample	$\hat{\theta} = 3.47$ (from one dataset)

■ Key Insight

The **estimator** is a random variable — it has a sampling distribution.

The **estimate** is one realization of that random variable.

Analogy: “Take the average” is the estimator (rule). “The average was 3.47” is the estimate (result).

13.3 Properties of Estimators

■ Bias

$$B(\hat{\theta}) = E[\hat{\theta}] - \theta$$

- **Unbiased:** $B(\hat{\theta}) = 0$, i.e., $E[\hat{\theta}] = \theta$ (on average, correct)
- **Biased:** $E[\hat{\theta}] \neq \theta$ (systematic error)

Examples:

- $\bar{X}$ is unbiased for μ : $E[\bar{X}] = \mu$ ✓
- S^2 (with $n-1$) is unbiased for σ^2 : $E[S^2] = \sigma^2$ ✓
- S (sample std dev) is biased for σ : $E[S] \neq \sigma$ (slight downward bias)

■ Consistency

$\hat{\theta}_n$ is **consistent** if $\hat{\theta}_n \rightarrow \theta$ in probability as $n \rightarrow \infty$.

Practical meaning: with enough data, the estimator gets arbitrarily close to the truth.

Sufficient condition: If bias $\rightarrow 0$ and variance $\rightarrow 0$ as $n \rightarrow \infty$, then consistent.

■ Efficiency

Among all unbiased estimators, the **efficient** one has the smallest variance.

The **Cramér-Rao Lower Bound** (CRLB) gives the minimum possible variance:

$$\text{Var}(\hat{\theta}) \geq \frac{1}{nI(\theta)}$$

where $I(\theta)$ = Fisher information = $-E[\partial^2 \ln f(X;\theta)/\partial \theta^2]$.

An estimator achieving the CRLB is called **efficient** or **minimum variance unbiased (MVU)**.

■ Mean Squared Error (MSE)

$$\text{MSE}(\hat{\theta}) = E[(\hat{\theta} - \theta)^2] = \text{Var}(\hat{\theta}) + [B(\hat{\theta})]^2$$

MSE = Variance + Bias²

This is the **bias-variance tradeoff**: sometimes a slightly biased estimator with much lower variance has smaller MSE than an unbiased one.

13.4 Method of Moments (MoM)

■ Principle

Set population moments equal to sample moments, then solve for parameters.

- ▶ k-th population moment: $\mu'_k = E[X^k]$
- ▶ k-th sample moment: $m'_k = (1/n)\sum X_i^k$

■ Procedure

1. Express parameters in terms of population moments
2. Replace population moments with sample moments
3. Solve for parameter estimates

Example: Exponential Distribution

Data from $\text{Exp}(\lambda)$. Population moment: $E[X] = 1/\lambda$.

Set equal: $(1/n)\sum X_i = 1/\hat{\lambda} \rightarrow \hat{\lambda}_{\text{MoM}} = 1/\bar{X}$

Example: Gaussian Distribution

Data from $N(\mu, \sigma^2)$:

- ▶ 1st moment: $\hat{\mu} = \bar{X}$
- ▶ 2nd central moment: $\hat{\sigma}^2 = (1/n)\sum (X_i - \bar{X})^2$ (note: biased, uses n not n-1)

■ Advantages/Disadvantages

- ✓ Simple, always applicable, closed-form solutions often available
- ✗ Not always efficient, can produce biased estimates, may give invalid parameter values

13.5 Maximum Likelihood Estimation (MLE)

■ Principle

Choose the parameter value that makes the observed data **most probable**.

■ Likelihood Function

Given data $x_1, \dots, x_n$ from $f(x; \theta)$:

$$L(\theta) = \prod_{i=1}^n f(x_i; \theta)$$

■ Log-Likelihood (usually easier)

$$\ell(\theta) = \ln L(\theta) = \sum_{i=1}^n \ln f(x_i; \theta)$$

■ MLE Procedure

1. Write the log-likelihood $\ell(\theta)$
2. Differentiate: $d\ell/d\theta = 0$
3. Solve for $\hat{\theta}_{MLE}$
4. Verify it's a maximum ($d^2\ell/d\theta^2 < 0$)

■ Properties of MLE

1. **Consistent** (converges to true value)
2. **Asymptotically unbiased** (bias $\rightarrow 0$ as $n \rightarrow \infty$)
3. **Asymptotically efficient** (achieves CRLB for large n)
4. **Invariant:** If $\hat{\theta}$ is MLE of θ , then $g(\hat{\theta})$ is MLE of $g(\theta)$

Example: MLE for Gaussian Mean

Data $X_1, \dots, X_n \sim N(\mu, \sigma^2)$ with σ^2 known.

$$\ell(\mu) = -(n/2)\ln(2\pi\sigma^2) - (1/(2\sigma^2))\sum (x_i - \mu)^2$$

$$d\ell/d\mu = (1/\sigma^2)\sum (x_i - \mu) = 0 \rightarrow \hat{\mu}_{MLE} = \bar{X}$$

Example: MLE for Exponential Rate

Data $X_1, \dots, X_n \sim \text{Exp}(\lambda)$.

$$\ell(\lambda) = n \cdot \ln(\lambda) - \lambda \cdot \sum x_i$$

$$d\ell/d\lambda = n/\lambda - \sum x_i = 0 \rightarrow \hat{\lambda}_{MLE} = n/\sum x_i = 1/\bar{X}$$

Example: MLE for Gaussian Variance

Data $X_1, \dots, X_n \sim N(\mu, \sigma^2)$, μ known.

$$\ell(\sigma^2) = -(n/2)\ln(2\pi\sigma^2) - (1/(2\sigma^2))\sum(x_i - \mu)^2$$

$$d\ell/d(\sigma^2) = -n/(2\sigma^2) + (1/(2\sigma^4))\sum(x_i - \mu)^2 = 0$$

$$\hat{\sigma}_{\text{MLE}}^2 = (1/n)\sum(x_i - \mu)^2$$

Note: With μ unknown, $\hat{\sigma}_{\text{MLE}}^2 = (1/n)\sum(x_i - \bar{x})^2$ — biased! (Uses n , not $n-1$)

13.6 Engineering Examples

■ Estimating Noise Variance

Problem: Measure n samples of noise. Estimate noise power σ^2 .

Data: $x_1, \dots, x_n$ (noise voltage samples, assumed zero-mean)

MLE: $\hat{\sigma}^2 = (1/n)\sum x_i^2$ (biased by factor $(n-1)/n$)

Unbiased: $S^2 = (1/(n-1))\sum (x_i - \bar{x})^2$ or if $\mu=0$ known: $(1/n)\sum x_i^2$ is actually unbiased for $E[X^2]=\sigma^2$

■ Estimating Failure Rate

Problem: 20 components tested until failure. Lifetimes: $t_1, \dots, t_{20}$.

Model: $T \sim \text{Exp}(\lambda)$, parameter λ (failure rate).

MLE: $\hat{\lambda} = 20/\sum t_i = 1/\bar{T}$

If $\bar{T} = 500$ hours: $\hat{\lambda} = 1/500 = 0.002$ failures/hour.

■ Estimating Channel Parameter

Problem: Rayleigh fading channel. Observed power samples: $p_1, \dots, p_n$.

Model: Power $P \sim \text{Exp}(1/\Omega)$, where $\Omega = E[P]$ = average power.

MLE: $\hat{\Omega} = \bar{P} = (1/n)\sum p_i$

13.7 Comparing Estimators

■ Bias-Variance Tradeoff

Two estimators for σ^2 :

- ▶ $\hat{\theta}_1 = S^2$ (unbiased, variance = $2\sigma^4/(n-1)$)
- ▶ $\hat{\theta}_2 = (1/n)\Sigma(X_i - \bar{X})^2$ (biased by $-\sigma^2/n$, variance = $2\sigma^4(n-1)/n^2$)

$$\text{MSE}(\hat{\theta}_1) = 2\sigma^4/(n-1)$$

$$\text{MSE}(\hat{\theta}_2) = 2\sigma^4(n-1)/n^2 + \sigma^4/n^2 = \sigma^4(2n-1)/n^2$$

For any $n \geq 2$: $\text{MSE}(\hat{\theta}_2) < \text{MSE}(\hat{\theta}_1)$! The biased MLE actually has smaller total error.

13.8 MATLAB Examples

Example 1: MLE for Exponential Distribution

MATLAB

```
% MLE for Exponential: Estimate failure rate
lambda_true = 0.005; % True failure rate
n = 30;

% Simulate data
data = exprnd(1/lambda_true, 1, n);

% MLE
lambda_hat = 1 / mean(data);
fprintf('True  $\lambda$  = %.4f, MLE  $\hat{\lambda}$  = %.4f\n', lambda_true, lambda_hat);

% Repeated experiments to see variability
N_exp = 10000;
lambda_estimates = zeros(1, N_exp);
for i = 1:N_exp
    d = exprnd(1/lambda_true, 1, n);
    lambda_estimates(i) = 1/mean(d);
end

fprintf('E[ $\hat{\lambda}$ ] = %.5f (biased: true is %.5f)\n', mean(lambda_estimates), lambda_true);
fprintf('Std( $\hat{\lambda}$ ) = %.5f\n', std(lambda_estimates));

figure;
histogram(lambda_estimates, 80, 'Normalization', 'pdf');
xlabel(' $\hat{\lambda}$ '); ylabel('PDF'); title('Sampling Distribution of MLE  $\hat{\lambda}$ ');
```

Example 2: MLE vs Method of Moments

MATLAB

```

%% Compare MLE and MoM for Uniform(0, θ)
% MLE:  $\hat{\theta} = \max(X_1, \dots, X_n)$ 
% MoM:  $\hat{\theta} = 2\bar{X}$ 
theta_true = 10; n = 20;
N_exp = 50000;

MLE_est = zeros(1, N_exp);
MoM_est = zeros(1, N_exp);

for i = 1:N_exp
    data = theta_true * rand(1, n);
    MLE_est(i) = max(data);
    MoM_est(i) = 2 * mean(data);
end

fprintf('Method      | E[ $\hat{\theta}$ ] | Bias | Var | MSE\n');
fprintf('MLE (max)    | %.3f | %.3f | %.3f | %.3f\n', ...
    mean(MLE_est), mean(MLE_est)-theta_true, var(MLE_est), ...
    var(MLE_est)+(mean(MLE_est)-theta_true)^2);
fprintf('MoM (2*mean) | %.3f | %.3f | %.3f | %.3f\n', ...
    mean(MoM_est), mean(MoM_est)-theta_true, var(MoM_est), ...
    var(MoM_est)+(mean(MoM_est)-theta_true)^2);

```

Example 3: MLE for Gaussian Parameters

MATLAB

```

%% MLE for Gaussian: Estimate both  $\mu$  and  $\sigma^2$ 
mu_true = 5; sigma2_true = 4; n = 50;
N_exp = 10000;

mu_hat = zeros(1, N_exp);
sigma2_MLE = zeros(1, N_exp);
sigma2_unbiased = zeros(1, N_exp);

for i = 1:N_exp
    data = mu_true + sqrt(sigma2_true)*randn(1, n);
    mu_hat(i) = mean(data);
    sigma2_MLE(i) = mean((data - mean(data)).^2); % MLE: divides by n
    sigma2_unbiased(i) = var(data); % Unbiased: divides by n-1
end

fprintf('μ: E=%.3f (true=%.1f), Var=%.4f\n', mean(mu_hat), mu_true, var(mu_hat));
fprintf('σ²_MLE: E=%.3f (true=%.1f, biased!)\n', mean(sigma2_MLE), sigma2_true);
fprintf('σ²_unbiased: E=%.3f (true=%.1f)\n', mean(sigma2_unbiased), sigma2_true);

```

13.9 Practice Problems

Problem 1

Data from $N(\mu, 9)$: $\bar{x} = 4.2$, $n = 25$.

- (a) Give the MLE of μ . (b) What is $\text{Var}(\hat{\mu})$? (c) Is $\bar{X}$ efficient for μ ?

Solution

- (a) $\hat{\mu}_{\text{MLE}} = \bar{X} = 4.2$
 (b) $\text{Var}(\bar{X}) = \sigma^2/n = 9/25 = 0.36$
 (c) CRLB for $N(\mu, \sigma^2)$: $1/(n \cdot I(\mu)) = \sigma^2/n = 0.36$. $\bar{X}$ achieves CRLB $\rightarrow$ yes, efficient.

Problem 2

Component lifetimes (hours): 120, 350, 200, 480, 90, 560, 310, 175, 420, 280.

- (a) Estimate λ (failure rate) by MLE assuming $\text{Exp}(\lambda)$. (b) Estimate MTTF.

Solution

- (a) $\bar{X} = (120+350+200+480+90+560+310+175+420+280)/10 = 298.5$.
 $\hat{\lambda} = 1/298.5 = 0.00335$ failures/hour.
 (b) $\text{MTTF} = 1/\hat{\lambda} = \bar{X} = 298.5$ hours.

Problem 3

For $\text{Poisson}(\lambda)$ data: $x_1, \dots, x_n$. Derive the MLE of λ .

Solution $\ell(\lambda) = \sum [x_i \ln(\lambda) - \lambda - \ln(x_i!)] = (\sum x_i) \ln(\lambda) - n\lambda - \sum \ln(x_i!)$

$d\ell/d\lambda = (\sum x_i)/\lambda - n = 0 \rightarrow \hat{\lambda}_{\text{MLE}} = (1/n) \sum x_i = \bar{X}$.

The MLE for Poisson rate is the sample mean.

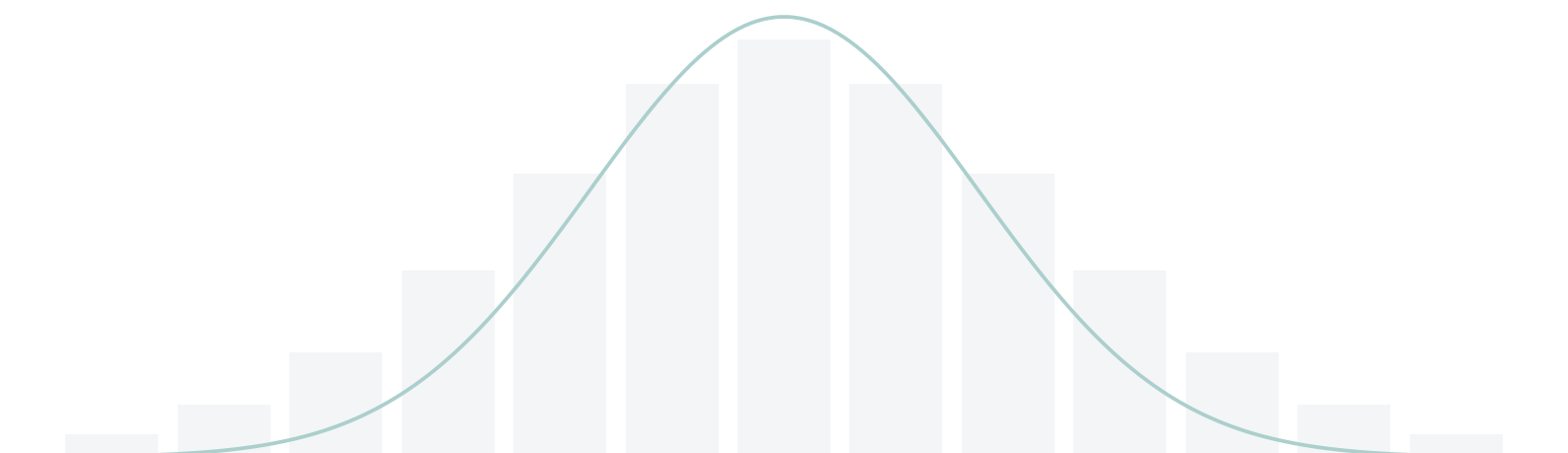
Next Module: Module 14 — Confidence Intervals

Module 14

Confidence Intervals

بازه‌های اطمینان

14



In This Module

14.1	Introduction: Why Point Estimates Aren't Enough	238
14.2	Correct Interpretation of Confidence Intervals	239
14.3	CI for Mean — Known Variance (Z-interval)	240
14.4	CI for Mean — Unknown Variance (t-interval)	241
14.5	CI for Variance	242
14.6	CI for Proportions	243
14.7	Factors Affecting CI Width	244
14.8	Sample Size Determination	245
14.9	MATLAB Demonstrations	246
14.10	Practice Problems	248

Learning Objectives

After completing this module, students will be able to:

1. Correctly interpret confidence intervals (frequentist interpretation)
2. Construct CIs for means (known and unknown variance)
3. Apply the t-distribution for small samples
4. Construct CIs for variance
5. Determine required sample sizes
6. Avoid common misconceptions about CIs

14.1 Introduction: Why Point Estimates Aren't Enough

A point estimate (e.g., $\bar{X} = 3.47$) gives no information about **precision**. We need to quantify uncertainty:

“The true mean is estimated to be 3.47, **give or take** some amount.”

A **confidence interval** provides a range that, with a specified confidence level, captures the true parameter.

14.2 Correct Interpretation of Confidence Intervals

■ ✓ CORRECT (Frequentist) Interpretation

“If we repeat this experiment many times, each time computing a 95% CI, then approximately 95% of those intervals will contain the true parameter μ .”

The **procedure** has a 95% success rate. Each specific interval either contains μ or it doesn't.

■ ✗ WRONG Interpretation

“There is a 95% probability that μ is inside this particular interval [2.1, 4.8].”

Why this is wrong: μ is a fixed (non-random) number. It's either in [2.1, 4.8] or not — there is no probability about it. The **interval** is random (varies from sample to sample), not the parameter.

■ Analogy

Think of CIs like a fishing net:

- ▶ Before you cast: 95% chance your net will catch the fish (the procedure works 95% of the time)
- ▶ After you cast: the fish is either caught or not — no probability anymore
- ▶ The fish (μ) doesn't move; your net (CI) does

14.3 CI for Mean — Known Variance (Z-interval)

■ Setup

Data: $X_1, \dots, X_n$ i.i.d. from $N(\mu, \sigma^2)$ with σ^2 **known**.

■ Derivation

$\bar{X} \sim N(\mu, \sigma^2/n)$, so $Z = (\bar{X} - \mu)/(\sigma/\sqrt{n}) \sim N(0,1)$

$$P(-z_{\alpha/2} \leq Z \leq z_{\alpha/2}) = 1 - \alpha$$

Solving for μ :

$$P\left(\bar{X} - z_{\alpha/2} \frac{\sigma}{\sqrt{n}} \leq \mu \leq \bar{X} + z_{\alpha/2} \frac{\sigma}{\sqrt{n}}\right) = 1 - \alpha$$

■ Formula

$$\text{CI: } \bar{X} \pm z_{\alpha/2} \cdot \frac{\sigma}{\sqrt{n}}$$

■ Common z-values

Confidence Level	α	$z_{\alpha/2}$
90%	0.10	1.645
95%	0.05	1.960
99%	0.01	2.576

Example

Noise measurements: $n = 36$, $\bar{X} = 0.52\text{V}$, $\sigma = 0.12\text{V}$ (known from equipment spec).

95% CI: $0.52 \pm 1.96 \times 0.12/\sqrt{36} = 0.52 \pm 0.0392 = [0.481, 0.559] \text{ V}$

14.4 CI for Mean — Unknown Variance (t-interval)

■ The Problem

Usually σ is unknown. We estimate it with S . But $\bar{X}/(S/\sqrt{n})$ does NOT follow $N(0,1)$.

■ Student's t-Distribution

$$T = \frac{\bar{X} - \mu}{S/\sqrt{n}} \sim t(n-1)$$

The t-distribution:

- ▶ Symmetric about 0
- ▶ Heavier tails than $N(0,1)$ (more uncertainty because S is estimated)
- ▶ Degrees of freedom: $\nu = n - 1$
- ▶ As $\nu \rightarrow \infty$: $t(\nu) \rightarrow N(0,1)$
- ▶ For $\nu \geq 30$: $t \approx z$ (practically)

■ Formula

$$\text{CI: } \bar{X} \pm t_{\alpha/2, n-1} \cdot \frac{S}{\sqrt{n}}$$

Example

Component delay: $n = 15$, $\bar{X} = 2.34$ ns, $S = 0.45$ ns.

95% CI: $t_{\{0.025, 14\}} = 2.145$

CI: $2.34 \pm 2.145 \times 0.45/\sqrt{15} = 2.34 \pm 0.249 = [2.09, 2.59]$ ns

(Wider than z-interval because of additional uncertainty in S .)

14.5 CI for Variance

■ Setup

Data from $N(\mu, \sigma^2)$. Want CI for σ^2 .

■ Key Distribution

$$(n-1)S^2/\sigma^2 \sim \chi^2(n-1)$$

■ Formula

$$\left[\frac{(n-1)S^2}{\chi_{\alpha/2, n-1}^2}, \frac{(n-1)S^2}{\chi_{1-\alpha/2, n-1}^2} \right]$$

Example

Noise power: $n = 20$, $S^2 = 0.045 \text{ V}^2$.

95% CI for σ^2 : $\chi^2_{\{0.025, 19\}} = 32.852$, $\chi^2_{\{0.975, 19\}} = 8.907$

CI: $[(19)(0.045)/32.852, (19)(0.045)/8.907] = [0.026, 0.096] \text{ V}^2$

95% CI for σ : $[\sqrt{0.026}, \sqrt{0.096}] = [0.161, 0.310] \text{ V}$

14.6 CI for Proportions

■ Setup

n trials, $\hat{x} = k/n$ successes. Estimate $p = \text{true proportion}$.

■ Large-Sample Formula (Wald Interval)

$$\hat{p} \pm z_{\alpha/2} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}$$

Example: Bit Error Rate

Transmitted 100,000 bits, observed 53 errors. $\hat{p} = 53/100000 = 5.3 \times 10^{-4}$.

95% CI: $5.3 \times 10^{-4} \pm 1.96 \sqrt{(5.3 \times 10^{-4} \times 0.9995 / 100000)} = 5.3 \times 10^{-4} \pm 1.43 \times 10^{-4}$

CI: $[3.9 \times 10^{-4}, 6.7 \times 10^{-4}]$

14.7 Factors Affecting CI Width

Factor	Effect on Width	Explanation
↑ Sample size n	Narrower	More data → more precision
↑ Confidence level	Wider	More confidence → wider net
↑ Variability σ	Wider	More noise → harder to pin down μ

■ The Tradeoff

- ▶ Want narrow CI (precise) → need more data or accept lower confidence
- ▶ Want high confidence → accept wider CI or collect more data
- ▶ Want both narrow AND high confidence → need LOTS of data

14.8 Sample Size Determination

■ Question: How many measurements do I need?

For CI width $\leq 2E$ (margin of error E):

$$n \geq \left(\frac{z_{\alpha/2} \cdot \sigma}{E} \right)^2$$

Example

Want to estimate mean delay within $\pm 0.5\text{ms}$ with 95% confidence. Expect $\sigma \approx 3\text{ms}$.

$n \geq (1.96 \times 3 / 0.5)^2 = (11.76)^2 = 138.3 \rightarrow \text{need } n = 139 \text{ measurements}$

14.9 MATLAB Demonstrations

Example 1: CI Coverage Demonstration

MATLAB

```

%% Demonstrate: ~95% of 95% CIs contain the true mean
mu_true = 5; sigma = 2; n = 25;
N_experiments = 1000;
alpha = 0.05;
z_crit = norminv(1-alpha/2);

covers = 0;
figure; hold on;
for i = 1:N_experiments
    data = mu_true + sigma*randn(1, n);
    x_bar = mean(data);
    margin = z_crit * sigma / sqrt(n);
    ci_low = x_bar - margin;
    ci_high = x_bar + margin;

    if ci_low <= mu_true && mu_true <= ci_high
        covers = covers + 1;
        color = 'b';
    else
        color = 'r';
    end

    if i <= 100 % Plot first 100
        plot([ci_low ci_high], [i i], color, 'LineWidth', 0.5);
    end
end
plot([mu_true mu_true], [0 101], 'k--', 'LineWidth', 2);
xlabel('μ'); ylabel('Experiment #');
title(sprintf('95% CI Coverage: %.1f%% of %d intervals contain μ', ...
    100*covers/N_experiments, N_experiments));
fprintf('Coverage: %.1f%% (theory: 95%)\n', 100*covers/N_experiments);

```

Example 2: t-interval in Practice

```

%% t-Confidence Interval: Unknown Variance
data = [2.3, 2.7, 2.1, 2.8, 2.5, 2.9, 2.4, 2.6, 2.2, 2.7];
n = length(data);
x_bar = mean(data);
s = std(data);
alpha = 0.05;

t_crit = tinv(1-alpha/2, n-1);
margin = t_crit * s / sqrt(n);

fprintf('Sample mean: %.3f\n', x_bar);
fprintf('Sample std: %.3f\n', s);
fprintf('t_crit (df=%d): %.3f\n', n-1, t_crit);
fprintf('95% CI: [%.3f, %.3f]\n', x_bar-margin, x_bar+margin);

```

MATLAB

Example 3: Sample Size Calculation

```

%% How many samples do we need?
sigma = 0.1; % Expected std dev (from pilot study)
E_values = [0.05, 0.02, 0.01, 0.005]; % Desired margins
confidence = 0.95;
z = norminv(1 - (1-confidence)/2);

fprintf('Desired Margin | Required n\n');
fprintf('-----+-----\n');
for E = E_values
    n_req = ceil((z*sigma/E)^2);
    fprintf(' ±%.3f | %d\n', E, n_req);
end

```

MATLAB

14.10 Practice Problems

Problem 1

A sensor measures voltage. From 50 measurements: $\bar{X} = 3.32\text{V}$, $S = 0.15\text{V}$.

- (a) Construct 95% CI for μ . (b) Construct 99% CI for μ . (c) Which is wider and why?

Solution

(a) $t_{0.025,49} \approx 2.010$. CI: $3.32 \pm 2.010(0.15/\sqrt{50}) = 3.32 \pm 0.043 = [3.277, 3.363]$

(b) $t_{0.005,49} \approx 2.680$. CI: $3.32 \pm 2.680(0.15/\sqrt{50}) = 3.32 \pm 0.057 = [3.263, 3.377]$

(c) 99% CI is wider — higher confidence requires a wider interval.

Problem 2

From 25 noise power measurements: $S^2 = 0.048 \text{ V}^2$. Construct 95% CI for σ^2 .

Solution $\chi^2_{0.025,24} = 39.364$, $\chi^2_{0.975,24} = 12.401$

CI: $[24(0.048)/39.364, 24(0.048)/12.401] = [0.029, 0.093] \text{ V}^2$

Problem 3

How many BER measurements needed to estimate BER within $\pm 20\%$ of its true value with 95% confidence, if expected BER $\approx 10^{-3}$?

Solution For proportion: $n \geq z^2 p(1-p)/E^2$ where $E = 0.2p = 2 \times 10^{-4}$.

$n \geq (1.96)^2(10^{-3})(0.999)/(2 \times 10^{-4})^2 = 3.84 \times 10^{-3} / (4 \times 10^{-8}) \approx 96,000$ bits.

Need to transmit approximately 96,000 bits (or observe ~ 96 errors).

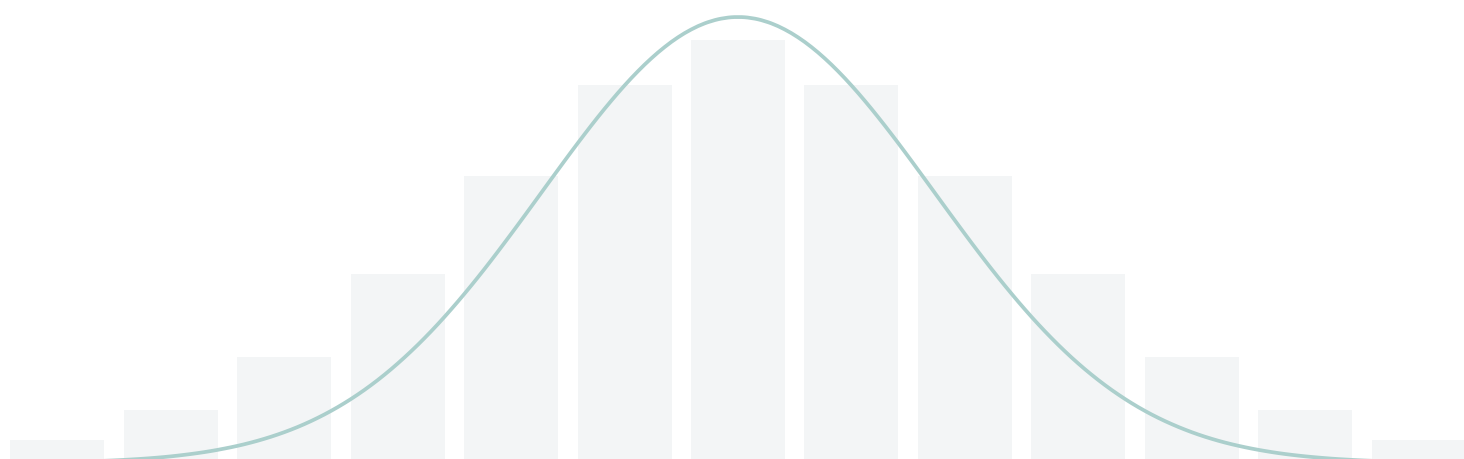
Next Module: Module 15 — Hypothesis Testing

Module 15

Hypothesis Testing

آزمون فرض

15



In This Module

15.1	Introduction: Making Decisions from Data	252
15.2	Framework	253
15.3	Types of Errors	254
15.4	Significance Level α	255
15.5	Power of a Test	256
15.6	The p-value	257
15.7	One-Sided vs. Two-Sided Tests	258
15.8	Tests for Means	259
15.9	Test for Variance (Chi-Square Test)	260
15.10	Step-by-Step Hypothesis Testing Procedure	261
15.11	Engineering Examples	262
15.12	MATLAB Examples	263
15.13	Practice Problems	265

Learning Objectives

After completing this module, students will be able to:

1. Formulate null and alternative hypotheses for engineering problems
2. Identify Type I and Type II errors and their engineering consequences
3. Compute test statistics and p-values
4. Perform Z-tests and t-tests for means
5. Correctly interpret p-values (and avoid common misconceptions)
6. Choose between one-sided and two-sided tests

15.1 Introduction: Making Decisions from Data

Engineering question: “Is this sensor biased?” “Did the process change?” “Does the new algorithm perform better?”

We need a **systematic framework** to answer such questions from data, while controlling the probability of making errors.

15.2 Framework

■ Null Hypothesis H_0

The **default assumption** — typically “nothing changed” or “no effect”:

- ▶ $H_0: \mu = \mu_0$ (sensor is not biased)
- ▶ $H_0: \mu_1 = \mu_2$ (two methods perform equally)
- ▶ $H_0: \sigma^2 = \sigma_0^2$ (variance hasn't changed)

■ Alternative Hypothesis H_1

What we want to **detect** or prove:

- ▶ $H_1: \mu \neq \mu_0$ (sensor IS biased — two-sided)
- ▶ $H_1: \mu > \mu_0$ (performance improved — one-sided)
- ▶ $H_1: \sigma^2 > \sigma_0^2$ (variance increased)

■ Decision

Based on data, either:

- ▶ **Reject H_0** → evidence supports H_1
- ▶ **Fail to reject H_0** → insufficient evidence against H_0

▲ “Fail to reject H_0 ” $\neq$ “ H_0 is true”. Absence of evidence $\neq$ evidence of absence.

15.3 Types of Errors

	H_0 True (reality)	H_1 True (reality)
Reject H_0 (decision)	Type I Error (α)	Correct! (Power = $1-\beta$)
Fail to reject H_0 (decision)	Correct!	Type II Error (β)

■ Type I Error (False Alarm) — probability α

Rejecting H_0 when it's actually true. "Crying wolf."

Engineering analogy: Declaring a process has changed when it hasn't → unnecessary recalibration.

■ Type II Error (Missed Detection) — probability β

Failing to reject H_0 when H_1 is actually true. "Missing the signal."

Engineering analogy: Failing to detect a sensor bias → biased measurements continue.

■ Engineering Analogy: Radar Detection

Statistical Term	Radar Equivalent
H_0 : no target	No aircraft present
H_1 : target present	Aircraft present
Type I error (α)	False alarm
Type II error (β)	Missed detection
Power ($1-\beta$)	Probability of detection

■ The Tradeoff

Reducing α (fewer false alarms) → increases β (more missed detections), and vice versa.

Fix α (significance level), then maximize power = $1 - \beta$.

15.4 Significance Level α

The **maximum acceptable Type I error rate**, chosen BEFORE looking at data.

Common choices: $\alpha = 0.05, 0.01, 0.10$

Choosing α : Consider the cost of a false alarm:

- ▶ $\alpha = 0.01$: Very conservative (e.g., medical device safety)
- ▶ $\alpha = 0.05$: Standard in most engineering/science
- ▶ $\alpha = 0.10$: More tolerant (exploratory analysis)

15.5 Power of a Test

$$\text{Power} = 1 - \beta = P(\text{reject } H_0 | H_1 \text{ is true})$$

Power depends on:

1. **Effect size:** Larger true difference $\rightarrow$ easier to detect $\rightarrow$ more power
2. **Sample size n :** More data $\rightarrow$ more power
3. **Significance level α :** Larger $\alpha \rightarrow$ more power (but more false alarms)
4. **Variability σ :** Less noise $\rightarrow$ more power

15.6 The p-value

Definition

The **p-value** is the probability of obtaining a test statistic **as extreme or more extreme** than the observed value, **assuming H_0 is true**.

$$p\text{-value} = P(\text{data this extreme or more} | H_0 \text{ true})$$

Decision Rule

- ▶ If $p\text{-value} < \alpha \rightarrow \text{Reject } H_0$
- ▶ If $p\text{-value} \geq \alpha \rightarrow \text{Fail to reject } H_0$

✓ CORRECT Interpretation

“The p-value is the probability of seeing data this extreme (or more) if H_0 were true. A small p-value means the data is unlikely under H_0 .”

✗ WRONG Interpretation

“The p-value is the probability that H_0 is true.”

Why wrong: H_0 is either true or false (fixed). The p-value says nothing about $P(H_0 \text{ true})$. It measures how surprising the data is under H_0 .

Calibration

p-value	Evidence against H_0
> 0.10	Weak or none
$0.05 - 0.10$	Marginal
$0.01 - 0.05$	Moderate
$0.001 - 0.01$	Strong
< 0.001	Very strong

15.7 One-Sided vs. Two-Sided Tests

■ Two-Sided ($H_1: \mu \neq \mu_0$)

Used when deviation in **either** direction is important.

- ▶ Reject if $|T| > t_{\{\alpha/2, n-1\}}$
- ▶ p-value = $2 \cdot P(T > |t_{\text{obs}}|)$

■ One-Sided ($H_1: \mu > \mu_0$ or $H_1: \mu < \mu_0$)

Used when only one direction of deviation matters.

- ▶ $H_1: \mu > \mu_0$: Reject if $T > t_{\{\alpha, n-1\}}$
- ▶ $H_1: \mu < \mu_0$: Reject if $T < -t_{\{\alpha, n-1\}}$

■ When to Use Which

Scenario	Test Type
Is the sensor biased (either direction)?	Two-sided
Does the new algorithm IMPROVE BER?	One-sided ($H_1: \text{BER} < \text{BER}_{\text{old}}$)
Has noise power INCREASED?	One-sided ($H_1: \sigma^2 > \sigma_0^2$)
Did the process mean CHANGE?	Two-sided

15.8 Tests for Means

■ One-Sample Z-Test (σ known)

Test: $H_0: \mu = \mu_0$

Test statistic: $Z = (\bar{X} - \mu_0)/(\sigma/\sqrt{n})$

Under H_0 : $Z \sim N(0,1)$

■ One-Sample t-Test (σ unknown)

Test: $H_0: \mu = \mu_0$

Test statistic: $T = (\bar{X} - \mu_0)/(S/\sqrt{n})$

Under H_0 : $T \sim t(n-1)$

■ Two-Sample t-Test (comparing two means)

Test: $H_0: \mu_1 = \mu_2$

Test statistic: $T = (\bar{X}_1 - \bar{X}_2) / \sqrt{(S_1^2/n_1 + S_2^2/n_2)}$

Approximate df by Welch-Satterthwaite formula.

15.9 Test for Variance (Chi-Square Test)

■ One-Sample Test

$$H_0: \sigma^2 = \sigma_0^2$$

$$\text{Test statistic: } \chi^2 = (n-1)S^2/\sigma_0^2$$

$$\text{Under } H_0: \chi^2 \sim \chi^2(n-1)$$

■ Decision

- ▶ $H_1: \sigma^2 > \sigma_0^2$: Reject if $\chi^2 > \chi^2_{\{\alpha, n-1\}}$
- ▶ $H_1: \sigma^2 \neq \sigma_0^2$: Reject if $\chi^2 < \chi^2_{\{1-\alpha/2, n-1\}}$ or $\chi^2 > \chi^2_{\{\alpha/2, n-1\}}$

15.10 Step-by-Step Hypothesis Testing Procedure

1. **State hypotheses:** H_0 and H_1
2. **Choose significance level:** α (e.g., 0.05)
3. **Select test statistic:** Z, t, or χ^2 depending on scenario
4. **Determine critical value(s)** or compute p-value
5. **Compute test statistic** from data
6. **Make decision:** Reject H_0 if test statistic in critical region (or $p < \alpha$)
7. **State conclusion** in engineering terms

15.11 Engineering Examples

Example 1: Is a Sensor Biased?

A sensor should read 0V with no input. 20 measurements give $\bar{X} = 0.012\text{V}$, $S = 0.03\text{V}$.

$H_0: \mu = 0$ (no bias), $H_1: \mu \neq 0$ (biased), $\alpha = 0.05$

$T = (0.012 - 0)/(0.03/\sqrt{20}) = 0.012/0.00671 = 1.789$

$t_{\{0.025, 19\}} = 2.093$. Since $|T| = 1.789 < 2.093 \rightarrow$ **Fail to reject H_0** .

p-value = $2 \cdot P(t(19) > 1.789) \approx 0.090$. Not significant at 5% level.

Conclusion: Insufficient evidence to conclude the sensor is biased.

Example 2: Did Manufacturing Process Change?

Before: $\mu_0 = 100\Omega$ (established). After modification: $n = 30$, $\bar{X} = 101.2\Omega$, $S = 3.5\Omega$.

$H_0: \mu = 100$, $H_1: \mu \neq 100$, $\alpha = 0.05$

$T = (101.2 - 100)/(3.5/\sqrt{30}) = 1.2/0.639 = 1.878$

$t_{\{0.025, 29\}} = 2.045$. $|T| = 1.878 < 2.045 \rightarrow$ **Fail to reject H_0** .

Example 3: Does New Algorithm Improve BER?

Old algorithm: $\text{BER}_0 = 10^{-3}$. New algorithm tested: $n = 50000$ bits, 38 errors.

$\text{BER}_{\text{new}} = 38/50000 = 7.6 \times 10^{-4}$

$H_0: p = 0.001$, $H_1: p < 0.001$ (improvement), $\alpha = 0.05$

$Z = (\hat{p} - p_0)/\sqrt{(p_0(1-p_0)/n)} = (0.00076 - 0.001)/\sqrt{(0.001 \times 0.999/50000)} = -0.00024/0.000141 = -1.70$

p-value = $P(Z < -1.70) = 0.0446 < 0.05 \rightarrow$ **Reject H_0** .

Conclusion: Evidence supports that the new algorithm has lower BER.

Example 4: Has Noise Power Increased?

Historical: $\sigma_0^2 = 0.04 \text{ V}^2$. New measurements: $n = 25$, $S^2 = 0.058 \text{ V}^2$.

$H_0: \sigma^2 = 0.04$, $H_1: \sigma^2 > 0.04$, $\alpha = 0.05$

$\chi^2 = (24)(0.058)/0.04 = 34.8$

$\chi^2_{\{0.05, 24\}} = 36.415$. Since $34.8 < 36.415 \rightarrow$ **Fail to reject H_0** .

15.12 MATLAB Examples

Example 1: One-Sample t-test

```
%% Is the sensor biased? One-sample t-test
data = [0.012, -0.005, 0.031, 0.008, -0.012, 0.022, 0.015, ...
        0.003, 0.028, -0.001, 0.018, 0.009, 0.025, -0.008, ...
        0.014, 0.007, 0.020, 0.011, -0.003, 0.019];

mu_0 = 0; % Hypothesized mean (no bias)
[h, p_value, ci, stats] = ttest(data, mu_0);

fprintf('Test result: H = %d (1=reject H0)\n', h);
fprintf('p-value = %.4f\n', p_value);
fprintf('t-statistic = %.3f, df = %d\n', stats.tstat, stats.df);
fprintf('95% CI for  $\mu$ : [%.4f, %.4f]\n', ci);
```

MATLAB

Example 2: Two-Sample t-test

```
%% Compare two algorithms: is there a significant difference?
BER_algo1 = [0.0012, 0.0008, 0.0015, 0.0010, 0.0013, ...
             0.0011, 0.0009, 0.0014, 0.0012, 0.0010];
BER_algo2 = [0.0007, 0.0009, 0.0006, 0.0008, 0.0005, ...
             0.0008, 0.0007, 0.0006, 0.0009, 0.0007];

[h, p_value, ci, stats] = ttest2(BER_algo1, BER_algo2);
fprintf('Two-sample t-test:\n');
fprintf(' H = %d (1=reject H0: means are equal)\n', h);
fprintf(' p-value = %.4e\n', p_value);
fprintf(' t-stat = %.3f, df = %.1f\n', stats.tstat, stats.df);
fprintf(' Mean diff: %.4f\n', mean(BER_algo1) - mean(BER_algo2));
```

MATLAB

Example 3: Power Analysis

MATLAB

```

%% Power: How likely are we to detect a true difference?
mu_0 = 0;           % Null hypothesis mean
mu_true = 0.015;    % True mean (sensor has bias of 15mV)
sigma = 0.03;       % Known std dev
alpha = 0.05;

n_values = 5:5:100;
power = zeros(size(n_values));

for i = 1:length(n_values)
    n = n_values(i);
    % Critical value for two-sided test
    z_crit = norminv(1-alpha/2);
    % Power = P(reject | mu_true)
    % Reject when |Z| > z_crit, where Z ~ N((mu_true-mu_0)/(sigma/sqrt(n)), 1)
    ncp = (mu_true - mu_0) / (sigma/sqrt(n)); % Non-centrality parameter
    power(i) = 1 - normcdf(z_crit - ncp) + normcdf(-z_crit - ncp);
end

figure;
plot(n_values, power, 'b-o', 'LineWidth', 2);
xlabel('Sample Size n'); ylabel('Power (1 - \beta)');
title(sprintf('Power vs Sample Size (true bias = %.0f mV)', mu_true*1000));
grid on;
yline(0.8, 'r--', 'Desired Power = 0.8');
fprintf('Need n \approx %d for 80%% power\n', n_values(find(power >= 0.8, 1)));

```

15.13 Practice Problems**Problem 1**

A manufacturing spec requires mean resistance = 100Ω . A sample of 16 resistors gives $\bar{X} = 101.5\Omega$, $S = 3\Omega$. At $\alpha = 0.05$, is there evidence the process has drifted?

Solution $H_0: \mu=100$, $H_1: \mu \neq 100$. $T = (101.5-100)/(3/4) = 2.0$. $t_{0.025,15} = 2.131$. $|T|=2.0 < 2.131 \rightarrow$ Fail to reject. $p\text{-value} \approx 0.064 > 0.05$.

Problem 2

Average packet delay was 5ms. After network upgrade, 40 measurements give $\bar{X} = 4.6\text{ms}$, $S = 1.2\text{ms}$. Test if delay decreased at $\alpha = 0.01$.

Solution $H_0: \mu=5$, $H_1: \mu < 5$. $T = (4.6-5)/(1.2/\sqrt{40}) = -0.4/0.190 = -2.11$. $t_{0.01,39} \approx -2.426$. $T = -2.11 > -2.426 \rightarrow$ Fail to reject at 1% level. (Would reject at 5% since $t_{0.05,39} \approx -1.685$.)

Problem 3

Explain why “ $p = 0.04$ means there is only a 4% chance H_0 is true” is WRONG.

Solution The p-value is $P(\text{data this extreme} \mid H_0 \text{ true})$, NOT $P(H_0 \text{ true} \mid \text{data})$. It conditions on H_0 being true and asks about data extremity. To get $P(H_0 \text{ true} \mid \text{data})$ would require Bayesian analysis with a prior. The p-value is a property of the data-generating process under H_0 , not a posterior probability about H_0 .

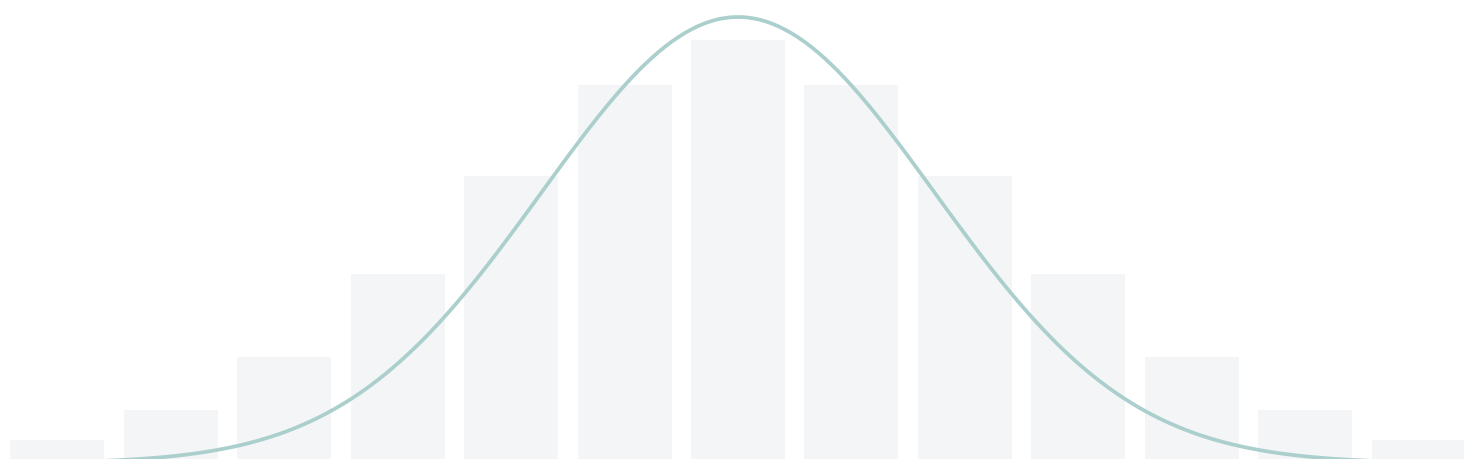
Next Module: Module 16 — Engineering Statistical Applications

Module 16

Engineering Statistical Applications

کاربردهای آماری مهندسی

16



In This Module

16.1	Communications	269
16.2	Signal Processing	271
16.3	Electronics	273
16.4	Power Systems	274
16.5	Control Systems	275
16.6	Machine Learning / AI Connections	276
16.7	Comprehensive MATLAB Project: Communication System Analysis	277
16.8	Key Integration Points	279

Learning Objectives

This capstone module integrates ALL previous modules into real Electrical Engineering applications. Students will be able to:

1. Apply probabilistic modeling to communication systems
2. Analyze signal processing problems statistically
3. Model electronic component variations
4. Assess power system reliability
5. Handle measurement uncertainty in control systems
6. Connect probability/statistics to machine learning foundations

16.1 Communications

■ 16.1.1 BER Estimation and Confidence Intervals

Problem: Estimate the Bit Error Rate of a communication link.

Model: Each bit is a Bernoulli trial with error probability p (the BER).

- ▶ n bits transmitted, k errors observed
- ▶ $\hat{p} = k/n$ (MLE of BER)
- ▶ 95% CI: $\hat{p} \pm 1.96\sqrt{(\hat{p}(1-\hat{p}))/n}$

MATLAB: BER Simulation and CI

```
% BER Estimation with Confidence Interval
EbN0_dB = 8; EbN0 = 10^(EbN0_dB/10);
N_bits = 1e6;

% BPSK transmission
bits = 2*randi([0,1], 1, N_bits) - 1;
noise = (1/sqrt(2*EbN0)) * randn(1, N_bits);
received = bits + noise;
decisions = sign(received);
errors = sum(decisions ~= bits);

BER_hat = errors / N_bits;
BER_theory = qfunc(sqrt(2*EbN0));

% 95% Confidence Interval
margin = 1.96 * sqrt(BER_hat*(1-BER_hat)/N_bits);
fprintf('BER estimate: %.4e\n', BER_hat);
fprintf('BER theory:   %.4e\n', BER_theory);
fprintf('95% CI: [%.4e, %.4e]\n', BER_hat-margin, BER_hat+margin);
```

MATLAB

■ 16.1.2 AWGN Noise Modeling

Additive White Gaussian Noise is the fundamental noise model:

- ▶ **Distribution:** $N(0, N_0/2)$ per dimension
- ▶ **Why Gaussian?** CLT — thermal noise is sum of many electron contributions
- ▶ **Characterization:** Fully described by power spectral density N_0 or variance $\sigma^2 = N_0 B$ (B = bandwidth)

■ 16.1.3 Rayleigh Fading Channel

Model: Channel coefficient $h = h_I + j \cdot h_Q$ where $h_I, h_Q \sim N(0, \sigma^2)$ independent.

- ▶ Envelope $|h| \sim \text{Rayleigh}(\sigma)$
- ▶ Power $|h|^2 \sim \text{Exponential}(1/(2\sigma^2))$
- ▶ **Outage probability:** $P(|h|^2 < \gamma_{th}) = 1 - \exp(-\gamma_{th}/(2\sigma^2))$

MATLAB

```

%% Rayleigh Fading: Outage Probability
sigma = 1;
SNR_avg_dB = 10; % Average SNR in dB
SNR_avg = 10^(SNR_avg_dB/10);
SNR_threshold_dB = 5; % Minimum required SNR
gamma_th = 10^(SNR_threshold_dB/10);

% Analytical outage probability
P_outage = 1 - exp(-gamma_th / SNR_avg);
fprintf('Outage Probability (analytical): %.4f\n', P_outage);

% Simulation
N = 100000;
h = (randn(1,N) + 1j*randn(1,N)) / sqrt(2); % Rayleigh fading
SNR_inst = SNR_avg * abs(h).^2;
P_outage_sim = mean(SNR_inst < gamma_th);
fprintf('Outage Probability (simulated): %.4f\n', P_outage_sim);

```

■ 16.1.4 Rician Fading

When a line-of-sight (LOS) component exists:

- ▶ $h = h_{\text{LOS}} + h_{\text{scatter}}$
- ▶ $|h| \sim \text{Rician}(\nu, \sigma)$ where $K = \nu^2/(2\sigma^2)$ is the K-factor

K-factor: $K = 0$ (Rayleigh) $\rightarrow K = \infty$ (no fading, AWGN)

■ 16.1.5 SNR Statistics

Instantaneous SNR: $\gamma = |h|^2 \times (E_s/N_0)$

For Rayleigh: $\gamma \sim \text{Exponential}(\bar{\gamma})$ where $\bar{\gamma}$ = average SNR

Average BER over fading:

$$\bar{P}_e = \int_0^{\infty} P_e(\gamma) \cdot f_{\gamma}(\gamma) d\gamma$$

For BPSK over Rayleigh: $\bar{P}_e = (1/2)(1 - \sqrt{\bar{\gamma}/(1+\bar{\gamma})})$

16.2 Signal Processing

■ 16.2.1 Measurement Noise Characterization

Problem: Characterize noise in measured signal $x(t) = s(t) + n(t)$.

Approach:

1. Collect noise-only samples (signal absent)
2. Estimate noise statistics: $\hat{\mu}_n, \hat{\sigma}_n^2$
3. Test for Gaussianity (histogram, QQ-plot)
4. Compute SNR: $\text{SNR} = P_{\text{signal}} / \hat{\sigma}_n^2$

MATLAB

```
%% Noise Characterization
fs = 1000; T = 10; % 1kHz sample rate, 10 seconds
t = 0:1/fs:T-1/fs;
N_samples = length(t);

% Simulate: known signal + noise
signal = 2*sin(2*pi*5*t); % 5 Hz sine wave
noise = 0.5*randn(1, N_samples);
measured = signal + noise;

% Estimate noise from signal-free segments (or residuals)
% Here: subtract known signal
noise_estimate = measured - signal;

fprintf('Noise Statistics:\n');
fprintf(' Mean: %.4f (should be ~0)\n', mean(noise_estimate));
fprintf(' Std: %.4f (true: 0.5)\n', std(noise_estimate));
fprintf(' SNR: %.1f dB\n', 10*log10(var(signal)/var(noise_estimate)));

% Gaussianity test
figure;
subplot(1,2,1); histogram(noise_estimate, 50, 'Normalization', 'pdf');
hold on; x = linspace(-2,2,200); plot(x, normpdf(x,0,0.5),'r','LineWidth',2);
title('Noise Histogram vs Gaussian');
subplot(1,2,2); qqplot(noise_estimate); title('QQ Plot');
```

■ 16.2.2 Correlation-Based Detection

Problem: Detect a known signal in noise.

Method: Correlate received signal with known template. Under H_0 (noise only), the correlator output has known distribution → set threshold.

MATLAB

```
%% Correlation Detector
N = 100; % Signal length
s = ones(1,N)/sqrt(N); % Known signal template (normalized)
sigma_n = 1;

% H0: noise only; H1: signal + noise
N_trials = 10000;
y_H0 = zeros(1, N_trials); y_H1 = zeros(1, N_trials);
A = 2; % Signal amplitude

for i = 1:N_trials
    n = sigma_n * randn(1, N);
    y_H0(i) = s * n'; % Correlator output under H0
    y_H1(i) = s * (A*s + n)'; % Correlator output under H1
end

% Set threshold for P_FA = 0.01
threshold = sigma_n/sqrt(N) * norminv(0.99);
P_FA = mean(y_H0 > threshold);
P_D = mean(y_H1 > threshold);
fprintf('P_FA = %.4f (target: 0.01)\n', P_FA);
fprintf('P_D = %.4f\n', P_D);
```

16.3 Electronics

■ 16.3.1 Component Tolerance Modeling

Problem: Resistors have nominal value R_0 with manufacturing tolerance.

Model: $R \sim N(R_0, \sigma_R^2)$ where σ_R depends on tolerance class.

For $\pm 5\%$ tolerance (99.7% within bounds): $3\sigma = 0.05R_0 \rightarrow \sigma = 0.0167R_0$

■ 16.3.2 Monte Carlo Circuit Analysis

Problem: A voltage divider uses R_1 and R_2 . What is the distribution of output voltage?

$$V_{\text{out}} = V_{\text{in}} \times R_2 / (R_1 + R_2)$$

With R_1, R_2 random: V_{out} is a random variable!

MATLAB

```
%% Monte Carlo: Voltage Divider with Tolerant Components
V_in = 5;           % Input voltage
R1_nom = 10e3;      R2_nom = 10e3; % 10kΩ each
tol = 0.05;         % 5% tolerance
sigma_R = tol * R1_nom / 3; % 3-sigma = 5%

N = 100000;
R1 = R1_nom + sigma_R*randn(1, N);
R2 = R2_nom + sigma_R*randn(1, N);
V_out = V_in * R2 ./ (R1 + R2);

fprintf('Nominal V_out = %.3f V\n', V_in * R2_nom/(R1_nom+R2_nom));
fprintf('Mean V_out = %.4f V\n', mean(V_out));
fprintf('Std V_out = %.4f V\n', std(V_out));
fprintf('99.7%% range: [%.3f, %.3f] V\n', ...
    mean(V_out)-3*std(V_out), mean(V_out)+3*std(V_out));

% Yield: fraction within ±2% of nominal
V_nom = 2.5;
yield = mean(abs(V_out - V_nom) < 0.02*V_nom);
fprintf('Yield (±2%%): %.2f%%\n', 100*yield);
```

■ 16.3.3 Measurement Uncertainty Propagation

If $Z = f(X, Y)$ and X, Y have small uncertainties:

$$\sigma_Z^2 \approx \left(\frac{\partial f}{\partial X} \right)^2 \sigma_X^2 + \left(\frac{\partial f}{\partial Y} \right)^2 \sigma_Y^2$$

Example: Power $P = V^2/R$. $\sigma_P^2 \approx (2V/R)^2 \sigma_V^2 + (V^2/R^2)^2 \sigma_R^2$

16.4 Power Systems

■ 16.4.1 Load Uncertainty

Problem: Peak load is uncertain. Model: $L \sim N(\mu_L, \sigma_L^2)$

Design question: What capacity C ensures $P(\text{load} > \text{capacity}) < 0.01$?

$C = \mu_L + 2.326\sigma_L$ (for 1% exceedance probability)

■ 16.4.2 System Reliability

Series system: All components must work.

$R_{\text{sys}} = R_1 \times R_2 \times \dots \times R_n$ (for independent components)

Parallel system: At least one must work.

$R_{\text{sys}} = 1 - (1-R_1)(1-R_2)\dots(1-R_n)$

MATLAB

```
%% Reliability Analysis: Series-Parallel System
% System: Two subsystems in series, each with 3 parallel redundant units
R_unit = 0.95; % Individual component reliability
n_parallel = 3;
n_series = 2;

% Each parallel subsystem
R_subsystem = 1 - (1 - R_unit)^n_parallel;
% Series combination
R_system = R_subsystem^n_series;

fprintf('Unit reliability: %.4f\n', R_unit);
fprintf('Subsystem (3 parallel): %.6f\n', R_subsystem);
fprintf('System (2 in series): %.6f\n', R_system);

% Monte Carlo verification
N = 100000;
sys_works = true(1, N);
for s = 1:n_series
    sub_works = false(1, N);
    for p = 1:n_parallel
        unit_works = rand(1, N) < R_unit;
        sub_works = sub_works | unit_works;
    end
    sys_works = sys_works & sub_works;
end
fprintf('System reliability (sim): %.6f\n', mean(sys_works));
```

16.5 Control Systems

■ 16.5.1 Sensor Noise Effects

Problem: A control system uses noisy sensor readings.

Measurement: $y(k) = x(k) + n(k)$ where $n \sim N(0, \sigma_n^2)$

Controller acts on $y(k) \rightarrow$ tracking error includes noise contribution.

Solution approaches:

- ▶ Averaging (low-pass filter): reduces noise by $\sqrt{n}$
- ▶ Kalman filter: optimal fusion of model prediction and measurement

```

%% Noisy Sensor in Control Loop: Effect of Averaging
true_value = 10;    % Constant being measured
sigma_n = 0.5;     % Sensor noise std dev
N_measurements = 1000;

% Raw measurements
measurements = true_value + sigma_n*randn(1, N_measurements);

% Moving average filter (window size M)
M_values = [1, 5, 10, 20, 50];
figure; hold on;
for M = M_values
    filtered = movmean(measurements, M);
    plot(filtered(100:500));
end
yline(true_value, 'k--', 'LineWidth', 2);
legend(arrayfun(@(m) sprintf('M=%d, σ=%.3f', m, sigma_n/sqrt(m)), ...
    M_values, 'UniformOutput', false));
xlabel('Sample'); ylabel('Estimate');
title('Noise Reduction by Averaging');

```

MATLAB

■ 16.5.2 Parameter Uncertainty

System model: $G(s) = K/(\tau s + 1)$ where K and τ are uncertain.

Monte Carlo approach: sample K and τ from their distributions, simulate system response many times, compute statistics of output.

16.6 Machine Learning / AI Connections

■ 16.6.1 Sampling and Data Collection

- ▶ Training data = sample from population
- ▶ Test data = independent sample for validation
- ▶ i.i.d. assumption underlies most ML theory

■ 16.6.2 Estimation as Foundation of ML

- ▶ Linear regression = MLE under Gaussian noise assumption
- ▶ Neural network training = MLE (or MAP) for model parameters
- ▶ Loss function minimization = maximum likelihood principle

■ 16.6.3 Probability Distributions in ML

- ▶ Generative models: learn $P(X)$ directly
- ▶ Gaussian Mixture Models: weighted sum of Gaussians
- ▶ Variational Autoencoders: latent variables with Gaussian priors

■ 16.6.4 Bias-Variance Tradeoff

$$\text{Test Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

This is exactly the MSE decomposition from estimation theory (Module 13)!

- ▶ High bias: underfitting (too simple model)
- ▶ High variance: overfitting (too complex model)
- ▶ Optimal: balance between the two

■ 16.6.5 Confidence and Uncertainty in Predictions

- ▶ Confidence intervals for model predictions
- ▶ Bayesian neural networks: posterior over weights $\rightarrow$ prediction uncertainty
- ▶ Calibration: predicted probabilities should match observed frequencies

16.7 Comprehensive MATLAB Project: Communication System Analysis

MATLAB

```

%% Complete Communication System Statistical Analysis
% Simulate BPSK over Rayleigh fading with AWGN
% Apply estimation, CI, and hypothesis testing

% System parameters
N_bits = 1e5;
EbN0_dB = 10;
EbN0 = 10^(EbN0_dB/10);

% Generate channel and signal
bits = randi([0,1], 1, N_bits);
symbols = 2*bits - 1; % BPSK: ±1

% Rayleigh fading channel
h = (randn(1,N_bits) + 1j*randn(1,N_bits)) / sqrt(2);
h_mag = abs(h);

% AWGN noise
noise_var = 1/(2*EbN0);
noise = sqrt(noise_var) * randn(1, N_bits);

% Received signal (with fading)
received = h_mag .* symbols + noise;

% Detection (coherent, known channel)
detected = sign(real(received ./ h_mag)); % Zero-forcing
errors = sum(detected ~= symbols);
BER_measured = errors / N_bits;

% --- Statistical Analysis ---
fprintf('=== Communication System Statistical Analysis ===\n');
fprintf('Eb/N0 = %d dB, N = %d bits\n\n', EbN0_dB, N_bits);

% 1. BER Estimation
fprintf('1. BER ESTIMATION\n');
fprintf('   Measured BER: %.4e (%d errors)\n', BER_measured, errors);
BER_theory_rayleigh = 0.5*(1 - sqrt(EbN0/(1+EbN0)));
fprintf('   Theory (Rayleigh): %.4e\n', BER_theory_rayleigh);

% 2. Confidence Interval
fprintf('\n2. CONFIDENCE INTERVAL (95%)\n');
margin = 1.96*sqrt(BER_measured*(1-BER_measured)/N_bits);
fprintf('   CI: [%.4e, %.4e]\n', BER_measured-margin, BER_measured+margin);
fprintf('   Theory in CI? %s\n', ...
    string(BER_theory_rayleigh >= BER_measured-margin & ...
        BER_theory_rayleigh <= BER_measured+margin));

% 3. Channel Statistics
fprintf('\n3. CHANNEL STATISTICS\n');
fprintf('   Mean |h|: %.3f (theory: %.3f)\n', mean(h_mag), sqrt(pi/4));
fprintf('   Var |h|^2: %.3f (theory: %.3f)\n', var(h_mag.^2), 1-(pi/4));

```

```
% 4. Outage Probability
SNR_threshold = 3; % dB
gamma_th = 10^(SNR_threshold/10);
P_outage_sim = mean(EbN0*h_mag.^2 < gamma_th);
P_outage_theory = 1 - exp(-gamma_th/EbN0);
fprintf('\n4. OUTAGE PROBABILITY (threshold = %d dB)\n', SNR_threshold);
fprintf('    Simulated: %.4f\n', P_outage_sim);
fprintf('    Theory:    %.4f\n', P_outage_theory);
```

16.8 Key Integration Points

Module	Application in EE
1. Probability	Event probabilities, fault diagnosis
2. Random Variables	Modeling uncertain quantities
3. Distributions	Choosing noise/fading/failure models
4. Moments	Signal power, noise power, SNR
5. Transformations	dB conversion, power from voltage
6-7. Joint/Functions	Signal+noise, system combinations
8. Conditional	Performance under different states
9. Correlation	MIMO channels, sensor arrays
10. Multiple RVs	Vector processing, beamforming
11. CLT	Why noise is Gaussian, averaging
12. Statistics	Estimating system parameters
13. Estimation	MLE for channel/noise parameters
14. Confidence	Quantifying measurement uncertainty
15. Hypothesis	Quality control, performance comparison

End of course modules. See Supplementary Materials for decision guides, MATLAB reference, and formula sheet.